Curso de reversing

Tomo I · Tronco Principal (Etapas 0 a 5) y Material de Apoyo

De los fundamentos al análisis guiado en user-mode: arquitectura x86/x64, formato PE y DLLs (ciclo de vida y sideloading), IDA, Ghidra, x64dbg (flujo táctico y extracción masiva de strings), reversing de Go y Rust, evasión en user-mode (API hooking, direct/indirect syscalls y call stack spoofing), inyección de procesos en memoria, criptografía híbrida y APIs nativas (CryptoAPI/CNG), .NET, C++ profundo, WinDbg/TTD, malware y vulnerabilidades.

Contenido de este volumen (Tomo I): Apertura y 3 Rutas de Estudio · Etapas 0, 1, 2 (2A, 2B, 2C, bloque avanzado 2E con 2.25–2.35, y optativas 2.22–2.24), 3, 4 (incluida 4.6 Protocolos de terceros) y 5 · Hilo conductor multi-nivel y Kit de 15 plantillas y laboratorios · 3 Hojas de referencia rápida (*Cheat Sheets*, con la Hoja 3.4 de desofuscación y DBI) · Glosario · Preguntas y respuestas · Fuentes y límites.

En el Tomo II y Separata: La **Etapa 6 (Kernel y drivers KMDF)** y la **Rama 2D (Windows internals y Hyper-V)** se encuentran encuadernadas en el **Tomo II**. Para consulta rápida junto al teclado, usá la **Separata de Escritorio**.

> **Material educativo y defensivo.** Las prácticas usan programas propios, software abierto o crackmes diseñados para aprender. No se enseña a eludir licencias, DRM ni controles de acceso.

Contenido — Tomo I

Apertura

- Cómo estudiar y qué se revisó — p. 3-9 · 7 págs.
- Mapa del curso y rutas de estudio — p. 10-13 · 4 págs.

Etapa 0 — **p. 14 · 1 pág.**

- 0.1 Laboratorio — p. 15-20 · 6 págs.

Etapa 1 — **p. 21-22 · 2 págs.**

- 1.1 Números y hexadecimal — p. 23-32 · 10 págs.
- 1.2 Memoria y stack — p. 33-39 · 7 págs.
- 1.3 Procesos e hilos — p. 40-45 · 6 págs.
- 1.4 Formato PE — p. 46-51 · 6 págs.
- 1.5 C y Python — p. 52-58 · 7 págs.

Etapa 2 — **p. 59-62 · 4 págs.**

- 2.1 Registros y flags — p. 63-67 · 5 págs.
- 2.2 Instrucciones — p. 68-73 · 6 págs.
- 2.3 Punto flotante y SSE — p. 74-78 · 5 págs.
- 2.4 Flujo — p. 79-85 · 7 págs.
- 2.5 Llamadas — p. 86-91 · 6 págs.
- 2.6 Debugger — p. 92-103 · 12 págs.
- 2.7 IDA — p. 104-114 · 11 págs.
- 2.8 Optimizaciones — p. 115-118 · 4 págs.
- 2.9 Herramientas — p. 119-124 · 6 págs.
- 2.10 Ofuscación nativa — p. 125-131 · 7 págs.
- 2.11 TLS callbacks — p. 132-135 · 4 págs.
- 2.12 MFC — p. 136-139 · 4 págs.
- 2.13 Huella de compilador — p. 140-149 · 10 págs.
- 2.14 PE fino — p. 150-159 · 10 págs.
- 2.15 Taxonomía anti-análisis — p. 160-169 · 10 págs.
- 2.16 Ghidra/Cutter — p. 170-174 · 5 págs.
- 2.17 Packers didácticos — p. 175-183 · 9 págs.
- 2.18 C++ profundo — p. 184-192 · 9 págs.
- 2.19 WinDbg, TTD y heap — p. 193-196 · 4 págs.
- 2.20 Patch diffing y CVE — p. 197-201 · 5 págs.
- 2.21 Scripting de debugger — p. 202-207 · 6 págs.
- 2.22 (opt) Malware dinámico — p. 208-211 · 4 págs.
- 2.23 (opt) Shellcode, red y C2 — p. 212-214 · 3 págs.
- 2.24 (opt) Fuzzing y simbólico — p. 215-219 · 5 págs.
- 2.25 Ejecución simbólica y concólica — p. 220-224 · 5 págs.
- 2.26 Ofuscación por máquina virtual — p. 225-228 · 4 págs.
- 2.27 Protectores y packers multicapa — p. 229-232 · 4 págs.
- 2.28 Neutralizar anti-análisis — p. 233-236 · 4 págs.
- 2.29 DBI: DynamoRIO y Pin — p. 237-240 · 4 págs.
- 2.30 Frida Stalker — p. 241-243 · 3 págs.
- 2.31 Extensiones de Ghidra y SLEIGH — p. 244-246 · 3 págs.
- 2.32 Go y Rust en profundidad — p. 247-251 · 5 págs.
- 2.33 VB6 y p-code — p. 252-255 · 4 págs.
- 2.34 Desvirtualización y SLEIGH — p. 256-261 · 6 págs.
- 2.35 ARM64EC, WinRT y Native AOT — p. 262-268 · 7 págs.

Etapa 3 — **p. 269-270 · 2 págs.**

- 3.1 .NET runtime — p. 271-276 · 6 págs.
- 3.2 Leer IL — p. 277-283 · 7 págs.

Etapa 4 — **p. 284-285 · 2 págs.**

- 4.1 Método de análisis — p. 286-296 · 11 págs.
- 4.2 Reconocer criptografía — p. 297-304 · 8 págs.
- 4.3 Informe — p. 305-309 · 5 págs.
- 4.4 Dinámico seguro — p. 310-314 · 5 págs.
- 4.5 Reconstruir formatos y protocolos — p. 315-319 · 5 págs.
- 4.6 Protocolos de terceros — p. 320-323 · 4 págs.

Etapa 5 — **p. 324-325 · 2 págs.**

- 5.1 Malware — p. 326-330 · 5 págs.
- 5.2 Vulnerabilidades — p. 331-339 · 9 págs.
- 5.3 Automatización — p. 340-348 · 9 págs.
- 5.4 Fuera de alcance — p. 349-352 · 4 págs.

Material de apoyo (en este Tomo I)

- Hilo conductor y plantillas C/C++ — p. 353-372 · 20 págs.
- Hojas de referencia rápida (Cheat Sheets) — p. 373-389 · 17 págs.
- Glosario — p. 390-401 · 12 págs.
- Preguntas y respuestas — p. 402-411 · 10 págs.
- Fuentes y límites — p. 412-415 · 4 págs.

En el Tomo II (Ramas Especializadas)

- Etapa 6 · Kernel y drivers en Windows (6.1 a 6.10) — ver Tomo II
- Rama 2D · Windows internals y Hyper-V (2D.1 a 2D.17) — ver Tomo II

Cómo estudiar este curso de ingeniería inversa en Windows

Esta obra estructura un recorrido completo de ingeniería inversa en Windows en español, libre, progresivo y verificable. No es un simple índice ni una lista de enlaces: cada lección explica un concepto desde cero, lo conecta con lo visto en las páginas anteriores, desarrolla ejemplos paso a paso con direcciones y valores reales, propone una práctica guiada de laboratorio y cierra con preguntas de autoevaluación.

Está diseñada tanto para quien empieza sin experiencia previa en bajo nivel como para quien busca profundizar en análisis de binarios modernos, siempre dentro de un marco **educativo, defensivo y autorizado**. Todas las prácticas se realizan sobre programas propios compilados por vos, software abierto o retos didácticos (*crackmes*) creados específicamente para aprender.

> **Empezá por acá**
>
> 1. Leé esta portada completa (10 minutos).
> 2. Abrí el Mapa del curso y elegí la Etapa 0.
> 3. Armá el laboratorio de la lección 0.1.
> 4. Estudiá una lección por vez y hacé la práctica antes de avanzar.
> 5. Anotá tu avance en seguimiento.csv.

Qué vas a poder hacer

Al terminar el recorrido base vas a poder:

- Explicar cómo se representa un programa en memoria y en disco.
- Leer assembly x86/x64 y reconstruir condiciones, bucles y llamadas.
- Usar x64dbg e IDA (o Ghidra) con un método, no a los botones.
- Analizar un programa .NET propio y comparar fuente, IL y decompilación.
- Escribir un informe reproducible con evidencia, límites y hashes.
- Decidir cuándo un programa se ejecuta y cuándo no.
- Reconocer código ofuscado, TLS callbacks, binarios MFC, la huella del compilador y técnicas anti-análisis, y leer el mismo binario en Ghidra (bloque **2B · Windows intermedio**).

El curso: siete etapas numeradas y una rama 2D

Las **Etapas 0–4** forman el recorrido obligatorio. La **Etapa 5**, la **Etapa 6** y la rama optativa **2D · Windows internals** (asociada a la Etapa 2, después de 2C) no agregan requisitos para completar el curso base.

Etapa	Foco	Horas estimadas	Puerta de salida
0	Laboratorio y método seguro	4–6 h	VM aislada, snapshot, hash y bitácora
1	Fundamentos	21–25 h	Números, memoria, PE y un programa propio en C
2	Introducción al reversing	32–40 h (2A) + ~25 h (2B) + ~30–40 h (2C) + ~25–30 h (2E)	Assembly, debugger e IDA; bloques **2B intermedio**, **2C avanzado** y **2E desofuscación/DBI** (+ 3 rutas optativas)
3	.NET como primera victoria	7–10 h	Leer IL y comparar tres vistas
4	Práctica guiada	19–25 h	Método, formatos/protocolos, cripto, informe y dinámico seguro (6 lecciones: 5 base + 4.6)
5	Ramas opcionales	variable	Una rama con práctica propia y límites claros
6	Kernel y drivers (rama optativa)	45–65 h estimadas	Revisar un driver propio y sus controles en VM aislada

La profundización **Etapa 2 · Rama optativa 2D — Windows internals** tiene 13 lecciones (estimación editorial 55–70 h). Se muestra junto a la Etapa 2, pero queda fuera de sus requisitos obligatorios. La extensión Hyper-V también es optativa y separada.

Las horas son **estimaciones editoriales**, no mediciones de alumnos ni duraciones rígidas de laboratorio. Las de 2B/2C se estiman por bloque; 2D/Etapa 6 pueden requerir más tiempo según las prácticas y experiencia previa.

> **Cómo se cuentan las lecciones y en qué orden cursar 2.22–2.24**
>
> La **Etapa 2** reúne 2A (2.1–2.9), 2B (2.10–2.16) y 2C (2.17–2.21), además de 3 rutas optativas (2.22–2.24) y el Bloque 2E avanzado (2.25–2.33). Como las optativas 2.22–2.24 declaran como prerrequisitos las lecciones 4.1, 4.2, 4.4 y 5.1, se recomienda cursarlas después de completar la Etapa 4 y la lección 5.1, manteniendo intacta la numeración del curso. La Etapa 4 comprende **5 lecciones base** (4.1 a 4.5; 19–25 h) más la ampliación **4.6** de protocolos de terceros. La rama optativa 2D aporta 13 lecciones fuera de las horas obligatorias y la Etapa 6 suma 6 lecciones optativas. El apartado 5.4 «Fuera de alcance» es una página de límites, no una lección a cursar.

Cómo estudiar

Cada lección tiene la misma estructura:

1. **Objetivo** y prerrequisitos.
2. **Explicación** con lenguaje sencillo y conexión con lo anterior.
3. **Ejemplo paso a paso**, con valores y direcciones cuando el material los aporta.
4. **Práctica** en el laboratorio.
5. **Preguntas de repaso** con respuestas razonadas.
6. **Puerta de salida**: qué tenés que poder hacer antes de avanzar.

7. **Fuentes**: rutas locales, capítulos y timestamps.

La regla es simple: **no avances si no podés hacer la práctica**. Leer sin ejecutar deja la sensación de entender y no la comprensión.

Bases documentales y fuentes del curso

Esta obra combina el rescate y sistematización de materiales didácticos clásicos en español con una amplia labor de investigación, verificación directa en laboratorio y redacción técnica independiente basada en estándares modernos de la industria (Microsoft Learn, Intel SDM, Ghidra, angr, Frida, Z3, LLVM y Windows Driver Kit):

Fuente o colección	Contenido	Integración en el curso
Reversing desde cero	47 capítulos en video (28,6 h)	42 capítulos integrados con trazas paso a paso en las Etapas 1, 2A y 2B (se excluyen 5 capítulos sobre protectores comerciales por criterio ético y defensivo).
Videos y material adicional	40 sesiones (22,5 h, MASM32 + OllyDbg)	Integrados como base histórica de ensamblador de 32 bits, pila, llamadas y transición hacia x64dbg.
Curso IDA desde cero	69 tutoriales ilustrados (3.278 capturas)	Integrados en las lecciones de análisis estático, estructuras, control de flujo, depuración e IDAPython.
Lógica y fundamentos (Python ISO)	55 sesiones de algoritmia (12,7 h)	Aprovechados para la base lógica (secuencia, decisión, bucles y arreglos) y complementados con Python 3 moderno (`bytes`, `struct`, `pefile`, `capstone`).
Documentación primaria y estándares	Microsoft Learn, Intel SDM, RFCs y herramientas abiertas	Base de los módulos de 64 bits, SSE/AVX, C++ profundo, .NET moderno, criptografía (CNG/CryptoAPI), DBI, análisis simbólico, Windows Internals (2D), Kernel/KMDF (Etapa 6) y Tomo III.

El detalle documental completo se encuentra en registro_de_cobertura.csv, mapa_de_cobertura.csv y en el capítulo final Fuentes y límites.

> **Criterio de higiene y seguridad documental**
>
> - Todas las prácticas del curso se basan en **código fuente propio compilable** (incluido en el kit de plantillas C/C++) o en ejercicios controlados dentro de una máquina virtual aislada.
> - Por seguridad, nunca ejecutes binarios de origen desconocido, archivos comprimidos sin verificar ni herramientas descargadas fuera de sus repositorios oficiales.
> - Cuando un material clásico de audio o video presenta una imprecisión técnica o refleja el comportamiento de versiones antiguas de Windows, la lección lo aclara explícitamente y muestra el comportamiento verificado en sistemas actuales de 32 y 64 bits.

Cómo se organiza cada lección

A lo largo de los tomos vas a encontrar cuatro tipos de bloques claramente diferenciados:

Capa	Qué aporta a tu estudio
Núcleo conceptual	El fundamento técnico central explicado paso a paso con desensamblado, estado de registros/memoria y código reproducible.
Síntesis conceptual	Tablas comparativas, mapas de estructuras y esquemas de arquitectura para consulta rápida.
Conocimiento externo verificado	Especificaciones oficiales de Microsoft Learn, Intel SDM y documentación de herramientas modernas (IDA, Ghidra, WinDbg, Frida).
Recomendación editorial	Buenas prácticas de laboratorio, advertencias sobre errores frecuentes de principiantes y límites éticos/defensivos.

Qué hay en el paquete

Archivo o carpeta	Para qué
guia/	Las lecciones HTML, etapa por etapa
Materiales	Hub: un solo lugar con enlaces a todo
Buscar	Buscador offline por palabra en todo el curso
Banco de prácticas	Crackmes del corpus para practicar en VM, con hash y mapeo al curso
Índice de videotutoriales	Dónde está en Windows cada video mencionado, con enlace y ruta
Curso IDA · índice por tema	Los 69 tutes de IDA, con tema y cantidad de palabras
Teorias_Numeradas · educativo vs. cracking	Los 2.023 contenedores (2.017 con título legible) clasificados por título
Telegram por tema	523 transcripciones tamizadas; dónde están las sesiones de práctica
fichas/	Qué es cada recurso, qué se revisó y qué falta
hilo-conductor-y-plantillas.html	Evolución de un mismo binario en los 3 niveles (`check.c` → `BlocLab`) y kit de 7 plantillas C/C++ en `plantillas_codigo/`
hojas-referencia-rapida.html	Hojas de referencia rápida (*Cheat Sheets*) para Principiante, Intermedio y Avanzado
glosario.html	Términos en lenguaje de principiante
preguntas_y_respuestas.html	Repaso por etapa, con respuestas
mapa_de_cobertura.csv	Tema ↔ recurso ↔ archivo/capítulo ↔ estado
registro_de_cobertura.csv	Lotes, archivos, revisados, pendientes y motivo
seguimiento.csv	Tu planilla de avance
Fuentes y límites	Criterios, alcance y limitaciones
Curso_Reversing_Windows.pdf	Edición de estudio completa consolidada (PDF) para leer sin pantalla; las prácticas y los materiales están acá en el HTML
orden_impresion.html	Especificación y orden técnica de imprenta (Tomo I Partes A y B, Tomo II y Separata de Escritorio)
plantilla_informe.md	Base para tus informes

Todo funciona **offline**. Las fuentes locales se enlazan con rutas relativas; los enlaces a transcripciones, videos y documentos abren el archivo original. El paquete trae HTML, CSV, el PDF y las **imágenes de los tutes ilustrados y los fotogramas de las juntadas** (~130 MB). Los **videos y audios** (decenas de GB) **se vinculan, no se copian**: para que abran, mantené las carpetas del proyecto una al lado de la otra (o usá los ZIP de compartir).

Compartir el curso

Los enlaces locales son **relativos** y funcionan si se conserva la estructura del paquete. Para compartir, prepará una copia curada con los archivos que tengas derecho a redistribuir; no compartas la carpeta de proyecto completa ni exportaciones de chats, corpus sin revisar o medios de terceros. La edición pública de presentación contiene el PDF de teoría; el paquete local conserva el HTML y los tutoriales de apoyo. Más detalle en el Índice de videotutoriales.

Requisitos

- Una computadora con **Windows**. Esta edición cubre **solo reversing de Windows** (binarios PE, x86/x64); Linux, macOS, Android y ARM quedan fuera, aunque los fundamentos se trasladen.
- VirtualBox, VMware o Hyper-V para el laboratorio.
- x64dbg, un editor hexadecimal y un visor de PE (gratuitos).
- IDA Free o Ghidra. **No** hace falta IDA Pro.
- Ganas de escribir programas propios: son el mejor material de práctica.

Seguridad y legalidad

Leé esto antes de ejecutar cualquier cosa

- No ejecutes adjuntos del chat, cracks, keygens, instaladores desconocidos ni archivos en cuarentena.
- No analices malware en tu máquina personal. La rama de malware de esta guía es **estática y defensiva**.
- No uses las técnicas para eludir licencias, DRM, autenticación o controles de acceso.
- No analices software ajeno sin permiso. Los ejercicios de vulnerabilidades se hacen en programas propios o laboratorios diseñados para eso.
- Si encontrás una vulnerabilidad, reportala en privado y no publiques el exploit.

Alcance temático y organización de la obra

Esta obra está organizada en **tres tomos principales** más una **Separata de Escritorio** de consulta rápida, con un alcance claramente delimitado:

- **Tomo I · Tronco Principal y Material de Apoyo (Parte A y Parte B):** cubre las Etapas 0 a 5 completas (laboratorio, fundamentos binarios y PE, ensamblador x86/x64, punto flotante SSE, x64dbg, IDA, Ghidra, optimizaciones, ofuscación, TLS, MFC, huella de compilador, anti-análisis, packers didácticos, C++ profundo, WinDbg/TTD, diffing, scripting, ejecución simbólica, desvirtualización, DBI, Frida Stalker, SLEIGH, Go/Rust, VB6, ARM64EC/WinRT, .NET, criptografía, formatos/protocolos, malware defensivo y vulnerabilidades), junto con el Hilo conductor C/C++, el Glosario y el Repaso.

- **Tomo II · Kernel, Drivers y Windows Internals:** reúne la Etapa 6 completa (arquitectura del kernel de Windows, drivers KMDF, IOCTLs, depuración con WinDbg/KDNET, minifilters, mitigaciones VBS/HVCI, callbacks, firmas WHQL/ELAM y BYOVD) y la Rama 2D (objetos, memoria/VAD, Registro, servicios, Task Scheduler, COM/DCOM/RPC, WMI, IPC, excepciones, ETW, Win32k, arranque UEFI, AMSI y LSASS/DPAPI).
- **Tomo III · Fronteras Modernas:** aborda LLVM IR y pases de ofuscación/desofuscación, ingeniería inversa profunda de Rust y Go, contenedores Windows/Docker/WSL2 y asistencia al reversing con modelos de lenguaje (IA/MCP) y verificación formal con Z3.
- **Límites éticos y de plataforma:** el curso se enfoca en **Windows** (binarios PE en x86, x64 y ARM64EC) con un criterio estrictamente educativo y defensivo; por ello se excluye cualquier práctica orientada a vulnerar licencias, DRM o protectores comerciales sobre software de terceros sin autorización.

Atribución

Esta obra es un manual libre y abierto de ingeniería inversa. El material didáctico principal es de **Ricardo Narvaja y Solid** («Reversing desde cero»), **Cayetano de Juan Úbeda** (MASM32 y OllyDbg) y **Daniel Bermúdez** (lógica de programación). Las fichas indican autoría, ruta y límites de cada recurso. Esta obra es un manual técnico independiente, libre y abierto con fines educativos y de investigación defensiva. La redacción, el desarrollo de código didáctico y las ampliaciones técnicas son de autoría libre y anónima.

> **Empezá ahora**
>
> Etapa 0 · Laboratorio · Mapa del curso · Fichas · Glosario

Mapa del curso

El recorrido obligatorio son las Etapas 0–4. Las Etapas 5–6 y la rama 2D son optativas y no agregan requisitos para completar el curso base. Cada rama tiene prerrequisitos y una puerta de salida propios.

Etapa	Foco	Horas estimadas	Lecciones
0	Laboratorio y método seguro	4 a 6 horas	1
1	Fundamentos	21 a 25 horas	5
2	Introducción al reversing	32 a 40 h (2A) + ~25 h (2B) + ~30–40 h (2C) + ~25–30 h (2E)	33
3	.NET como primera victoria	7 a 10 horas	2
4	Práctica guiada	19 a 25 horas	6
5	Ramas opcionales	Variable: 5 a 10 horas por rama	4
6	Rama optativa · Kernel y drivers	45–65 horas como estimación inicial; puede requerir más tiempo si C, WinDbg o WDK son nuevos.	6

Cómo estudiar

1. Leé la lección completa, sin saltear los ejemplos.
2. Hacé la práctica en el laboratorio; no solo leas.
3. Respondé las preguntas **antes** de abrir las respuestas.
4. Verificá la puerta de salida. Si no la pasás, repetí la práctica.
5. Anotá en `seguimiento.csv` qué hiciste, cuánto tardaste y qué duda quedó.

Tres perfiles o rutas de estudio (Principiante, Intermedio y Avanzado)

Aunque el curso completo supera las 400 páginas al sumar el tronco obligatorio (Etapas 0–4), los bloques intermedio y avanzado de la Etapa 2 (2B y 2C), la rama de Windows internals (2D) y la de Kernel y drivers (Etapa 6), **no hace falta leerlo de corrido de principio a fin en una sola sentada**. Podés recorrerlo en tres hitos progresivos claramente delimitados según tu nivel actual y tu objetivo:

Perfil y nivel	Carga estimada	Secuencia exacta de etapas y lecciones	Herramientas centrales	Hito práctico al cerrar el nivel
Ruta 1 · Fundamentos Firmes *(Nivel Principiante)*	~65 a 75 h	Etapa 0 (0.1) → Etapa 1 (1.1–1.5) → Etapa 2A (2.1–2.9) → Etapa 3 (3.1–3.2) → Etapa 4 base (4.1, 4.2, 4.3 y 4.4)	Visor PE (PE-bear / DIE), editor hex, IDA Free, x64dbg, ILSpy / dnSpyEx, compilador C y Python	Analizar un binario propio en C y uno en .NET de punta a punta sin ver el fuente, confirmar hipótesis con breakpoints y redactar un informe reproducible.
Ruta 2 · Analista de Binarios y Malware Defensivo *(Nivel Intermedio)*	+45 a 55 h *(acumulado ~115–130 h)*	Bloque **2B** (2.10–2.16) → 2.17 (Packers) → 2.21 (Scripting y Frida) → 4.5 (Formatos y protocolos) → 5.1 (Malware estático) y 5.3 (Automatización) → Optativas 2.22, 2.23 y 2D.1–2D.5	Ghidra, Scylla, trazas condicionales de x64dbg, Frida, IDAPython (`ida_typeinf`), `pefile`, `capstone`, YARA y Sysinternals / WinDbg básico	Desempaquetar un binario UPX propio (x86/x64) hasta el OEP, recuperar strings ocultas y APIs por PEB/hash, reconstruir un formato binario propio (`CLSF`) y escribir reglas YARA e informes de persistencia.
Ruta 3 · Investigador de Vulnerabilidades, Internals y Kernel *(Nivel Avanzado)*	+80 a 110 h *(curso completo)*	Núcleo duro de **2C** (2.18 C++ profundo, 2.19 WinDbg/TTD/Heap, 2.20 Patch diffing) → 5.2 (Vulnerabilidades) → 2.24 (Fuzzing y simbólico) → 2D.6–2D.17 (COM/RPC/WMI/IPC/Excepciones/ETW) → Etapa 6 (6.1–6.10, Kernel y drivers)	WinDbg Preview + Time-Travel Debugging (TTD), GFlags / Page Heap, BinDiff / Diaphora / Version Tracking, OleViewDotNet, WDK / KMDF y Driver Verifier en VM descartable	Reconstruir clases C++ (vtable, RTTI, SSO, excepciones x64), diagnosticar corrupciones de heap con Page Heap/TTD, aislar un parche de seguridad con diffing, auditar canales IPC/COM y revisar un driver `.sys` estática y dinámicamente.

Consejo para elegir tu punto de entrada

Si empezás desde cero, completá primero la **Ruta 1** sin desviarte a 2B ni a 2C cuando llegues al final de 2.9: saltar de 2.9 directo a la Etapa 3 (.NET) y a la Etapa 4 base te da una victoria real y consolida el método en la mitad de tiempo. Una vez aprobada la puerta de salida de la Etapa 4, volvé para cursar la **Ruta 2** y luego la **Ruta 3**. En cualquier nivel, acompañate con el Hilo conductor multi-nivel y kit de plantillas C/C++ y las Hojas de referencia rápida (Cheat Sheets).

Qué es cada etapa

Etapa 0 · Laboratorio y método seguro

Armar un entorno aislado, tomar snapshots, verificar hashes y registrar cada práctica con su alcance.

Prerrequisitos: Ninguno.

Etapa 1 · Fundamentos

Leer un binario sencillo: números, memoria, procesos, formato PE y programas propios en C y Python.

Prerrequisitos: Etapa 0.

Etapa 2 · Introducción al reversing

Leer assembly x86/x64, seguir el flujo y usar debugger y desensamblador con criterio. Se divide en 2A (base, 2.1–2.9), 2B (Windows intermedio, 2.10–2.16: ofuscación, TLS, MFC, huella, PE fino, anti-análisis, Ghidra) y 2C (Windows avanzado, 2.17+: packers didácticos, C++ profundo, WinDbg/heap, patch diffing, scripting) y 2E (desofuscación y análisis dinámico avanzado, 2.25+: ejecución simbólica, ofuscación por VM, protectores multicapa, neutralizar anti-análisis, DBI con DynamoRIO/Pin y Frida Stalker, extensiones de Ghidra/SLEIGH, y ampliaciones de Go/Rust y VB6). Las rutas de malware dinámico, shellcode/C2 y fuzzing quedan como optativas, fuera de la puerta obligatoria.

Prerrequisitos: Etapa 1.

Etapa 3 · .NET como primera victoria

Entender el runtime .NET, leer IL y comparar fuente, IL y decompilación en un programa propio.

Prerrequisitos: Etapa 2.

Etapa 4 · Práctica guiada

Analizar programas propios con método, reconstruir un formato o protocolo desde muestras, contener la ejecución y escribir un informe reproducible.

Prerrequisitos: Etapas 0 a 3.

Etapa 5 · Ramas opcionales

Elegir una especialización dentro de Windows, con prerrequisitos y práctica propia, sin convertir el corpus en una lista de tareas peligrosas.

Prerrequisitos: Etapas 0 a 4 completas.

Etapa 6 · Rama optativa · Kernel y drivers

Comprender, construir y revisar un driver KMDF didáctico con una interfaz inocua, depurarlo solo en VM descartable y explicar mitigaciones actuales.

Prerrequisitos: Etapa 0 (laboratorio aislado), C, assembly, memoria y sincronización, WinDbg y 2D.1 (objetos/tokens), 2D.2 (memoria/loader) y 2D.11 (excepciones/debugging). Etapa 2C es recomendada, no requisito; no hace falta completar toda 2D.

Material de apoyo

- Índice de videotutoriales: dónde está en Windows cada video que menciona el curso, con enlace y ruta absoluta.
- Curso IDA desde cero (69 tutoriales): índice por tema, con el texto extraído de cada uno.
- Teorias_Numeradas clasificadas: los 2.017 títulos separados en educativos vs cracking de software comercial.
- Transcripciones de Telegram por tema: las 523 transcripciones tamizadas; dónde están las sesiones de práctica.
- Fichas de recursos: qué se revisó, qué falta y qué límites tiene cada fuente.
- Glosario: términos en lenguaje de principiante.
- Preguntas y respuestas: repaso por etapa.
- Fuentes y límites: criterios, cobertura y exclusiones.

Dónde seguir practicando (legal y seguro)

Esta guía practica con **programas propios**, pero conviene seguir con retos pensados para aprender. Estos recursos son **recomendación externa** (no salen del corpus tradicional) y permiten practicar reversing de Windows sin problemas legales:

- crackmes.one: miles de crackmes subidos para aprender; filtrá por plataforma **Windows** y por dificultad. Analizalos en tu VM.
- Flare-On Challenge: competencia anual y gratuita de reversing, centrada en Windows; los retos de años anteriores quedan publicados con soluciones.
- **CTFs (categoría rev)**: CTFtime lista competencias; muchos retos de reversing son binarios de Windows. Buenísimos para practicar con límite de tiempo.
- **Talleres abiertos**: materiales públicos de análisis de malware y reversing (por ejemplo, OALabs o Malware Unicorn) con binarios de práctica.
- **Tus propios programas**: seguí compilando ejemplos con condiciones cada vez más complejas; es la práctica más segura y la que más enseña.

Regla que no cambia: todo en la VM aislada de la Etapa 0, y nada de software ajeno sin permiso.

Etapa 0 · Laboratorio y método seguro

> **Objetivo medible**
>
> Armar un entorno aislado, tomar snapshots, verificar hashes y registrar cada práctica con su alcance.

Prerrequisitos: Ninguno. **Duración estimada:** 4 a 6 horas. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración rígida de laboratorio.

Lecciones

#	Lección	Qué aprendés
0.1	Un laboratorio que podés romper sin miedo	VM, snapshots, red aislada, transferencia por ISO y hashes.

Recursos y lecturas de referencia

- Inventario de recursos de reversing
- Transcripciones de Telegram

Evaluación de salida

- Mostrar una VM con snapshot limpia y red aislada.
- Transferir un archivo sin carpeta compartida y verificar su SHA-256 en host y guest.
- Escribir una bitácora con snapshot, programa, hash y observaciones.
- Explicar qué harías si el programa deja el sistema en un estado raro.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Usar carpeta compartida "solo para pasar el archivo".
- Tomar la snapshot después de copiar la muestra.
- Confundir una snapshot con un backup de los archivos del host.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Un laboratorio que podés romper sin miedo

Objetivo de la lección

Al terminar vas a poder armar una máquina virtual aislada, tomar una snapshot, transferir un programa de práctica sin compartir carpetas y registrar su hash antes y después de analizarlo. Ese es el piso de todo lo que sigue.

Prerrequisitos: ninguno. **Duración sugerida:** 4 a 6 horas, casi todas de trabajo práctico. Es una estimación aproximada de esta guía, no una duración rígida de laboratorio.

Por qué el laboratorio va primero

Reversing significa ejecutar y observar programas que no escribiste. A veces son programas de práctica inofensivos; otras veces son binarios de origen desconocido. La diferencia entre aprender con tranquilidad y arruinar tu computadora personal está en **dónde** los ejecutás.

Un laboratorio de reversing tiene una regla de oro: **lo que se rompe, se descarta; lo que importa, vive afuera**. El programa bajo análisis corre dentro de una **máquina virtual (VM)** —una computadora simulada por software dentro de tu propia PC— que podés congelar, borrar y restaurar en segundos. Tus documentos, tus cuentas, tus claves y tus carpetas personales se quedan en tu sistema principal, completamente fuera de alcance.

Los cursos clásicos de reversing parten de esta idea desde el primer día: al abrir un ejecutable de práctica en un depurador como **x64dbg**, lo que hacés es observar y controlar el comportamiento del programa **mientras está corriendo en memoria** (`Reversing desde cero, cap. 01`, 00:00:26). Como para observar en dinámico hay que ejecutar, aislar el entorno es el primer paso obligatorio.

Tres conceptos clave si empezás desde cero

- **Máquina Virtual (VM):** Es una PC completa (con su propio disco virtual, memoria RAM y placa de red) que corre dentro de una ventana en tu computadora real. Si algo se rompe adentro de la VM, tu computadora real no sufre ningún daño.
- **Snapshot (instantánea):** Es una "fotografía exacta" del estado de la VM en un instante dado. Si después de probar un programa querés dejar todo limpio como recién instalado, restaurás la snapshot en diez segundos.
- **Hash (SHA-256):** Es la "huella digital" matemática de un archivo (una cadena de 64 caracteres hexadecimales). Si un solo byte del archivo cambia, el hash cambia por completo; sirve para verificar que el archivo que copiaste a la VM es exactamente el mismo que tenías afuera.

Lo que este curso no hace

No vas a ejecutar adjuntos del chat, cracks, keygens, instaladores desconocidos ni muestras en cuarentena. Tampoco vas a analizar malware dinámicamente en tu máquina personal. Las prácticas usan **programas propios**, software abierto o crackmes diseñados para aprender, y siempre dentro de la VM.

Las tres capas: host, hipervisor e invitado

Conviene fijar tres palabras que vas a ver todo el tiempo:

Capa	Qué es	Ejemplos	Qué va adentro
Host (anfitrión)	Tu computadora real	Windows 11, tu notebook	Tus documentos, cuentas, claves
Hipervisor	El programa que crea y administra máquinas virtuales	VirtualBox, VMware Workstation Player, Hyper-V	La configuración de las VMs
Guest (invitado)	El sistema operativo que corre dentro de la VM	Windows 10/11, Linux	Las herramientas de análisis y las muestras

El guest es descartable. El host no. Todo lo que hagas para "compartir" entre ambos —carpetas compartidas, portapapeles bidireccional, unidades de red, USB— es un puente, y los puentes se cruzan en las dos direcciones.

El error más común

Crear la VM, activar "carpeta compartida" para pasar el programa, ejecutarlo y descubrir que el malware cifró la carpeta del host. Si necesitás pasar un archivo, usá una imagen ISO de solo lectura o una unidad extraíble virtual que montás, copiás y desmontás. Nunca una carpeta compartida permanente.

Reglas de diseño del laboratorio

Estas siete reglas resumen las buenas prácticas estándar de un laboratorio de análisis seguro:

1. **Una VM por objetivo.** Si analizás dos programas distintos, usá dos snapshots o dos VMs. Así sabés qué cambió cada uno.
2. **Snapshot limpia antes de empezar.** La snapshot se toma con el sistema actualizado, sin la muestra y sin conexión a tu red personal. Ese es tu "punto cero".
3. **Red Host-only o NAT sin acceso a la LAN.** La VM no debe ver tus otras computadoras ni tus unidades de red. Si el programa necesita internet, evaluá si de verdad lo necesita; para un crackme, no.
4. **Sin credenciales reales.** Dentro de la VM usá una cuenta local descartable, sin tus contraseñas, sin tu correo, sin tus tokens.

5. **Sin carpetas compartidas ni portapapeles bidireccional.** Si el hipervisor lo permite, desactivalos explícitamente.
6. **Herramientas instaladas desde el inicio.** x64dbg, un editor hexadecimal, Python, un desensamblador. Instalarlas después de la muestra contamina la línea base.
7. **Registro desde el minuto cero.** Anotá qué VM, qué snapshot, qué archivo, qué hash y qué hiciste.

Ejemplo paso a paso: laboratorio mínimo con VirtualBox

Mostramos el procedimiento con VirtualBox porque es gratuito y fácil de configurar. Los mismos conceptos se trasladan exactamente igual a VMware Workstation o Hyper-V; solo cambian los nombres de los menús.

Paso 1. Crear la VM

1. Descargá VirtualBox desde el sitio oficial y una imagen ISO de Windows 10/11 desde Microsoft. **No** uses imágenes "preactivadas" ni ISOs de origen dudoso: son un riesgo y además complican la trazabilidad.
2. Creá una VM nueva: 4 GB de RAM, 2 CPUs y un disco virtual de 60 GB. Para reversing de programas de práctica alcanza.
3. Instalá el sistema operativo. Cuando termine, aplicá las actualizaciones y reiniciá.

Por qué: una VM con recursos suficientes evita que el debugger se trabe y que confundas un problema de rendimiento con un comportamiento del programa.

Paso 2. Aislar la red

En la configuración de la VM, revisá la pestaña **Red**:

- Adaptador 1: **NAT** o **Host-only**. NAT permite salir a internet aislando tu red; Host-only solo permite comunicación directa con el host.
- Desactivá el adaptador puente (*Bridged*): ese conecta la VM directamente como un equipo más dentro de tu red local (LAN).
- En **General > Avanzado**, desactivá el portapapeles compartido y el arrastrar y soltar.
- En **Carpetas compartidas**, no agregues ninguna.

Por qué: el adaptador puente es el que convierte una VM en otro dispositivo de tu red. Para analizar un crackme no lo necesitás.

Paso 3. Tomar la snapshot "limpia"

1. Apagá la VM correctamente (no la suspendas).
2. En VirtualBox, seleccioná la VM y usá **Instantáneas > Tomar instantánea**.
3. Nombrala `00-limpia-sin-muestra` y escribí una descripción con la fecha.

Por qué: una snapshot guarda el estado del disco. Si algo sale mal, restaurás y volvés exactamente al punto anterior. No es un backup de tus archivos del host; es el estado de la VM.

Paso 4. Preparar la transferencia sin puentes

Para pasar un programa de práctica:

1. Creá una imagen ISO con el archivo. En Windows podés usar `oscdimg` o una herramienta gráfica; en Linux, `genisoimage`.
2. Montá la ISO en la VM como unidad de CD/DVD, copiá el archivo al disco del guest y **desmontá** la ISO.
3. Verificá el hash dentro del guest y comparalo con el del host.

En PowerShell, el comando es:

```
Get-FileHash -Algorithm SHA256 .\programa.exe
```

Salida típica (el hash es inventado a modo de ejemplo):

```
Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          A1B2C3D4E5F60718293A4B5C6D7E8F90123456789ABCDEF0123456789ABCDEF0
C:\lab\programa.exe
```

Por qué el hash: dos archivos con el mismo SHA-256 son, en la práctica, el mismo archivo. Si el hash del host y el del guest coinciden, sabés que la transferencia no alteró nada. Si no coinciden, no analices: primero entendé por qué cambió.

Paso 5. Ejecutar y volver atrás

1. Confirmá que estás en la snapshot `00-limpia-sin-muestra`.
2. Anotá en tu bitácora: fecha, nombre del archivo, hash, herramienta y objetivo.
3. Ejecutá el programa **solo si es propio, abierto o un crackme de práctica**.
4. Al terminar, apagá la VM y restaurá la snapshot. No reutilices el estado "sucio" para la próxima muestra.

Por qué: restaurar no es opcional. Un programa puede dejar archivos, claves de registro o servicios corriendo. La snapshot te devuelve al punto conocido.

Cuándo observar sin ejecutar (triage estático)

En la literatura clásica suele mostrarse un laboratorio con dos máquinas virtuales conectadas entre sí: por ejemplo, en el video 37 de «Videos y material adicional» se observa una muestra (*Shelter*) dentro de una VM con Windows 7 de 32 bits y otra VM con Kali Linux para simular la red (`Video 37, introducción`, 00:00:00–00:00:40).

La separación en máquinas virtuales es correcta como principio de contención, pero tené presente una regla fundamental: **ante un archivo desconocido o sospechoso, nunca se empieza ejecutándolo**. Primero se realiza el análisis estático (inspección de cabeceras PE, secciones, cadenas, imports y hashes sin correr el binario), como verás en detalle en las Etapas 1, 2 y 5. En este curso solo ejecutamos programas propios compilados por nosotros mismos o retos didácticos verificados dentro de la VM aislada.

Práctica: tu laboratorio en una página

Hacé esto y guardá el resultado. Es el primer entregable del curso.

1. Creá la VM con las reglas de arriba.
2. Tomá la snapshot limpia.
3. Escribí tu propio programa de prueba. El más simple: un `.exe` que pida una palabra y muestre "hola" si es `clase`. Compilalo en el guest (o en el host y pasalo por ISO).
4. Calculá el SHA-256 en el host y en el guest. Pegá ambos en la bitácora.
5. Ejecutalo en x64dbg si ya lo instalaste; si no, ejecutalo suelto y anotá qué pasa.
6. Restaurá la snapshot y verificá que el archivo que copiaste ya no está.

> **Entregable mínimo**
>
> Un archivo `bitacora.md` con: nombre de la VM, snapshot usada, programa, hash del host, hash del guest, red configurada, qué hiciste y qué observaste. Sin ese registro, la práctica no cuenta como terminada.

Preguntas de repaso

1. ¿Qué diferencia hay entre una snapshot y un backup? Una snapshot guarda el estado de la VM en un momento; sirve para volver atrás después de un experimento. Un backup es una copia de tus datos para recuperarlos si se pierden. La snapshot no protege tus archivos del host y no reemplaza un backup.

2. Si encontrás una herramienta o ejecutable descargado de un foro y el nombre dice "seguro", ¿alcanza para ejecutarlo en tu host? No. El nombre de un archivo no garantiza su contenido ni su integridad. Toda muestra o práctica se analiza dentro de la VM aislada tras registrar su hash SHA-256, y las herramientas del taller se descargan únicamente desde sus sitios o repositorios oficiales.

3. ¿Por qué se desactiva la carpeta compartida? Porque es un canal bidireccional entre guest y host. Un programa con comportamiento malicioso puede leer, cifrar o borrar archivos del host a través de ese canal. La ISO de solo lectura evita esa vía.

4. ¿Para qué sirve comparar el hash antes y después de copiar? Para detectar alteraciones en la transferencia y para identificar el archivo sin ambigüedad. Si los hashes no coinciden, el archivo que analizás no es el que creías.

5. ¿La VM alcanza para analizar cualquier muestra? No. Una VM mal configurada puede tener acceso a la red local, al portapapeles o a carpetas compartidas. Además, algunas muestras detectan la VM y cambian de comportamiento. Para empezar, alcanza con programas propios y crackmes; para malware real hace falta un laboratorio dedicado y procedimientos que esta guía no cubre en la Etapa 0.

Antes de avanzar

Tenés que poder:

- Explicar con tus palabras la diferencia entre host, hipervisor y guest.

- Crear una VM, aislarla de la red local y tomar una snapshot.
- Transferir un archivo sin carpeta compartida y verificar su SHA-256 en ambos lados.
- Describir qué harías si un programa deja el sistema raro: apagar, restaurar, revisar la bitácora.

> **Puerta de salida de la Etapa 0**
>
> Si no podés mostrar tu `bitacora.md` con los dos hashes y la snapshot, no pases a la lección siguiente. El resto del curso asume que tenés un entorno donde equivocarte no cuesta caro.

Fuentes de esta lección

- `Reversing desde cero, cap. 01 (x64dbg primeros pasos)` — qué hace un debugger y por qué se ejecuta el programa.
- `Videos y material adicional, Video 37` — ejemplo observado de laboratorio con dos VMs y ejecución de malware; se cita como caso, no como modelo.
- `Inventario limpio, LEEME` — criterio de no ejecutar ni extraer archivos durante el inventario.
- `Fuentes y límites de esta edición` — criterios de alcance y seguridad.

> **Síntesis de estudio de esta lección**
>
> **Núcleo conceptual:** el uso de VMs en el video 37 y el criterio de no ejecución del inventario. **Conocimiento externo:** funcionamiento de hipervisores, snapshots y hashes. **Recomendación editorial:** las siete reglas de diseño y el procedimiento paso a paso.

Etapa 1 · Fundamentos

> **Objetivo medible**
>
> Leer un binario sencillo: números, memoria, procesos, formato PE y programas propios en C y Python.

Prerrequisitos: Etapa 0. **Duración estimada:** 21 a 25 horas. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración rígida de laboratorio.

Lecciones

#	Lección	Qué aprendés
1.1	Bits, bytes, hexadecimal y números con signo	Binario, hex, complemento a dos, little-endian y VA→offset.
1.2	Memoria: stack, heap y marcos de función	Stack, prólogo/epílogo, argumentos y shadow space en x64.
1.3	Procesos, hilos y mapa de memoria	Procesos, hilos, handles, mapa de memoria y permisos.
1.4	El formato PE: cabeceras, secciones e imports	Cabecera PE, secciones, IAT y ASLR.
1.5	C y Python mínimos para analizar	Programas propios en C y herramientas en Python.

Recursos y lecturas de referencia

- Reversing desde cero (caps. 01–05)
- Python ISO video tutes (lógica, no Python)
- Curso de MASM32 (Videos 1–22)

Evaluación de salida

- Convertir valores entre binario, hex y decimal sin calculadora.
- Dibujar el stack de una función de 32 bits y explicar el de 64 bits.
- Calcular un file offset a partir de una VA y verificarlo en el archivo.
- Compilar un programa propio en C y leer su hash y cabecera PE.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Memorizar la tabla hexadecimal sin entender el valor posicional.

- Confundir dirección virtual con offset del archivo.
- Creer que los tutoriales tradicionales de lógica enseñan Python moderno (enseñan lógica).

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Bits, bytes, hexadecimal y números con signo

> **Objetivo de la lección**
>
> Al terminar vas a poder convertir entre binario, hexadecimal y decimal sin calculadora, interpretar un valor de 32 bits como positivo o negativo según el contexto, y leer los bytes de un número en memoria sin confundir el orden. También vas a poder verificar una dirección virtual contra el archivo en disco.

Prerrequisitos: Etapa 0 (laboratorio y bitácora). **Duración sugerida:** 4 a 5 horas, con ejercicios. Estimación aproximada de esta guía.

Por qué esto no es un trámite

Cuando abrís un programa en un debugger ves tres columnas: direcciones, bytes y una versión legible de esos bytes. Todo está en hexadecimal. Si no podés pasar de `FB` a `251` ni entender por qué `FFFFFFFF` a veces es `-1`, cada pantalla se vuelve un jeroglífico y terminás copiando pasos sin entender.

La buena noticia: el reversing cotidiano usa muy pocos conceptos numéricos, y son los mismos siempre. Esta lección los junta.

Bit, byte y nibble

Un **bit** es la unidad mínima: vale 0 o 1. Un **byte** son 8 bits. Un **nibble** son 4 bits, que es exactamente lo que representa un dígito hexadecimal.

Por eso hexadecimal y binario se llevan tan bien: cada dígito hexadecimal equivale exactamente a cuatro bits.

1. Tabla maestra: hex ↔ binario ↔ decimal (un nibble)

Hex	Binario	Decimal
`0`	0000	0
`1`	0001	1
`2`	0010	2
`3`	0011	3
`4`	0100	4
`5`	0101	5
`6`	0110	6
`7`	0111	7
`8`	1000	8
`9`	1001	9
`A`	1010	10
`B`	1011	11
`C`	1100	12
`D`	1101	13
`E`	1110	14
`F`	1111	15

2. Pesos de los bits (dentro de un nibble y un byte)

Nibble (4 bits):

Bit	3	2	1	0
Peso	8	4	2	1

Byte (8 bits):

Bit	7	6	5	4	3	2	1	0
Peso	128	64	32	16	8	4	2	1

3. Pesos de las posiciones hex ($16^{\text{posición}}$)

Posición (desde la derecha)	Peso	Valor Decimal
0	16^0	1
1	16^1	16
2	16^2	256
3	16^3	4.096
4	16^4	65.536
5	16^5	1.048.576
6	16^6	16.777.216
7	16^7	268.435.456

Regla mnemotécnica: cada posición hacia la izquierda multiplica su peso por **16**. Cada byte completo (2 caracteres hexadecimales) hacia la izquierda multiplica su peso por **256**.

4. Potencias de 2 (para binario → decimal)

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7
1	2	4	8	16	32	64	128

2^8	2^9	2^{10}	2^{11}	2^{12}	2^{13}	2^{14}	2^{15}
256	512	1.024	2.048	4.096	8.192	16.384	32.768

5. Fórmulas

Conversión	Fórmula
Hex → Decimal	Σ (dígito × $16^{\text{posición}}$)
Binario → Decimal	Σ (bit × $2^{\text{posición}}$)
Hex → Binario	Cada dígito hexadecimal se convierte directamente a sus 4 bits (Tabla 1)
Binario → Hex	Agrupar de a 4 bits desde la derecha y traducir según Tabla 1
Decimal → Binario	Dividir por 2 sucesivamente, restos leídos de abajo hacia arriba
Decimal → Hex	Dividir por 16 sucesivamente, restos leídos de abajo hacia arriba
2 nibbles `XY`	`X × 16 + Y`
2 bytes `AB CD`	`0xAB × 256 + 0xCD`

6. Tamaños y rangos

Nibbles	Bits	Bytes	Rango Hex	Nombre
1	4	—	`0x0` – `0xF`	Nibble
2	8	1	`0x00` – `0xFF`	Byte
4	16	2	`0x0000` – `0xFFFF`	Word / short
8	32	4	`0x00000000` – `0xFFFFFFFF`	Dword / int
16	64	8	`0x0` – `0xFFFFFFFFFFFFFFFF`	Qword / long

7. Ejemplos resueltos (para chequear)

Hex	Binario	Cálculo	Decimal
`0x8F`	`1000 1111`	8 × 16 + 15 = 128 + 15	**143**
`0xFB`	`1111 1011`	15 × 16 + 11 = 240 + 11	**251**
`0x2C7`	`0010 1100 0111`	2 × 256 + 12 × 16 + 7 = 512 + 192 + 7	**711**
`0x1A3F`	`0001 1010 0011 1111`	0x1A × 256 + 0x3F = 26 × 256 + 63	**6.719**
`0x10`	`0001 0000`	1 × 16 + 0	**16**
`0x0A`	`0000 1010`	0 × 16 + 10	**10**

8. Trucos rápidos

- **Letra → Decimal:** `A=10, B=11, C=12, D=13, E=14, F=15` (A siempre empieza en 10).
- **Letra → Decimal (si te olvidás):** `9 + posición en el alfabeto` (A es la 1.ª → 9 + 1 = 10; F es la 6.ª → 9 + 6 = 15).
- **Hex → Binario:** Nunca dividas; usá directamente la tabla (4 bits por dígito).
- **Decimal → Binario:** Dividí por 2 y leé los restos de abajo hacia arriba.
- **Decimal → Hex:** Dividí por 16 y leé los restos de abajo hacia arriba.
- **Número par / impar en Binario:** Mirá únicamente el **bit 0** (1 = impar, 0 = par).
- **`0x10` no es 10:** Es **16**. El 10 en hexadecimal es `0x0A`.

9. Tabla de división por 16 (para decimal → hex)

Decimal	÷ 16 (Cociente)	Resto (Hex)	Resultado
10	0	10 → `A`	`0x0A`
16	1	0	`0x10`
143	8	15 → `F`	`0x8F`
251	15	11 → `B`	`0xFB`
255	15	15 → `F`	`0xFF`
256	16	0 (y 16 ÷ 16 = 1 resto 0)	`0x100`

Para leer un byte en hexadecimal en la práctica, separá los dos dígitos y convertí cada uno por separado:

```
0x8F  ->  8 = 1000     F = 1111   ->  1000 1111  ->  128 + 8 + 4 + 2 + 1 = 143
0xFB  ->  F = 1111     B = 1011   ->  1111 1011  ->  128+64+32+16+8+2+1 = 251
```

El método mental que usa Solid en el capítulo 01 es asignar a cada bit su potencia de dos, de derecha a izquierda: 1, 2, 4, 8, 16, 32, 64, 128. Para representar 10, prendés 8 y 2:

```
bit:     7   6   5   4   3   2   1   0
valor: 128  64  32  16   8   4   2   1
10  ->   0   0   0   0   1   0   1   0     = 0x0A
```

Esto aparece explicado en el capítulo 01 (`Reversing desde cero, cap. 01`, 00:19:00–00:22:00).

> **Cuidado con una confusión frecuente al definir little-endian**
>
> Al explicar el orden de los bytes es fácil confundir los términos "más significativo" y "menos significativo" (incluso en el capítulo 01 del curso en video, alrededor del minuto 00:22:33, el locutor los invierte por un lapsus). Recordá siempre la regla exacta: el **byte menos significativo** (el de menor peso, el de la derecha cuando escribís el número normal) es el que se guarda **primero en memoria** (en la dirección más baja).

> **En el Curso IDA: convertir bases sin calculadora**
>
> El Curso IDA aprovecha la **barra de Python** que IDA trae en la parte de abajo para pasar de una base a otra al instante: `hex(143)` devuelve `0x8f`, `bin(10)` te da `0b1010`, `0x8f` tipeado solo te lo muestra en decimal. Si esa barra no aparece, es que falta el soporte de Python de IDA (reinstalarlo lo soluciona) (Curso IDA · tute 2, Sistemas de numeración).

Little-endian: cómo se guarda un número en memoria

Los procesadores x86 y x64 son **little-endian**: cuando guardan un número de varios bytes en la memoria RAM o en un archivo, colocan el **byte menos significativo** (el de menor peso) en la dirección de memoria más baja. Por ejemplo, si en el volcado de memoria (*Dump*) ves estos cuatro bytes seguidos:

```
dirección:  1000  1001  1002  1003
contenido:    0D    0C    0B    0A
```

el número de 32 bits que representan es `0x0A0B0C0D`, y **no** `0x0D0C0B0A`. Para reconstruir el número humano a partir de la memoria, leés los bytes de derecha a izquierda (de la dirección más alta a la más baja).

La confusión típica cuando empezás es mirar la columna de bytes sueltos del debugger y creer que están escritos en orden normal de lectura. No: en el panel de memoria (*Dump*) ves los bytes en orden **little-endian** (dados vuelta), mientras que en el panel de **registros** (como `EAX` o `RAX`) el debugger ya te muestra el número armado y derecho (`0x0A0B0C0D`).

El capítulo 01 lo explica con un ejemplo de cuatro bytes `41 42 43 44` y aclara que en memoria se leen al revés (00:22:33–00:23:30). El capítulo 02 lo vuelve a mostrar al mover un valor desde una dirección a `EAX`: los bytes `36 00 00 38` en memoria se convierten en `0x38000036` en el registro (`Reversing desde cero, cap. 02`, 00:07:12–00:07:50).

Números con signo: complemento a dos

Un mismo patrón de bits puede interpretarse de dos maneras. `0xFB` es 251 si el dato es **sin signo** y −5 si es **con signo**. El procesador no guarda el signo en un bit aparte: usa una convención llamada **complemento a dos**.

La regla práctica:

- Si el bit más alto es 0, el número es positivo y vale lo mismo que en sin signo.
- Si el bit más alto es 1, el número es negativo. Para saber cuánto vale, invertí todos los bits y sumá 1 (o restá el valor a 2^n).

Ejemplo con 8 bits:

```
 5  = 0000 0101
-5  = 1111 1010 + 1 = 1111 1011 = 0xFB
```

Comprobación: `0xFB` = 251. Y 256 − 251 = 5, así que representa −5. La relación es `valor_con_signo = valor_sin_signo - 2^8`.

Los rangos que hay que saber de memoria:

Tamaño	Nombre	Bits	Sin signo	Con signo
1 byte	BYTE	8	0 a 255	−128 a 127
2 bytes	WORD	16	0 a 65.535	−32.768 a 32.767
4 bytes	DWORD	32	0 a 4.294.967.295	−2.147.483.648 a 2.147.483.647
8 bytes	QWORD	64	0 a 18.446.744.073.709.551.615	−9.223.372.036.854.775.808 a 9.223.372.036.854.775.807

Por eso `0xFFFFFFFF` es 4.294.967.295 sin signo y −1 con signo. Y `0x7FFFFFFF` es el máximo positivo de 32 bits: 2.147.483.647.

El capítulo 03 explica esta idea de dividir el rango total en dos mitades: la primera mitad (cuando el bit más alto es `0`) para el cero y los positivos, y la segunda mitad (cuando el bit más alto es `1`, es decir, cuando el primer dígito hexadecimal es `8, 9, A, B, C, D, E` o `F`) para los negativos (`Reversing desde cero, cap. 03`, 00:14:46–00:17:10).

> **El tamaño importa: una suma que "se da vuelta"**
>
> Si un programa acumula valores en un registro de **un byte**, al pasar de `0xFF` el resultado se envuelve: `0xFF + 2 = 0x01`, porque solo se guarda la parte baja. Lo mismo vale para `WORD` o `DWORD` en su tope. Es un detalle que cambia por completo lo que compara una función. El capítulo 34 lo usa: un programa suma los caracteres que ingresás en un byte, y una suma que pasa de `0xFF` deja un valor chico (por ejemplo `0x149` guarda `0x49`). Cuando leas una comparación, fijate **qué tamaño** tiene el acumulador (`Reversing desde cero, cap. 34`).

Ejemplo paso a paso: de una dirección virtual al offset del archivo

Antes de hacer la cuenta, fijemos las tres palabras que vas a ver en todas las herramientas (y que profundizaremos en la lección 1.4 sobre el formato PE):

- **Image Base (Base de imagen):** La dirección de memoria RAM donde Windows apoya el comienzo del programa al cargarlo (por ejemplo, `0x6D0000` o `0x400000`).
- **VA (*Virtual Address* · Dirección virtual):** La dirección completa de una instrucción o dato en la memoria RAM mientras el programa corre (por ejemplo, `0x6D1372`). Es lo que muestra el debugger.
- **RVA (*Relative Virtual Address* · Dirección virtual relativa):** La distancia desde el inicio del programa en memoria (`RVA = VA − Image Base`). No cambia aunque Windows cargue el programa en otra dirección.
- **File Offset (Offset en archivo):** La posición exacta de ese mismo byte dentro del archivo `.exe` guardado en el disco rígido (lo que muestra el editor hexadecimal). Es más chico que la RVA porque en el disco las secciones están guardadas de forma más compacta que en la memoria RAM.

Este cálculo es el que más vas a repetir cuando pases de un debugger a un editor hexadecimal o a un desensamblador. El capítulo 05 lo demuestra con un ejecutable concreto (`Reversing desde cero, cap. 05`, 00:00:40–00:04:20).

Datos del ejemplo:

- Image Base: `0x6D0000`
- Dirección virtual (VA) de la instrucción: `0x6D1372`
- Tamaño reservado para el header en memoria: `0x1000`
- `PointerToRawData` de la sección `.text`: `0x400`

Paso 1. Calculá la RVA (Relative Virtual Address), que es la distancia desde la base:

```
RVA = VA − Image Base
    = 0x6D1372 − 0x6D0000
    = 0x1372
```

Paso 2. Restá el espacio que el header ocupa en memoria (`0x1000`) y sumá el offset real donde empieza `.text` en el archivo (`0x400`):

```
File Offset = RVA − 0x1000 + 0x400
            = 0x1372 − 0x1000 + 0x400
            = 0x372 + 0x400
            = 0x772
```

Paso 3. Verificá en el archivo. En el offset `0x772` del `.exe` deberían estar los bytes `E8 C4 03 00 00`, los mismos que muestra el debugger en esa dirección. El capítulo 05 lo comprueba con un editor hexadecimal (00:03:41–00:04:30).

> **Por qué se resta 0x1000 y no el tamaño real del header**
>
> El cargador de Windows alinea el header a `0x1000` en memoria, aunque el header real sea más chico (en el ejemplo, `.text` empieza en el archivo en `0x400`). Por eso la fórmula usa la diferencia entre la alineación de memoria y el offset real de la sección. Cada sección tiene su propio `PointerToRawData`; no uses `0x400` para `.rdata` ni para `.data`.

Ejemplo paso a paso: a dónde salta un `call` directo

En un `call` directo, el opcode `E8` va seguido de un desplazamiento de 4 bytes. La dirección destino se calcula sumando al desplazamiento la dirección de la **instrucción siguiente** (no la del `call`).

En el capítulo 05, el `call` está en `0x6D1372`, mide 5 bytes y su desplazamiento es `0x3C4`:

```
destino = VA_del_call + tamaño_del_call + desplazamiento
        = 0x6D1372 + 5 + 0x3C4
        = 0x6D1377 + 0x3C4
        = 0x6D173B
```

El debugger muestra `0x6D173B` como destino. Si el salto fuera hacia atrás, el desplazamiento sería negativo y se rellenaría con `FF` a la izquierda antes de sumar (cap. 05, 00:09:00–00:12:00).

Práctica

1. Convertí a hexadecimal: 42, 200, 255, 1024, 4096.
2. Convertí a decimal: `0x2A`, `0xC8`, `0xFF`, `0x400`, `0x1000`.
3. ¿Qué valor con signo representa `0xFFFFFFFB` en 32 bits? ¿Y sin signo?
4. Escribí los bytes que ocupa `0x11223344` en memoria little-endian.
5. Con Image Base `0x400000`, VA `0x401040`, header `0x1000` y `.text` en `0x400`, calculá el file offset.
6. Un `call` en `0x401000` tiene desplazamiento `0x20`. ¿A qué dirección salta? (Ojo con el tamaño de la instrucción.)
7. Abrí PowerShell y comprobá tus conversiones:

```
'{0:X}' -f 42          # 2A
'{0:X8}' -f 4294967295 # FFFFFFFF
[Convert]::ToInt32('FFFFFFFB',16)  # -5
```

Preguntas de repaso

1. ¿Cuántos bits hay en un byte y cuántos dígitos hexadecimales lo representan? Ocho bits y dos dígitos hexadecimales, porque cada dígito representa cuatro bits.

2. ¿Por qué `0xFB` puede ser 251 o −5? Porque el patrón de bits es el mismo. El tipo del dato y la instrucción que lo usa deciden si se interpreta con signo o sin signo.

3. ¿Qué significa little-endian? Que el byte menos significativo se almacena en la dirección más baja. Para leer un valor de memoria hay que invertir el orden de los bytes.

4. ¿La dirección que muestra el debugger es la misma que la del archivo en disco? No. La del debugger es una dirección virtual; la del archivo es un offset. Se relacionan con la fórmula `offset = RVA - alineación_del_header + PointerToRawData`.

5. ¿Por qué el destino de un `call` se calcula desde la instrucción siguiente? Porque el desplazamiento se mide desde el final de la instrucción actual, que es donde el procesador ya actualizó `EIP`/`RIP`. Sumar el tamaño de la instrucción es parte del cálculo.

Antes de avanzar

Tenés que poder, sin mirar la tabla:

- Pasar un byte de hexadecimal a binario y a decimal.
- Explicar complemento a dos con el ejemplo de `0xFB`.
- Escribir `0x0A0B0C0D` como bytes en memoria.
- Calcular un file offset a partir de una VA y un `call` directo a partir de su desplazamiento.

Puerta de salida de la Etapa 1 · bloque 1

Si los ejercicios 3 a 6 te salen con seguridad, seguí con memoria y procesos. Si no, repetilos con lápiz y papel antes de abrir un debugger: la pantalla no perdona la aritmética floja.

Fuentes de esta lección

- `Reversing desde cero, cap. 01` — bits, bytes, little-endian, tamaños (00:18:00–00:24:00).
- `Reversing desde cero, cap. 02` — lectura de un valor desde memoria (00:07:00–00:08:00).
- `Reversing desde cero, cap. 03` — números con signo (00:14:46–00:17:10).
- `Reversing desde cero, cap. 05` — VA, RVA, file offset y desplazamientos (00:00:40–00:12:00).
- Curso IDA · tute 2, Sistemas de numeración — binario/decimal/hexadecimal y la barra de Python de IDA para convertir bases.

Síntesis de estudio de esta lección

Núcleo conceptual: el ejemplo numérico de la VA `0x6D1372`, la fórmula del offset y el cálculo del `call` a `0x6D173B`. **Conocimiento externo:** complemento a dos, rangos y aritmética hexadecimal. **Recomendación editorial:** el orden de los ejercicios y la advertencia sobre las imprecisiones del audio.

Memoria: stack, heap y marcos de función

> **Objetivo de la lección**
>
> Al terminar vas a poder dibujar el stack de una función antes y después de un `call`, ubicar los argumentos, el return address, el `EBP` guardado y las variables locales, y explicar por qué en 64 bits los primeros argumentos no están en el stack.

Prerrequisitos: números y little-endian (lección anterior). **Duración sugerida:** 4 horas. Estimación aproximada.

Las cuatro zonas que importan

Un proceso ve un espacio de direcciones dividido en regiones. Para empezar alcanza con cuatro:

Zona	Qué guarda	Crece hacia	Vida
Código (`.text`)	Instrucciones	—	Todo el proceso
Datos (`.data`, `.rdata`)	Variables globales, constantes, strings	—	Todo el proceso
Heap	Memoria pedida en tiempo de ejecución (`malloc`, `new`)	Direcciones altas	Hasta que la liberás
Stack	Argumentos, direcciones de retorno, variables locales	Direcciones bajas	Hasta que la función retorna

El stack y el heap son distintos no solo por su ubicación: cambian por motivos distintos. El stack lo administra el procesador automáticamente con cada llamada y retorno de función; el heap lo administra el runtime (la librería de C, el sistema operativo o el recolector de basura). Esta lección se concentra en el **stack** porque es el mapa vivo que vas a mirar todo el tiempo en el debugger.

Diccionario rápido para leer los ejemplos de esta lección (adelanto de la Etapa 2)

Los registros e instrucciones se estudian a fondo en las lecciones **2.1** y **2.2**. Para seguir los diagramas de esta lección sin que te falte vocabulario, tené a mano estas seis piezas:

- **Registros (`EAX` , `EBX` , `ECX` , `EDX`):** "Cajitas" de memoria ultrarrápida dentro del CPU (4 bytes en 32 bits; 8 bytes con prefijo `R` en 64 bits: `RAX` , `RCX` ...) para calcular y mover datos.
- **`ESP` / `RSP` (*Stack Pointer*) y `EBP` / `RBP` (*Base Pointer*):** `ESP` apunta siempre a la **cima actual del stack**; `EBP` se clava como "estaca fija" al iniciar la función para ubicar argumentos (`[EBP+8]`) y locales (`[EBP-4]`).
- **`push X` , `pop Y` , `mov A, B` y `sub A, N` :** `push` apila arriba del stack (resta 4/8 a `ESP`); `pop` desapila (suma 4/8 a `ESP`); `mov` copia `B` en `A` ; `sub` resta `N` a `A` .
- **Corchetes `[dirección]` vs. `lea` :** Los corchetes significan **"el valor guardado dentro de esa dirección"** (`mov eax, [ebp-4]` lee la memoria), mientras que `lea edx, [eax+4]` solo calcula el número de la dirección `eax+4` sin leer la memoria.

El stack es una pila LIFO

LIFO significa *last in, first out*: lo último que apilás es lo primero que sacás. En x86, el stack crece hacia direcciones **más bajas**. `ESP` (en 32 bits) o `RSP` (en 64) apunta siempre a la cima actual.

```
push eax      ; ESP = ESP - 4 ; [ESP] = EAX
pop  ebx      ; EBX = [ESP] ; ESP = ESP + 4
```

En 64 bits el decremento es de 8 bytes, salvo que empujes algo más chico.

Ejemplo paso a paso: el stack de una función de 32 bits

Este ejemplo sale del capítulo 18, donde Ricardo compara una función basada en `EBP` con una basada en `ESP` usando un ejecutable de práctica (`Reversing desde cero, cap. 18`, 00:02:40–00:08:30). La función `main` recibe tres argumentos: `argc` , `argv` y `envp` .

Antes de llamar

La función que llama apila los argumentos **de derecha a izquierda**:

```
push envp
push argv
push argc
call main
```

El stack queda así (cada celda son 4 bytes; las direcciones bajas abajo):

```
        +------------------+
esp --> | argc             |
        | argv             |
        | envp             |
        +------------------+
```

El `call`

`call` hace dos cosas: empuja la dirección de retorno y salta. La dirección de retorno es la de la instrucción **siguiente** al `call`, para que `ret` sepa dónde volver.

```
        +------------------+
        | argc             |
        | argv             |
        | envp             |
        | return address   |  <-- ESP
        +------------------+
```

En el ejemplo del video, la dirección de retorno es `0x011912EA` (cap. 18, 00:07:16).

El prólogo

Casi todas las funciones de 32 bits empiezan igual:

```
push ebp          ; guarda el EBP de la función que llamó
mov  ebp, esp     ; fija un punto de referencia estable
sub  esp, 0x28    ; reserva espacio para variables locales
```

Después del prólogo, el stack tiene una forma reconocible:

```
        +------------------+  <-- EBP + 0x10
        | envp (3.er arg)  |
        | argv (2.° arg)   |  <-- EBP + 0x0C
        | argc (1.er arg)  |  <-- EBP + 0x08
        | return address   |  <-- EBP + 0x04
        | EBP guardado     |  <-- [EBP]
        +------------------+
        | local [EBP-4]    |
        | local [EBP-8]    |  <-- espacio reservado por sub esp, 0x28
esp --> | ...              |
        +------------------+
```

Cuidado con el orden: los argumentos se apilan de derecha a izquierda, así que el primero de la función queda en la dirección más baja, justo arriba del return address. Por eso `[EBP+8]` es `argc`, `[EBP+0x0C]` es `argv` y `[EBP+0x10]` es `envp`. Lo importante no es memorizar los números de este ejemplo, sino el método: **contá desde `EBP` y verificá en el panel de stack**.

En el video, Ricardo muestra la correspondencia entre la vista estática del stack de IDA y la realidad del debugger: los tres `push`, el return address, el `push ebp`, el `mov ebp, esp` y el `sub esp, 0x28` aparecen en ese orden (cap. 18, 00:02:34–00:10:00).

El epílogo

Al final, la función deshace lo que hizo:

```
mov esp, ebp      ; libera las variables locales
pop  ebp          ; recupera el EBP de la función que llamó
ret               ; saca el return address y vuelve
```

`leave` es una instrucción que hace las dos primeras operaciones juntas. Si ves `leave; ret`, es el mismo patrón.

> **Por qué `EBP` y no `ESP`**
>
> `ESP` cambia cada vez que la función empuja algo para llamar a otra función. Si las variables locales se referenciaran con `ESP`, cada `push` cambiaría sus direcciones. Por eso el prólogo copia `ESP` a `EBP` y usa `EBP` como referencia estable: `[EBP-4]`, `[EBP-8]`, etc. (cap. 18, 00:10:30–00:12:10).

64 bits: los argumentos viajan en registros

En x64 cambia la convención. Los primeros cuatro argumentos enteros van en registros:

1.er argumento	2.º argumento	3.er argumento	4.º argumento	5.º en adelante
`RCX`	`RDX`	`R8`	`R9`	Stack (después de `0x20` bytes de *shadow space*)

Los demás van en el stack, y el llamador debe reservar **32 bytes de shadow space** (`0x20`) antes del `call`, aunque no use todos los registros. Por eso en el capítulo 18 se ve `sub rsp, 0x138` para las variables locales y referencias del tipo `[rsp+0x20]`, `[rsp+0x28]`, etc. (00:14:00–00:20:00).

En 64 bits es común que las funciones sean **basadas en RSP**: no hay `push rbp; mov rbp, rsp` porque `RSP` no cambia tanto (los argumentos no se apilan uno por uno). Las variables locales se referencian como `[rsp+0xNN]`, y el desensamblador suele mostrar el offset relativo al valor inicial de `RSP` para que no tengas que recalcular.

> **El error de contar mal el shadow space**
>
> Cuando el quinto argumento va al stack, no va inmediatamente después del return address: va después de los 32 bytes reservados para los cuatro registros. Si ves `[rsp+0x28]` como primer argumento apilado, es porque `0x20` son los shadow slots y `0x28` el primer argumento real (cap. 18, 00:17:00–00:18:30).

Heap: la memoria que pedís vos

El heap se usa cuando el tamaño no se conoce en tiempo de compilación o cuando el dato tiene que sobrevivir a la función que lo creó. En C:

```
char *buf = malloc(64);
if (buf == NULL) { /* error */ }
strcpy(buf, "hola");
free(buf);
```

En Python, `bytearray(64)` o una lista cumplen un rol parecido, pero el manejo lo hace el intérprete.

Tres diferencias prácticas con el stack:

1. **Vos administrás la vida** (en C). Si te olvidás de `free`, hay fuga de memoria; si liberás dos veces, corrupción.
2. **No tiene un orden LIFO.** Podés liberar en cualquier orden.
3. **Los errores son más silenciosos.** Un desborde de stack suele terminar en crash; un desborde de heap puede pasar desapercibido y explotar después.

Estructuras y objetos en memoria (vista previa)

Una **estructura** (`struct` en C) agrupa varios campos de distinto tamaño y tipo guardados uno detrás del otro en memoria. A diferencia de un arreglo (*array*), donde todos los casilleros miden lo mismo, en una estructura cada campo vive a una distancia fija (**offset** o desplazamiento) desde el inicio del bloque. En el código compilado vas a ver un registro con la **dirección base** de la estructura al que se le suma el offset de cada campo:

```
mov eax, [ebp-4]      ; EAX = dirección donde empieza la estructura (puntero base)
mov ecx, [eax+0x58]   ; Con mov + corchetes: lee el VALOR guardado en el campo +0x58
lea edx, [eax+4]      ; Con lea: NO lee memoria; calcula la DIRECCIÓN del campo +4
```

> **El padding cambia los offsets**
>
> El tamaño total de una estructura **no** es siempre la suma exacta de sus campos. Para que el CPU lea rápido, el compilador alinea los datos a múltiplos de 4 u 8 bytes insertando huecos vacíos llamados ***padding*** (relleno). Por ejemplo, en 32 bits un entero de 4 bytes (`DWORD`) seguido de 1 byte (`BYTE`) ocupa **8 bytes** (agrega 3 de relleno). Si ves offsets que "saltan" dejando huecos, es por el padding (cap. 20b).

¿Y qué es un objeto de C++ en memoria? Es una estructura a la que se le asocian funciones (métodos). Aunque C++ se estudia a fondo en la lección 2.18, te sirve reconocer tres ideas básicas cuando aparezcan nombradas:

- **Reserva y constructor:** al crear un objeto con `new`, se pide memoria en el heap (como `malloc`) y se llama a una función inicializadora llamada **constructor**.
- **Puntero `this` y `vtable`:** cada método recibe oculta la dirección del objeto (`this`, en `ECX` en 32 bits o `RCX` en 64 bits). Si la clase tiene métodos virtuales, los primeros 4 u 8 bytes del objeto (`offset 0`) guardan un puntero hacia su tabla de funciones o **vtable** (cap. 33; lo veremos en las lecciones 2.5 y 2.7).

Buffers, strings y el centinela (`0x00`)

Una cadena (*string*) en C es una fila de bytes consecutivos terminada con un byte cero (`0x00`, **terminador nulo**): por ejemplo, `"Hola"` ocupa **5 bytes** (`48 6F 6C 61 00`). Si una función copia datos en un arreglo local del stack (*buffer*) sin controlar el límite, puede desbordarlo y pisar el `return address`. Para detectarlo antes del `ret`, los compiladores colocan una ***security cookie*** (*stack canary*) entre los buffers y el `return address`, combinada con `XOR`, y verifican que siga intacta antes de retornar (`Reversing desde cero, cap. 15`, 00:07:34–00:08:00).

Práctica

1. Dibujá el stack de una función `int suma(int a, int b)` en 32 bits, desde el `push b` hasta el `ret`. Marcá `EBP`, `ESP`, argumentos, return address y variable local.
2. Escribí en C una función que reserve un buffer de 16 bytes en el stack y otra que use `malloc(16)`. Compilalas y mirá en x64dbg dónde queda cada buffer.
3. En el programa anterior, poné un breakpoint en la primera instrucción de la función y anotá `ESP` y `EBP`. Después del prólogo, volvé a anotarlos. ¿Cuánto cambió `ESP`?
4. Escribí el mismo programa para x64 y verificá en qué registros llegan `a` y `b`.
5. Explicá por qué `[ebp+8]` suele ser el primer argumento en 32 bits y por qué en 64 bits no lo es.

Preguntas de repaso

1. ¿Quién empuja el return address? La instrucción `call`, no la función llamada. `ret` lo consume.

2. ¿Por qué el stack crece hacia abajo? Es una convención de la arquitectura x86: `push` decrementa el puntero y `pop` lo incrementa. Lo importante es que el modelo mental sea consistente, no la dirección física.

3. ¿Qué guarda `push ebp` en el prólogo? El valor de `EBP` de la función que llamó, para poder restaurarlo al final y no romper las referencias de esa función.

4. ¿Dónde están las variables locales? Entre `EBP` y `ESP`, en direcciones menores que `EBP` (`[EBP-4]`, `[EBP-8]`, etc.). El `sub esp, N` reserva ese espacio.

5. ¿Los argumentos de una función de 64 bits siempre están en el stack? No. Los primeros cuatro enteros van en `RCX`, `RDX`, `R8` y `R9`; el resto va al stack después de 32 bytes de shadow space.

6. ¿Qué diferencia hay entre un desborde de stack y uno de heap? El de stack suele pisar el return address y provocar un crash inmediato; el de heap corrompe estructuras internas y puede no manifestarse hasta mucho después.

Antes de avanzar

Tenés que poder:

- Dibujar el stack de un `call` de 32 bits con argumentos, return address, `EBP` guardado y locales.
- Nombrar los registros de argumentos de x64 y explicar el shadow space.
- Explicar la diferencia entre stack y heap con un ejemplo de cada uno.
- Reconocer un prólogo y un epílogo en un listado de assembly.

> **Puerta de salida**
>
> Si podés explicar el stack sin mirar el dibujo, estás listo para verlo en vivo en el debugger. Si no, volvé al ejemplo del capítulo 18 y reconstruilo en papel.

Fuentes de esta lección

- `Reversing desde cero, cap. 18` — stack de 32 y 64 bits, prólogo, epílogo, shadow space.
- `Reversing desde cero, cap. 15` — security cookie y protección del return address.
- `Reversing desde cero, cap. 01` — ventana de stack y mapa de memoria.
- `Reversing desde cero, cap. 20b` — estructuras, acceso base+offset, stack vs heap y padding/alineación.
- `Reversing desde cero, cap. 33` — objetos C++: `new` / `malloc`, constructor, `this` en ECX y puntero a la vtable en el offset 0.

> **Síntesis de estudio de esta lección**
>
> **Núcleo conceptual:** el ejemplo del capítulo 18 (argumentos, return address `0x011912EA`, `sub esp, 0x28`, shadow space), la cookie del capítulo 15, el padding del 20b y el objeto con vtable del 33.
> **Conocimiento externo:** layout general del espacio de direcciones, heap y strings en C.
> **Recomendación editorial:** los diagramas y ejercicios.

Procesos, hilos y mapa de memoria

> **Objetivo de la lección**
>
> Al terminar vas a poder distinguir programa, proceso e hilo; explicar qué comparte y qué no comparte un hilo con los demás; crear un programa propio con dos hilos y encontrarlos en el debugger; y leer las regiones del mapa de memoria de un proceso.

Prerrequisitos: memoria y stack. **Duración sugerida:** 3 horas. Estimación aproximada.

Programa, proceso, hilo

Para no confundir estos tres términos en el debugger, pensalos con una analogía sencilla:

- Un **programa** es el archivo `.exe` quieto en el disco (como la receta impresa en un libro o el plano de un taller): solo contiene bytes, cabeceras e instrucciones guardadas.
- Un **proceso** es ese programa puesto en marcha por Windows en la memoria RAM (como el taller abierto y funcionando): tiene su propio espacio privado de memoria, sus archivos abiertos (*handles*) y sus permisos de seguridad.
- Un **hilo** (*thread*) es cada trabajador activo dentro de ese proceso que va leyendo y ejecutando instrucciones línea por línea. Todo proceso arranca con al menos un hilo principal (el que ejecuta `main`), pero puede crear varios hilos para hacer tareas al mismo tiempo.

La diferencia importa porque el debugger te muestra las tres cosas a la vez: el archivo que abriste, el proceso que Windows creó en memoria y los hilos que están corriendo adentro.

Concepto	Qué es	Qué comparte
Programa	Archivo `.exe` en disco	Nada: es estático
Proceso	Instancia en ejecución	Espacio de direcciones, handles, token
Hilo	Flujo de ejecución	Código, datos globales, heap, handles
Hilo (propio)	Contexto de CPU, stack, estado	Nada de eso con otros hilos

Cada hilo tiene **su propio stack** y **su propio contexto de CPU** (registros). Si dos hilos comparten una variable global, los dos ven la misma dirección; si cada uno declara una variable local, cada uno tiene su copia en su stack.

El capítulo 15b lo define así: un hilo es "una función o una parte del programa que se ejecuta" al mismo tiempo que otra (`Reversing desde cero, cap. 15b`, 00:00:55).

Ejemplo paso a paso: dos hilos en un programa propio

El capítulo 15b usa un programa de ejemplo con un hilo que incrementa un contador y un `main` que espera una tecla. La estructura es esta:

```c
#include <windows.h>
#include <stdio.h>

volatile int MyCounter = 0;   // 'volatile': impide que el compilador optimice/elimine el bucle

DWORD WINAPI MyThread(LPVOID lpParam) {
    volatile int *contador = (volatile int *)lpParam;
    while (*contador < 0xFFF) {
        (*contador)++;
    }
    return 0;
}

int main(void) {
    HANDLE h = CreateThread(NULL, 0, MyThread, &MyCounter, 0, NULL);
    if (h == NULL) return 1;
    printf("Presione Enter para terminar...\n");
    getchar();
    printf("Contador: %d\n", MyCounter);
    CloseHandle(h);
    return 0;
}
```

> **Compilá con `volatile` o el bucle desaparece en Release**
>
> Sin `volatile`, en Release (`/O2` en MSVC, `-O2` en GCC) el compilador asume que nadie más toca `MyCounter` y **elimina el bucle** (lo reduce a `*contador = 0xFFF;`) o lee el valor una sola vez en un registro. Con `volatile`, cada iteración lee/escribe memoria de verdad y el ejemplo es reproducible en cualquier flag de compilación.

Los argumentos de `CreateThread` que importan para el análisis son:

Posición	Nombre	En el ejemplo
3.º	`lpStartAddress`	`MyThread`: dónde empieza el hilo
4.º	`lpParameter`	`&MyCounter`: puntero que recibe el hilo
5.º	`dwCreationFlags`	`0` = arranca ya; `4` (`CREATE_SUSPENDED`) = creado detenido
Retorno	`HANDLE`	Se usa con `ResumeThread` o `CloseHandle`

El capítulo muestra exactamente esto: el tercer argumento es la dirección de inicio, el cuarto es el puntero a la variable, la bandera `0` hace que arranque al instante y `4` lo deja suspendido hasta un `ResumeThread` (cap. 15b, 00:02:00–00:03:00).

Qué ves en el debugger

1. Abrí el programa en x64dbg y dejalo en el entry point.
2. Abrí **Debugger** → **Thread List**. Vas a ver el hilo principal y el que creaste.

3. Poné un breakpoint en `MyThread` y dejá correr. Cuando el hilo llegue, el debugger se detiene **en ese hilo**.
4. En la lista de hilos, el que está detenido aparece en negrita.
5. Cambiá de hilo y mirá el stack: cada uno tiene el suyo.

En el ejemplo del curso aparecen dos hilos en la versión de 32 bits y cuatro en la de 64 bits (el principal más tres que crea el programa) (cap. 15b, 00:10:02 y 00:15:12).

> **El debugger no puede mostrar dos hilos a la vez**
>
> Cuando un hilo se detiene en un breakpoint, los demás siguen o quedan suspendidos según cómo esté configurado el debugger, pero la pantalla te muestra **un contexto**. Si "de repente salta a otro lado", puede ser un cambio de hilo, no un salto del programa. Revisá la columna de hilo antes de asustarte (cap. 15b, 00:09:50 y 00:14:05).

Handles: los números que identifican recursos

Un **handle** es un identificador opaco que el sistema operativo te da para referenciar un recurso: una ventana, un archivo, un hilo, un evento. No es una dirección de memoria ni un puntero; es un índice a una tabla interna del proceso.

El capítulo 01 muestra la pestaña **Handles** de x64dbg y explica que son "identificadores de ventanas y archivos y demás cosas" (`Reversing desde cero, cap. 01`, 00:06:20).

Para el análisis: cuando una API devuelve un handle y el programa lo guarda en una variable, esa variable no contiene "el archivo", contiene un número que el sistema traduce. Si ves un valor pequeño como `0x000000A4`, es un handle, no una dirección.

El mapa de memoria

La pestaña **Memory Map** de x64dbg muestra las regiones del espacio de direcciones del proceso: imagen del ejecutable, DLLs cargadas, heap, stack y regiones reservadas. Cada fila tiene base, tamaño, permisos y tipo.

En el capítulo 05 se lee así: el ejecutable está mapeado desde `0x6D0000`; los primeros `0x1000` bytes son el header; después vienen `.text`, `.rdata`, `.data`, `.rsrc` y `.reloc` (`Reversing desde cero, cap. 05`, 00:00:10–00:01:00).

Los permisos se leen en tres letras:

Permiso	Significado	Ejemplo
`R`	Lectura	`.rdata`
`W`	Escritura	`.data`, heap, stack
`X`	Ejecución	`.text`
`RWX`	Todo	Sospechoso en un binario normal; típico de unpacking o shellcode

Una región `RWX` no es prueba de malware, pero es una pregunta que vale la pena hacerse: ¿por qué este programa necesita escribir y ejecutar en la misma página?

Hilos, excepciones y TLS: el puente a temas avanzados

Cada hilo tiene su propio estado de excepciones. En 32 bits, el sistema encadena los manejadores en el stack y el primer DWORD del bloque de información del hilo (`FS:[0]`) apunta al primero de la cadena (cap. 16b, 00:02:29). En 64 bits, esa información se guarda en tablas `.pdata` / `.rdata` y no en el stack (cap. 36, 00:00:27).

También cada hilo puede tener **TLS** (*Thread Local Storage*): variables globales que en realidad tienen una copia por hilo. El capítulo 38 lo trata en detalle; acá alcanza con saber que existe y que explica por qué una variable "global" puede tener valores distintos según el hilo.

No necesitás dominar esto para seguir. Lo importante es que cuando veas `FS:[0]`, `GS:[0]`, `.pdata` o `TlsAlloc`, sepas que estás frente a infraestructura del sistema, no a código del programa.

> **Qué son FS y GS: apuntan al TEB**
>
> Los registros de segmento `FS` (x86, 32 bits) y `GS` (x64) no hacen segmentación clásica: apuntan siempre al **TEB** (*Thread Environment Block*) del hilo actual. En x86, `FS:[0]` es el primer campo del TEB y guarda el puntero a la cadena de manejadores de excepciones SEH en el stack, y `FS:[0x30]` lleva al **PEB**. En x64, `GS:[0x30]` apunta al TEB y `GS:[0x60]` al PEB. Lo vas a ver constantemente en la resolución de APIs por PEB (2.14) y en los chequeos anti-debug (2.15).

Práctica

1. Compilá el programa de dos hilos de arriba (en el guest, con Visual Studio o MinGW).
2. Abrilo en x64dbg y completá esta tabla:

Pregunta	Tu respuesta
¿Cuántos hilos hay al inicio?	
¿En qué dirección empieza `MyThread`?	
¿Qué valor tiene el argumento que recibe?	
¿Qué pasa si usás `CREATE_SUSPENDED` (`4`)?	
¿Qué región de memoria es el stack? ¿Qué permisos tiene?	

3. Agregá un `Sleep(5000)` dentro del hilo y observá la lista de hilos mientras duerme.
4. Poné un breakpoint en el `inc` / `add` del bucle y logueá el **valor del contador** leyendo la memoria apuntada, no `{EAX}` a secas: en x64, `MyCounter` llega por `RCX`, así que `{EAX}` loguearía los 32 bits bajos del *puntero*, no el dato. En x64dbg usá `Contador = {[&MyCounter]}` (o `{d:rcx}` si `RCX` tiene el puntero).

Preguntas de repaso

1. ¿Qué comparte un hilo con los demás hilos del mismo proceso? El espacio de direcciones (código, datos globales, heap) y los handles. No comparte el stack ni los registros.

2. ¿Un proceso puede tener varios hilos? ¿Y varios procesos el mismo código? Sí a las dos. Varios procesos pueden ejecutar la misma imagen (por ejemplo, dos instancias del mismo `.exe`), cada uno con su propio espacio de direcciones.

3. ¿Qué hace `CREATE_SUSPENDED`? Crea el hilo detenido. No empieza a ejecutar hasta que otro hilo llame a `ResumeThread`.

4. ¿Por qué el debugger muestra un solo hilo a la vez? Porque muestra el contexto de CPU de un hilo; los demás existen pero no se pueden "mirar" simultáneamente en la misma ventana.

5. ¿Qué indica una región `RWX`? Que se puede leer, escribir y ejecutar. Es inusual en un binario compilado normal y aparece en etapas de desempaquetado, generación de código o shellcode. No prueba nada por sí sola, pero orienta la investigación.

Antes de avanzar

Tenés que poder:

- Explicar la diferencia entre programa, proceso e hilo con un ejemplo.
- Nombrar los argumentos de `CreateThread` que importan y qué hace cada uno.
- Encontrar la lista de hilos y el mapa de memoria en x64dbg.
- Leer los permisos de una región y decir si es normal o sospechosa.

Puerta de salida

Si podés crear tu propio programa de dos hilos y encontrarlo en el debugger, estás listo para el formato PE. Si no, hacelo antes: es la mejor forma de entender qué significa "contexto de hilo".

Fuentes de esta lección

- `Reversing desde cero, cap. 15b` — hilos, `CreateThread`, lista de hilos, cambio de contexto.
- `Reversing desde cero, cap. 01` — handles y mapa de memoria.
- `Reversing desde cero, cap. 05` — image base y secciones en el mapa.
- `Reversing desde cero, cap. 16b` y `cap. 36` — estado de excepciones por hilo.

Síntesis de estudio de esta lección

Núcleo conceptual: el ejemplo de `CreateThread` con `MyCounter`, la lista de hilos y la lectura del mapa del capítulo 05. **Conocimiento externo:** definición de proceso, handles como tabla interna, permisos de página. **Recomendación editorial:** el programa de práctica y la tabla de ejercicios.

El formato PE: cabeceras, secciones e imports

> **Objetivo de la lección**
>
> Al terminar vas a poder abrir un `.exe` en un visor de PE, ubicar el entry point, la image base y las secciones; convertir una VA a un offset del archivo; y explicar qué es la IAT y por qué un `call` indirecto pasa por ella.

Prerrequisitos: números, memoria y procesos. **Duración sugerida:** 4 horas. Estimación aproximada.

Qué es un PE y por qué te importa

PE (*Portable Executable*) es el formato de los ejecutables y DLLs de Windows, tanto de 32 como de 64 bits. El cargador de Windows lo lee para saber qué código ejecutar, qué DLLs cargar, qué direcciones ajustar y qué recursos mostrar.

Cuando abrís un programa en un debugger, el sistema ya hizo ese trabajo: la imagen está mapeada en memoria. Cuando lo abrís en un editor hexadecimal, ves el archivo tal cual. **La diferencia entre ambos mundos es el formato PE**, y por eso el capítulo 05 insiste en la relación entre dirección virtual, RVA y offset.

El video 33 del curso de OllyDbg lo dice sin vueltas: el PE es "como un gran diccionario o una enciclopedia" en la cabecera, que le dice a Windows de qué se compone el archivo, dónde están los datos, en qué sección está el código y qué librerías puede usar (`Videos y material adicional, Video 33`, 00:00:00–00:02:00).

El recorrido de una cabecera

No hace falta memorizar cada campo. Alcanza con saber el recorrido y dónde mirar.

1. Cabecera DOS

Todo PE empieza con `MZ` (los bytes `4D 5A`). En el offset `0x3C` hay un DWORD llamado `e_lfanew` que indica dónde empieza la cabecera PE. Ese salto existe por compatibilidad histórica con MS-DOS.

2. Firma PE y cabecera de archivo

En `e_lfanew` están los bytes `50 45 00 00` ("PE\0\0") y después la **IMAGE_FILE_HEADER**: arquitectura (`Machine`), cantidad de secciones (`NumberOfSections`), tamaño de la cabecera opcional y características.

3. Cabecera opcional

Acá están los datos que más vas a usar:

Campo	Qué te dice
`Magic`	`0x10B` = PE32; `0x20B` = PE32+ (64 bits)
`AddressOfEntryPoint`	RVA del entry point
`ImageBase`	Dirección preferida de carga
`SectionAlignment`	Alineación en memoria (normalmente `0x1000`)
`FileAlignment`	Alineación en el archivo (normalmente `0x200` o `0x400`)
`DataDirectory[]`	Tablas de imports, exports, recursos, relocalizaciones, TLS, excepciones

4. Tabla de secciones

Cada sección tiene nombre, tamaño virtual, RVA, tamaño en archivo, offset en archivo y características. Las clásicas:

Sección	Contenido	Permisos típicos
`.text`	Código	R-X
`.rdata`	Constantes, strings, imports	R--
`.data`	Variables globales inicializadas	RW-
`.rsrc`	Recursos (iconos, diálogos, strings)	R--
`.reloc`	Relocalizaciones	R--
`.pdata`	Información de excepciones en x64	R--

Ejemplo paso a paso: de la VA al offset

Volvemos al ejecutable del capítulo 05, que ya usamos en la lección de números. El debugger muestra el entry point en `0x6D1372` y el mapa de memoria dice que la imagen empieza en `0x6D0000`.

Paso 1. RVA.

```
RVA = 0x6D1372 − 0x6D0000 = 0x1372
```

Paso 2. Offset en el archivo. El header ocupa `0x1000` en memoria y `.text` empieza en el archivo en `0x400`:

```
offset = 0x1372 − 0x1000 + 0x400 = 0x772
```

Paso 3. Verificación. En el offset `0x772` del archivo están los bytes `E8 C4 03 00 00`: el `call` que el debugger muestra en `0x6D1372`. La comprobación está en el capítulo 05, entre 00:02:30 y 00:03:10

(`Reversing desde cero, cap. 05`).

> **La fórmula general**
>
> `offset = VA − ImageBase − VirtualAddress_de_la_sección + PointerToRawData_de_la_sección`
>
> Para `.text` del ejemplo: `VA − 0x6D0000 − 0x1000 + 0x400`. Cada sección tiene su propio `VirtualAddress` y su propio `PointerToRawData`. Si usás los de `.text` para una dirección de `.rdata`, el cálculo da cualquier cosa.

Imports e IAT

Un programa de Windows casi nunca hace todo solo: llama a funciones de DLLs (`kernel32.dll`, `user32.dll`, etc.). Esas funciones no están en el `.exe`; están en la DLL que se carga en tiempo de ejecución. El PE guarda una **tabla de imports** con los nombres (o números) de las funciones que necesita.

Cuando el cargador carga el programa, completa la **IAT** (*Import Address Table*): una tabla de punteros donde cada entrada contiene la dirección real de la función en memoria.

Por eso un `call` a una API se ve así:

```
call dword ptr [0x6E300C]
```

No es un `call` directo a una dirección fija: es un `call` **indirecto** que lee el puntero guardado en `0x6E300C`. Si esa entrada de la IAT apunta a `kernel32!SetUnhandledExceptionFilter`, el `call` termina ahí.

El capítulo 05 lo muestra en vivo: el opcode es `FF15`, la dirección entre corchetes es `0x6E300C`, y adentro está el puntero a la función de `kernel32`. Cuando Ricardo cambia manualmente el contenido de esa entrada por `0x6D13B8`, el mismo `call` pasa a llamar a una dirección interna del programa. Eso demuestra que **la instrucción no cambió: cambió el puntero de la IAT** (cap. 05, 00:13:00–00:18:00).

> **IAT redirection: tema avanzado, no tarea inicial**
>
> Cambiar entradas de la IAT es una técnica que usan los empacadores para ocultar llamadas. El capítulo 17 la estudia en el marco del desempaquetado de protectores comerciales. En esta guía se explica **qué es** y cómo reconocerla, pero no se propone como ejercicio: no vas a desempaquetar protectores comerciales ni a eludir sus protecciones.

Imports por nombre y por ordinal

En la tabla de imports, cada función puede identificarse por **nombre** (`MessageBoxA`) o por **ordinal** (un número). Los ordinales se usan cuando el nombre no se exporta o para complicar el análisis. Si ves `ORDINAL 1234` en un import, no es un error: es una referencia por número.

Exports

Una DLL puede **exportar** funciones para que otros programas las usen. La tabla de exports tiene nombre, ordinal y RVA de cada función. En un `.exe` normalmente no hay exports; en una DLL sí, y son el punto de entrada para analizarla.

Relocalizaciones y ASLR

Si el cargador no puede cargar la imagen en su `ImageBase` preferida, tiene que **reubicarla**: sumar una diferencia constante a todas las direcciones absolutas. Para eso sirve la sección `.reloc`.

ASLR (*Address Space Layout Randomization*) es la mitigación que hace que la imagen se cargue en una dirección distinta cada vez. Por eso el debugger muestra `0x6D0000` hoy y quizá otra base mañana, y por eso IDA (que analiza el archivo estático) muestra la base preferida. El capítulo 26b lo explica con un ejemplo concreto (`Reversing desde cero, cap. 26b`).

Consecuencia práctica: **no compares direcciones absolutas entre el archivo y la ejecución**. Compará RVA u offsets, o rebaseá el desensamblador a la base que ves en el debugger. El capítulo 14 muestra cómo hacerlo en IDA (`Reversing desde cero, cap. 14`, 00:05:30–00:08:00).

Cómo reconocer un binario empacado

Un empacador comprime o cifra el código original y lo descomprime en tiempo de ejecución. Señales típicas en el PE:

- Nombres de sección como `UPX0`, `UPX1`, `.aspack`, `.packed`.
- Muy pocos imports, o imports raros para un programa de interfaz gráfica.
- Entry point en una sección que no es `.text`.
- Secciones con tamaños virtuales enormes y datos crudos pequeños (el espacio se llena al descomprimir).
- `RWX` en la sección de destino.

El capítulo 10 trabaja con UPX como ejemplo didáctico de desempaquetado. Esta guía **no** incluye el desempaquetado de protectores comerciales como práctica; si querés entender el concepto, estudiá el caso UPX sobre un binario propio y quedate con la idea de "código que se materializa en memoria", no con la receta para saltar protecciones.

Práctica

1. Compilá tu propio `.exe` (el de los dos hilos sirve) y abrilo con un visor de PE: PE-bear, CFF Explorer o Detect It Easy. Anotá:

- `Machine` y `Magic` (¿32 o 64 bits?).
- `ImageBase` y `AddressOfEntryPoint`.
- Secciones con sus RVA, `PointerToRawData` y permisos.
- Tres imports con nombre y la DLL de la que vienen.

2. Calculá el offset del entry point y verificalo en un editor hexadecimal: en ese offset deberían estar los primeros bytes de la función de entrada.
3. Cambiá la vista del visor a "RVA" y a "offset" y comprobá que los números cambian. Explicá por qué.
4. Escribí (o probá) un script en Python que lea la cabecera y muestre `ImageBase`, `AddressOfEntryPoint` y las secciones. Podés usar el módulo estándar `struct`; no necesitás librerías externas (en la lección siguiente, 1.5 · C y Python mínimos, explicamos paso a paso cómo funciona `struct.unpack_from`; acá te dejamos el código listo con comentarios):

```
import struct

with open("programa.exe", "rb") as f:
    data = f.read()

# En el offset 0x3C (e_lfanew) hay un entero de 4 bytes little-endian ("<I") que apunta a "PE\0\0"
pe = struct.unpack_from("<I", data, 0x3C)[0]
assert data[pe:pe+4] == b"PE\0\0"
# Leemos Machine y NumberOfSections (dos enteros cortos de 2 bytes cada uno: "<HH")
machine, nsec = struct.unpack_from("<HH", data, pe+4)
opt_size = struct.unpack_from("<H", data, pe+20)[0]
opt = pe + 24
magic = struct.unpack_from("<H", data, opt)[0]
entry = struct.unpack_from("<I", data, opt+16)[0]
# En PE32 (0x10B) ImageBase es de 4 bytes ("<I"); en PE32+ (0x20B, 64 bits) es de 8 bytes ("<Q")
base = struct.unpack_from("<I", data, opt+28)[0] if magic == 0x10B else struct.unpack_from("<Q",
data, opt+24)[0]
print(f"magic={magic:#x} machine={machine:#x} secciones={nsec}")
print(f"entry RVA={entry:#x} image base={base:#x}")
sec = pe + 24 + opt_size
for i in range(nsec):
    off = sec + i * 40
    name = data[off:off+8].rstrip(b"\0").decode(errors="replace")
    vsize, vaddr, rsize, raw = struct.unpack_from("<IIII", data, off+8)
    print(f"{name:8} RVA={vaddr:#08x} raw={raw:#08x} vsize={vsize:#x}")
```

Preguntas de repaso

1. ¿Qué diferencia hay entre RVA y offset? La RVA es una dirección relativa a la image base, en el espacio de memoria. El offset es una posición dentro del archivo en disco. Se relacionan con la alineación del header y el `PointerToRawData` de la sección.

2. ¿Qué es la IAT? La tabla de direcciones de imports. El cargador la completa con las direcciones reales de las funciones de las DLLs; las llamadas indirectas leen de ahí.

3. ¿Por qué un `call` a `MessageBoxA` no es un `call` a una dirección fija? Porque `MessageBoxA` vive en `user32.dll`, que se carga en una dirección que puede variar. El `.exe` llama a través de la IAT, que el cargador completa.

4. ¿Qué indica `Magic = 0x20B`? Que la cabecera opcional es PE32+, es decir, un binario de 64 bits.

5. ¿Por qué ASLR complica comparar IDA con el debugger? Porque la imagen se carga en una base distinta de la preferida. Las direcciones absolutas cambian; las RVA no. Hay que rebasar o comparar offsets.

6. ¿Un binario con secciones `UPX0` / `UPX1` es malware? No necesariamente: es un binario empacado con UPX. Puede ser un programa legítimo comprimido o un malware que usa el mismo empacador. La señal orienta, no concluye.

Antes de avanzar

Tenés que poder:

- Recorrer la cabecera PE y ubicar entry point, image base y secciones.
- Convertir una VA en offset y verificarla en el archivo.
- Explicar qué es la IAT y leer un `call dword ptr [dirección]`.
- Reconocer las señales de un binario empacado y decir por qué no lo vas a desempaquetar en este curso.

> **Puerta de salida de la Etapa 1**
>
> Si podés hacer el recorrido completo con tu propio `.exe` y explicar cada paso, terminaste los fundamentos. La Etapa 2 empieza con el lenguaje del procesador: assembly.

Fuentes de esta lección

- `Reversing desde cero, cap. 05` — VA, RVA, offset, secciones, call directo e indirecto, IAT.
- `Videos y material adicional, Video 33` — formato PE, secciones, imports, relocalizaciones.
- `Reversing desde cero, cap. 14` — rebase y file offset entre IDA y el debugger.
- `Reversing desde cero, cap. 26b` — ASLR.
- `Reversing desde cero, cap. 10` — UPX como caso de binario empacado.

> **Síntesis de estudio de esta lección**
>
> **Núcleo conceptual:** el ejemplo `0x6D1372 → 0x772`, la IAT en `0x6E300C`, la lectura del PE del video 33 y la explicación de ASLR del capítulo 26b. **Conocimiento externo:** estructura de la cabecera PE y del directorio de imports. **Recomendación editorial:** el script de Python y el límite explícito sobre desempaquetado.

C y Python mínimos para analizar

> **Objetivo de la lección**
>
> Al terminar vas a poder leer y escribir programas chicos en C (variables, condiciones, bucles, funciones, punteros) y en Python (tipos, listas, funciones, archivos binarios), compilarlos, ejecutarlos en el laboratorio y usarlos como conejillos de indias para el debugger.

Prerrequisitos: Etapa 0 y las lecciones anteriores de la Etapa 1. **Duración sugerida:** 6 a 8 horas. Estimación aproximada.

Una advertencia honesta sobre los materiales tradicionales

La carpeta `Python ISO video tutes` parece un curso de Python, pero **no lo es**. Las 55 transcripciones son un curso de **lógica y fundamentos de programación** dictado por Daniel Bermúdez, basado en pseudocódigo y con ejemplos en Basic, Pascal y C. En las transcripciones no hay una sola línea de Python ejecutable: ni `import`, ni `print(...)`, ni `raw_input`. El propio autor anuncia una segunda parte con Python "puro" que **no está en la carpeta**. El material cita Python 2.5.1 como versión vigente en abril de 2007 y usa `print hola` como sentencia, sintaxis que Python 3 ya no acepta.

Eso no lo vuelve inútil. Lo que sí enseña —algoritmos, secuencia, decisión, ciclos, arreglos, matrices y funciones— es exactamente el prerequisito mental para programar. Pero si venías a aprender Python con estos videos, no lo vas a lograr. Para Python moderno, esta guía usa conocimiento externo y prácticas propias, y lo marca como tal.

> **Por qué distinguir lógica de sintaxis**
>
> Los tutoriales clásicos de lógica de programación (como la colección histórica *Python ISO video tutes*) son excelentes para entrenar la mente en secuencias, condicionales, ciclos y arreglos, pero utilizan sintaxis antigua de Python 2.5 o pseudocódigo. En este curso aprovechamos esa base lógica y escribimos todo el código real directamente en **Python 3 moderno** y **C estándar**.

Cómo aprender Python de verdad (recursos externos)

Como el corpus tradicional no enseña Python moderno, esta guía completa el camino con **cursos y tutoriales externos**. No son material del grupo; son **recomendación editorial**, gratuitos o con parte gratuita, y actuales. Verificá siempre el enlace antes de usarlo.

Recurso	Qué es	Para quién
El Tutorial de Python (oficial) — docs.python.org/3/tutorial	La referencia gratuita y al día; Python 3	Base sólida, consulta permanente
Automate the Boring Stuff (Al Sweigart) — automatetheboringstuff.com	Libro gratuito online, muy práctico, pensado para Windows	Quien nunca programó y quiere resultados rápidos
Real Python — realpython.com	Tutoriales por tema (archivos, bytes, struct, módulos)	Afianzar temas puntuales
Python Crash Course (Eric Matthes)	Libro de pago, proyecto guiado	Quien prefiere un recorrido ordenado
freeCodeCamp — Python (YouTube)	Curso en video gratuito	Quien aprende mejor mirando

La ruta mínima que necesitás para esta guía (no hace falta dominar todo Python): tipos y variables → `str` vs `bytes` → listas y diccionarios → `if` / `for` / `while` → funciones y módulos (`import`) → leer y escribir archivos **binarios** → el módulo `struct` para desempaquetar enteros. Con eso ya podés escribir las herramientas que usa el resto del curso.

> **El puente con la lógica de programación**
>
> Lo que los tutoriales tradicionales de lógica sí te dejan —secuencia, decisión, ciclos, arreglos, funciones— es el prerrequisito mental. Si lo mirás, usalo para **la lógica**, y aprendé la **sintaxis de Python 3** con los recursos de arriba. Es la combinación que te lleva al siguiente tema: scripts que leen binarios.

El subconjunto de C que vas a leer en un debugger

No necesitás ser experto en C. Necesitás reconocer cinco construcciones:

1. **Variables y tipos:** `int`, `char`, `unsigned`, punteros (`int *p`).
2. **Condiciones:** `if`, `else`, `switch`.
3. **Bucles:** `for`, `while`.
4. **Funciones:** parámetros, valor de retorno, variables locales.
5. **Punteros y arrays:** `&x` (dirección de), `*p` (contenido de), `a[i]`.

Ejemplo paso a paso: el programa del capítulo 15

El capítulo 15 analiza un crackme de práctica que pide dos números. La lógica que Ricardo deduce del listado es:

- el primer número tiene que ser **menor o igual** que `0x32` (50 decimal);
- el segundo tiene que ser **mayor o igual** que `0x32`;
- la suma de los dos, multiplicada por 4, tiene que dar `0x1E0` (480 decimal).

Un programa equivalente, escrito por vos, es este:

```c
#include <stdio.h>

int main(void) {
    int a, b;
    printf("Primer valor: ");
    if (scanf("%d", &a) != 1) return 1;
    printf("Segundo valor: ");
    if (scanf("%d", &b) != 1) return 1;

    if (a > 50 || b < 50) {
        puts("No cumple las condiciones");
        return 0;
    }
    if ((a + b) * 4 == 480) {
        puts("Correcto");
    } else {
        puts("Incorrecto");
    }
    return 0;
}
```

Cómo se lee, línea por línea:

- `int a, b;` reserva dos enteros en el stack.
- `scanf("%d", &a)` lee un decimal y lo guarda en la dirección de `a`. El `&` es la dirección; sin él, la función no podría modificar la variable.
- `a > 50 || b < 50` usa `||` (OR lógico): si cualquiera de las dos condiciones se cumple, rechaza.
- `(a + b) * 4 == 480` reproduce la comparación del binario. Con `a = 40` y `b = 80`, da `(120) * 4 = 480`, así que imprime `Correcto`.

Compilación (en el guest, con MinGW o Visual Studio):

```
gcc -O0 -g -o check.exe check.c
```

`-O0` desactiva optimizaciones para que el assembly sea fácil de seguir; `-g` agrega información de depuración. En Visual Studio, compilá en modo Debug.

Qué mirar en el debugger: el `if (a > 50 || b < 50)` se convierte en dos comparaciones y saltos; el `(a+b)*4` se convierte en un `add` seguido de un `shl eax, 2` (desplazar dos bits a la izquierda es multiplicar por 4). Eso conecta directo con la lección de instrucciones.

El subconjunto de Python que vas a usar como herramienta

Python no compila a un `.exe` nativo (en el sentido clásico), así que no es el lenguaje que vas a ver en un debugger de Windows. Es la **navaja suiza** para procesar archivos, calcular hashes, parsear cabeceras y automatizar tareas.

Ejemplo: leer los primeros bytes de un ejecutable

```
from pathlib import Path

data = Path("check.exe").read_bytes()
print("Tamaño:", len(data))
print("Primeros dos bytes:", data[:2])          # b'MZ'
print("Primeros 16 bytes:", data[:16].hex(" "))
```

Salida esperada (los valores exactos dependen del compilador):

```
Tamaño: 53248
Primeros dos bytes: b'MZ'
Primeros 16 bytes: 4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00
```

`read_bytes()` devuelve un objeto `bytes`: una secuencia inmutable de valores entre 0 y 255. `data[:2]` toma los dos primeros; `.hex(" ")` los muestra en hexadecimal separados por espacios. No hace falta instalar nada.

Ejemplo: calcular un SHA-256

```
import hashlib
from pathlib import Path

datos = Path("check.exe").read_bytes()
print(hashlib.sha256(datos).hexdigest())
```

Ese hash es el mismo que devuelve `Get-FileHash` en PowerShell. Sirve para la bitácora de la Etapa 0.

Ejemplo: desempaquetar un entero con `struct`

Los bytes sueltos no alcanzan: para leer una cabecera hay que interpretar grupos de bytes como números. Eso lo hace el módulo `struct`. En un PE, en el offset `0x3C` hay un entero de 4 bytes (little-endian) que apunta a dónde empieza la firma `PE\0\0`:

```
import struct
from pathlib import Path

data = Path("check.exe").read_bytes()
(pe_off,) = struct.unpack_from("<I", data, 0x3C)   # "<I" = entero de 4 bytes, little-endian
print("La cabecera PE empieza en:", hex(pe_off))
print("Firma en ese offset:", data[pe_off:pe_off+4])  # b'PE\x00\x00'
```

`struct.unpack_from("<I", data, 0x3C)` lee 4 bytes desde el offset `0x3C` y los arma como un entero sin signo (`I`) en little-endian (`<`). Esto conecta directo con la lección de PE: lo que ahí leés a mano, acá lo automatizás.

Ejemplo: el mismo cálculo del crackme, en Python

```
a = int(input("Primer valor: "))
b = int(input("Segundo valor: "))
if a <= 50 and b >= 50 and (a + b) * 4 == 480:
    print("Correcto")
else:
    print("Incorrecto")
```

Compará las dos versiones: en C el `scanf` recibe una dirección; en Python `input()` devuelve un string y `int()` lo convierte. En C la condición se escribe con `||` y `&&`; en Python, con `or` y `and`. La lógica es la misma.

> **Python 2 vs Python 3**
>
> Si encontrás material viejo con `print "hola"` o `raw_input`, es Python 2. Hoy se usa Python 3: `print("hola")`, `input()`, divisiones con `/` que devuelven float y `//` para entero. El curso de reversing es de esa época; no copies esa sintaxis.

Hacia dónde sigue este camino

No hace falta parsear cada cabecera a mano con `struct`: hay librerías que ya lo hacen. El camino natural desde acá es:

1. `pefile` — lee cabeceras PE, secciones e imports de un ejecutable de Windows con unas pocas líneas.
2. `capstone` — desensambla bytes a instrucciones x86/x64 desde Python.
3. **IDAPython / Ghidra scripting** — automatizar el análisis dentro del desensamblador.

Esos tres se desarrollan, con ejemplos, en la rama de automatización de la Etapa 5. Lo de esta lección — `bytes`, `hex`, `struct`, archivos— es la base que te deja usarlos sin que sean magia.

Práctica

1. Compilá el programa `check.c` y probalo con `40 80`, `50 50`, `51 80` y `40 79`. Anotá qué imprime cada caso.
2. Abrilo en x64dbg, poné un breakpoint en `main` y seguí con F8 hasta la comparación. Anotá en qué instrucción se convierte `(a+b)*4`.
3. Escribí un programa en C que pida una palabra y la compare con `"clase"` usando `strcmp`. Compilalo y encontralo en el debugger.
4. Escribí un script en Python que recorra una carpeta, calcule el SHA-256 de cada archivo y escriba un CSV con `ruta,hash`. Usalo sobre tu carpeta de prácticas.
5. Modificá el script para que lea la cabecera PE y muestre `ImageBase` y `AddressOfEntryPoint` (podés reutilizar el de la lección de PE). Usá `struct.unpack_from` sobre los offsets correctos.

6. Instalá `pefile` (`pip install pefile`) y obtené los mismos valores con la librería (`pe.OPTIONAL_HEADER.ImageBase`, `pe.OPTIONAL_HEADER.AddressOfEntryPoint`). Compará con tu versión a mano: ¿coinciden? Así ves qué te ahorra la librería.
7. Hacé el Tutorial oficial de Python (secciones 3 a 7) o los primeros capítulos de *Automate the Boring Stuff*, y rehacé el script del item 4 sin mirar el ejemplo.

Preguntas de repaso

1. ¿Por qué `scanf` necesita `&a` y `printf` no? Porque `scanf` tiene que **escribir** en la variable, y para eso necesita su dirección. `printf` solo lee el valor.

2. ¿Qué diferencia hay entre `int` y `unsigned int`? El rango y la interpretación. `int` usa complemento a dos e incluye negativos; `unsigned int` solo representa valores no negativos y puede llegar al doble del máximo positivo.

3. ¿Por qué `x << 2` equivale a multiplicar por 4? Porque cada desplazamiento a la izquierda multiplica por 2. Dos desplazamientos multiplican por 4. Es la operación que usa el compilador para optimizar multiplicaciones por potencias de dos.

4. ¿Python sirve para analizar un `.exe`? Sí, como herramienta: leer bytes, parsear cabeceras, calcular hashes, automatizar. No es el lenguaje del binario que analizás, salvo que el programa esté hecho en Python y empaquetado con un generador de ejecutables.

5. ¿Los tutoriales tradicionales de lógica enseñan Python moderno? No. Enseñan lógica de programación con pseudocódigo y ejemplos en Basic, Pascal y C. Son útiles como base mental, pero no reemplazan un curso de Python 3 moderno.

Antes de avanzar

Tenés que poder:

- Leer un programa en C de 20 líneas y decir qué hace.
- Escribir, compilar y ejecutar un programa propio en C.
- Leer y escribir un archivo binario en Python y calcular su hash.
- Explicar por qué los tutoriales tradicionales de lógica no son un curso de Python moderno.

> **Puerta de salida de la Etapa 1**
>
> Con esto terminás los fundamentos. Si podés escribir tus propios binarios, ya tenés con qué practicar todo lo que sigue sin depender de software ajeno.

Fuentes de esta lección

- `Python ISO video tutes, Fundamentos de programación` — qué es programar, intérprete vs compilador, `print` en Python 2.
- `Python ISO video tutes, Qué es Python` — presentación del curso y versión de la época.

- `Python ISO video tutes, Arreglos` — ejemplo del tipo de contenido real de la colección.
- `Reversing desde cero, cap. 15` — lógica del programa de práctica y comparación con `0x32` / `0x1E0`.

Síntesis de estudio de esta lección

Núcleo conceptual: la lógica del programa del capítulo 15 y el contenido real de las transcripciones de Python ISO. **Conocimiento externo:** sintaxis de C y Python 3, compilación, hash, `struct`, y los recursos de aprendizaje externos (Tutorial oficial, Automate the Boring Stuff, Real Python, etc.). **Recomendación editorial:** los programas de práctica, la corrección sobre el curso de Python de reversing y la ruta de Python hacia `pefile` / `capstone`.

Etapa 2 · Introducción al reversing

Objetivo medible

Leer assembly x86/x64, seguir el flujo y usar debugger y desensamblador con criterio. Se divide en 2A (base, 2.1–2.9), 2B (Windows intermedio, 2.10–2.16: ofuscación, TLS, MFC, huella, PE fino, anti-análisis, Ghidra) y 2C (Windows avanzado, 2.17+: packers didácticos, C++ profundo, WinDbg/heap, patch diffing, scripting) y 2E (desofuscación y análisis dinámico avanzado, 2.25+: ejecución simbólica, ofuscación por VM, protectores multicapa, neutralizar anti-análisis, DBI con DynamoRIO/Pin y Frida Stalker, extensiones de Ghidra/SLEIGH, y ampliaciones de Go/Rust y VB6). Las rutas de malware dinámico, shellcode/C2 y fuzzing quedan como optativas, fuera de la puerta obligatoria.

Prerrequisitos: Etapa 1. **Duración estimada:** 32 a 40 h (2A) + ~25 h (2B) + ~30–40 h (2C) + ~25–30 h (2E). Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración rígida de laboratorio.

Lecciones

#	Lección	Qué aprendés
2.1	Registros, subregistros y flags	Registros, solapamiento y flags ZF/SF/CF/OF.
2.2	Instrucciones: mover, calcular y manipular bits	MOV, LEA, aritmética, lógica, shifts y cadenas.
2.3	Punto flotante y SSE: los registros xmm	x87, SSE escalar (movsd/addsd/ucomisd) e IEEE 754.
2.4	Flujo: comparar, saltar y repetir	CMP/TEST, Jcc, CMOVcc, bucles y tablas de salto.
2.5	Llamadas, convenciones y marcos de función	CALL/RET, cdecl/stdcall/fastcall y x64.
2.6	El debugger: x64dbg de punta a punta	Interfaz, F7/F8/F9, breakpoints y parcheo ético.
2.7	IDA y el análisis estático	Strings, xrefs, grafo, rebase y decompilador.
2.8	Cómo cambia el código con optimizaciones	Inlining, strength reduction, división por constante y tail calls.
2.9	Herramientas: del OllyDbg histórico al taller actual	Qué usar para cada tarea y qué está discontinuado.
2.10	Ofuscación nativa: leer lo que no quiere ser leído	Secciones invertidas, basura, saltos al medio y strings ocultas; hallar el bloque real.
2.11	TLS: código que corre antes de main	Directorio TLS, AddressOfCallBacks y callbacks previos al entry point.
2.12	MFC: llegar al handler cuando todo pasa por clases	Reconocer MFC y alcanzar el handler de un control por el message map.
2.13	Huella de compilador: saber con qué se hizo	MSVC, MinGW, Delphi, VB6 y MFC por imports, strings y runtime.
2.14	PE fino: los directorios que cuentan la historia	Rich header, recursos, delay-load, .pdata, debug/PDB y Load Config.
2.15	Taxonomía anti-análisis: reconocer, no evadir	Anti-debug, anti-VM, anti-disassembly y anti-dump: detectarlos.
2.16	Ghidra (y Cutter): la alternativa libre, con práctica	El recorrido de IDA replicado en Ghidra: autoanálisis, strings, tipos y decompilador.
2.17	Packers didácticos: UPX, stub, OEP, dump e IAT	Desempaquetado didáctico de UPX sobre binarios propios; llegar al OEP y reconstruir la IAT.
2.18	C++ profundo: RTTI, vtables, STL y excepciones	RTTI, std::string con SSO, contenedores STL y manejo de excepciones.
2.19	WinDbg, TTD y heap	WinDbg, time-travel debugging y diagnóstico de heap (Page Heap, LFH).
2.20	Patch diffing y CVE: leer qué cambió	BinDiff/Diaphora: comparar dos versiones y leer qué arregló un parche (defensivo).

#	Lección	Qué aprendés
2.21	Scripting de debugger: automatizar y dejar evidencia	Automatizar x64dbg (breakpoints, logs) y guardar evidencia; OllyScript como legado.
2.22	(optativa) Malware dinámico defensivo	Laboratorio aislado, red simulada, observación de comportamiento e IOCs.
2.23	(optativa) Shellcode, red y C2: reconocer y describir	Identificar shellcode y describir tráfico de C2; análisis y clasificación, no construcción.
2.24	(optativa) Fuzzing y análisis simbólico	Fuzzear un parser/crackme propio y usar ejecución simbólica para hallar condiciones.
2.25	Ejecución simbólica y concólica	angr/Triton/Miasm y Z3 para recuperar condiciones y simplificar MBA en binarios propios.
2.26	Ofuscación por máquina virtual	Dispatcher, bytecode y handlers; reconocer virtualización y practicar con una mini-VM propia.
2.27	Protectores comerciales y packers multicapa	Reconocer familias y capas (VMProtect, Themida…) sin desempacar software ajeno.
2.28	Neutralizar anti-análisis en laboratorio	ScyllaHide y hooks de Frida sobre binarios propios o malware en VM descartable.
2.29	Instrumentación binaria dinámica: DynamoRIO y Pin	Cobertura (drcov/Lighthouse), conteo y desempaquetado genérico por W⊕X.
2.30	Frida avanzado: Stalker y tracing	Trazar bloques/instrucciones de un hilo, cobertura (frida-drcov) y código en runtime.
2.31	Extensiones de Ghidra y SLEIGH	Scripts (Jython/PyGhidra), headless, P-Code y SLEIGH para modelar bytecode propio.
2.32	Go y Rust en profundidad (amplía 2.13)	pclntab/itab en Go; manglado v0, monomorfización y trait objects en Rust.
2.33	VB6 y p-code (amplía 2.13)	Nativo vs p-code, VBHeader y MSVBVM60; VB Decompiler/WKTVBDE (legado).

Recursos y lecturas de referencia

- Etapa 2 · Rama optativa 2D — Windows internals (después de 2C)
- Reversing desde cero (tronco principal)
- Curso IDA desde cero (69 documentos)
- Curso de OllyDbg (Videos 23–40)
- Introducción al reversing con IDA Pro (docx)

Evaluación de salida

- **2A** · Explicar qué hace un bloque de 10 instrucciones y qué flags cambia.
- **2A** · Reconstruir un if/else y un bucle a partir del listado.

- **2A** · Poner los tres tipos de breakpoint y justificar cuál usar.
- **2A** · Sincronizar IDA y x64dbg por base o por file offset.
- **2B** · Reconocer tres señales de ofuscación y hallar el bloque real sin NOPear a ciegas.
- **2B** · Encontrar un TLS callback en el PE y demostrar que corre antes de `main`.
- **2B** · Llegar al handler de un control en un binario MFC por una API de texto y el call stack.
- **2B** · Identificar el compilador de un binario y navegar los directorios finos del PE (Rich header, Load Config, debug/PDB).
- **2B** · Clasificar una técnica anti-análisis y detectarla, y leer el mismo binario en IDA y en Ghidra.
- **2C** · Desempaquetar un UPX propio: llegar al OEP, dumpear, reconstruir la IAT y validar con `upx -d`.
- **2C** · Reconocer RTTI, `std::string` (SSO), un `std::vector` y un `try/catch` en un binario C++ propio.
- **2C** · Diagnosticar un bug de heap propio con WinDbg/Page Heap y leer, con un differ, qué cambió un parche.
- **2C** · Automatizar una tarea de análisis con un script de x64dbg y guardar la evidencia.
- **2E** · Resolver un crackme propio con un solver (angr/Z3), reconocer una VM de protección y sacar cobertura con DBI o Frida Stalker.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Confundir F7 con F8 y creer que F8 no ejecuta la función.
- Leer un salto al revés (pensar que el salto es la condición verdadera).
- Usar una copia ilegal de IDA Pro.
- Parchear software ajeno para saltar una licencia.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Registros, subregistros y flags

> **Objetivo de la lección**
>
> Al terminar vas a poder nombrar los registros de propósito general y su función típica, descomponer un registro de 32 bits en sus partes de 16 y 8 bits, y explicar qué flag se activa en una suma o resta concreta.

Prerrequisitos: Etapa 1 completa. **Duración sugerida:** 3 horas. Estimación aproximada.

Qué es un registro

Un **registro** es una porción de memoria pequeña y muy rápida que está dentro del procesador. Se usa para operar con los datos que la CPU necesita en el momento. En 32 bits cada registro mide 4 bytes (32 bits); en 64 bits, 8 bytes (64 bits).

El capítulo 03 lo define así: "una porción de memoria pequeña, ultra rápida, que forma parte del procesador y que sirve para intercambiar datos" (`Reversing desde cero, cap. 03`, 00:00:30).

Los registros de propósito general

Registro 32	Registro 64	Nombre	Uso típico
`EAX`	`RAX`	Acumulador	Resultado de funciones y operaciones
`EBX`	`RBX`	Base	Datos generales; a veces puntero a datos
`ECX`	`RCX`	Contador	Bucles, `rep`, shifts variables
`EDX`	`RDX`	Datos	Parte alta de multiplicaciones/divisiones; E/S
`ESI`	`RSI`	Source index	Origen en copias de memoria
`EDI`	`RDI`	Destination index	Destino en copias de memoria
`ESP`	`RSP`	Stack pointer	Cima del stack
`EBP`	`RBP`	Base pointer	Base del marco de la función

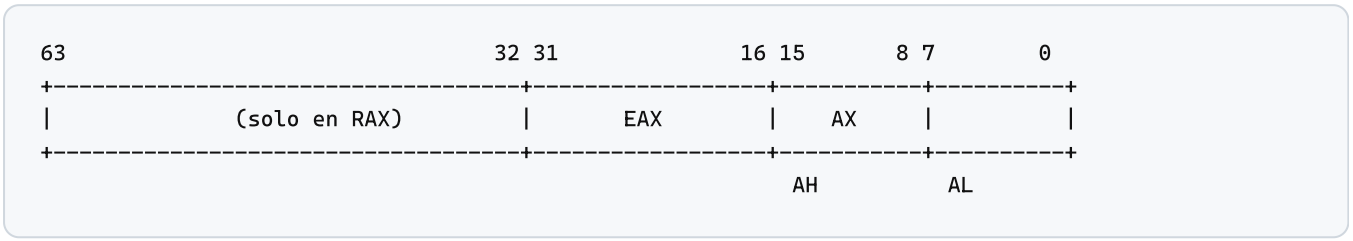
Además, `EIP` / `RIP` es el **instruction pointer**: apunta a la instrucción que se va a ejecutar. No se modifica con `mov`; lo cambia el flujo (saltos, llamadas, retornos).

El capítulo 03 asigna funciones concretas: `EAX` guarda el resultado de las funciones, `ECX` actúa como contador en bucles y operaciones repetitivas, `ESI` / `EDI` se usan para copiar bloques de memoria, `ESP` apunta a la cima del stack y `EBP` a la base del marco (00:01:00–00:03:00). Son convenciones, no reglas absolutas: un compilador puede usar `EAX` para cualquier cosa.

Subregistros: el mismo espacio, distintos tamaños

Cuando necesités trabajar con un dato de solo 1 o 2 bytes (por ejemplo, una letra ASCII o un entero corto), el procesador no usa un registro aparte: te permite abrir **ventanas más chicas** sobre la parte derecha (los bits bajos) del mismo registro de 32 o 64 bits. Por eso modificar `AL` o `AX` modifica directamente el interior de `EAX`:

Para `EAX`:



Ejemplo con `EAX = 0x008FFB18`:

Vista	Valor	Tamaño
`EAX`	`0x008FFB18`	32 bits
`AX`	`0xFB18`	16 bits (los dos bytes bajos)
`AH`	`0xFB`	8 bits (byte alto de `AX`)
`AL`	`0x18`	8 bits (byte bajo de `AX`)

Esto se aplica a `EAX`, `EBX`, `ECX` y `EDX`. `ESI`, `EDI`, `EBP` y `ESP` tienen versión de 16 bits (`SI`, `DI`, `BP`, `SP`) pero no se dividen en dos de 8. El capítulo 03 lo explica con este mismo ejemplo (00:04:30–00:09:00).

En 64 bits, `RAX` contiene a `EAX`, que contiene a `AX`, que contiene a `AL` / `AH`. Además hay ocho registros nuevos, `R8` a `R15`, con subregistros `R8D` (32), `R8W` (16) y `R8B` (8).

> **Escribir en la parte baja no borra la alta (con matices)**
>
> En x86, escribir en `AL` no cambia el resto de `RAX`. Pero en x64, escribir en `EAX` **sí borra los 32 bits altos de** `RAX`. Es una diferencia que sorprende cuando comparás el código de 32 y 64 bits. Para el análisis inicial, alcanza con saber que los tamaños se solapan y que el debugger te muestra cada vista.

Flags: el resumen de la última operación

Los **flags** son bits de estado que se activan según el resultado de una operación. No son registros de datos: son banderas. Los cuatro que importan en el uso diario:

Flag	Nombre	Se activa cuando...
`ZF`	Zero flag	El resultado es cero
`SF`	Sign flag	El resultado es negativo (bit más alto en 1)
`CF`	Carry flag	Hubo acarreo/desborde en una operación **sin signo**
`OF`	Overflow flag	Hubo desborde en una operación **con signo**

Hay más (`PF` de paridad, `AF` auxiliar, `DF` de dirección), pero casi nunca los mirás al reversear.

Ejemplo 1: `ZF`

```
mov eax, 3
sub eax, 3      ; resultado 0 -> ZF = 1
```

Si en cambio hacés `mov eax, 0`, el flag `ZF` **no** se activa: `mov` no es una operación aritmética y no actualiza los flags. El capítulo 04 lo muestra explícitamente (`Reversing desde cero, cap. 04`, 00:00:30–00:02:00).

Ejemplo 2: `CF`

```
mov eax, 0xFFFFFFFF   ; máximo sin signo
add eax, 5            ; resultado 4, con acarreo
```

El resultado "real" (4.294.967.300) no entra en 32 bits. El procesador guarda `4` y activa `CF`. Para valores sin signo, eso es un desborde (cap. 04, 00:05:00–00:06:00).

Ejemplo 3: `OF` y `SF`

```
mov eax, 0x7FFFFFFF   ; máximo positivo con signo
add eax, 1            ; pasa a 0x80000000
```

El resultado cambió de positivo a negativo: para valores con signo es un desborde, así que se activa `OF`. Además, como el bit más alto quedó en 1, se activa `SF` (cap. 04, 00:07:30–00:09:00).

La regla de oro para no confundirte nunca entre `CF` y `OF`: **la instrucción `add` o `sub` no sabe si tu variable en C era `unsigned int` o `int` con signo**, así que hace la cuenta una sola vez y calcula **ambas banderas al mismo tiempo**. Si pensabas el dato como sin signo (0 a 4.294 millones), mirás `CF` (como un cuentakilómetros que pasa de `FFFFFFFF` a `00000000`); si lo pensabas con signo, mirás `OF` (que avisa cuando sumás dos positivos y el bit 31 se prende dando negativo). La instrucción de salto que venga después (que veremos en la lección 2.4) es la que elige cuál de las dos banderas mirar.

Multiplicación y división: registros dobles

`MUL` y `DIV` usan `EDX:EAX` como un registro de 64 bits:

- `MUL operando`: multiplica `EAX` por el operando y guarda el resultado en `EDX:EAX`.
- `DIV operando`: divide `EDX:EAX` por el operando; cociente en `EAX`, resto en `EDX`.

Ejemplo del capítulo 04: `EAX = 4`, `EBX = 2`, `MUL EBX` deja `EAX = 8` y `EDX = 0`. Si el resultado no entra en 32 bits, la parte alta aparece en `EDX` (00:13:00–00:17:00).

`IMUL` / `IDIV` son las versiones con signo. La mecánica es la misma; cambia la interpretación de los bits.

Práctica

1. Completá la tabla para `EBX = 0x11223344`:

Vista	Valor
`EBX`	
`BX`	
`BH`	
`BL`	

2. ¿Qué flag se activa en cada caso?

- `sub eax, eax`
- `add eax, 1` con `EAX = 0xFFFFFFFF`
- `add eax, 1` con `EAX = 0x7FFFFFFF`
- `mov eax, 0`

3. Escribí en x64dbg `mov eax, 0x7FFFFFFF` y `add eax, 1`. Anotá `ZF`, `SF`, `CF` y `OF` antes y después.
4. Explicá por qué `mov` no cambia los flags y `add` sí.
5. ¿Qué registro usarías para contar las vueltas de un bucle? ¿Por qué no `ESP`?

Preguntas de repaso

1. ¿`AX` es un registro distinto de `EAX`? No. Es la parte baja de 16 bits de `EAX`. Escribir en `AX` modifica los dos bytes bajos de `EAX` y deja los altos intactos.

2. ¿Qué diferencia hay entre `CF` y `OF`? `CF` indica desborde en una interpretación sin signo; `OF`, en una interpretación con signo. El mismo resultado puede activar uno, otro o ambos.

3. ¿`EIP` se puede cambiar con `mov`? No directamente. Se cambia con instrucciones de control de flujo (`jmp`, `call`, `ret`, `jcc`) o con el debugger (por ejemplo, "set new origin here").

4. ¿Para qué sirve `EDX` en una división? Para la parte alta del dividendo y para el resto. `DIV` divide `EDX:EAX` por el operando y deja el cociente en `EAX` y el resto en `EDX`.

5. ¿Por qué `mov eax, 0` no activa `ZF`? Porque `mov` es una transferencia de datos, no una operación aritmética. Los flags se actualizan con instrucciones como `add`, `sub`, `cmp`, `test`, `and`, etc.

Antes de avanzar

Tenés que poder:

- Nombrar los ocho registros de propósito general y su función típica.
- Descomponer `EAX` en `AX`, `AH` y `AL` con un ejemplo.
- Explicar `ZF`, `SF`, `CF` y `OF` con un caso cada uno.
- Decir qué registros usa `MUL` / `DIV` para el resultado.

> **Puerta de salida**
>
> Si podés predecir los flags de una operación antes de ejecutarla, estás listo para las instrucciones. Si no, practicá con la calculadora del debugger y papel.

Fuentes de esta lección

- `Reversing desde cero, cap. 03` — registros, subregistros, flags y números con signo.
- `Reversing desde cero, cap. 04` — ejemplos de `ZF`, `CF`, `OF`, `SF`, `MUL` y `DIV`.
- `Videos y material adicional, Video 5` — registros de 32 bits y prefijo `E` en el curso de MASM32.

> **Síntesis de estudio de esta lección**
>
> **Núcleo conceptual:** la definición de registro, el ejemplo `EAX = 0x008FFB18` y los ejemplos de flags de los capítulos 03 y 04. **Conocimiento externo:** la tabla de registros de 64 bits y la semántica exacta de `CF` / `OF`. **Recomendación editorial:** los ejercicios y la advertencia sobre el borrado de la parte alta en x64.

Instrucciones: mover, calcular y manipular bits

> **Objetivo de la lección**
>
> Al terminar vas a poder leer un bloque de instrucciones y decir qué valor queda en cada registro, distinguiendo transferencias, operaciones aritméticas, lógicas, desplazamientos y accesos a memoria.

Prerrequisitos: registros y flags. **Duración sugerida:** 5 horas. Estimación aproximada.

Cómo se lee una instrucción

Casi todas las instrucciones tienen la forma:

```
MNEMÓNICO destino, fuente
```

La operación se calcula con los dos operandos y el resultado se guarda en el **destino** (el primero). `mov eax, ebx` copia `EBX` en `EAX`; `add eax, ebx` suma y guarda en `EAX`.

Los corchetes significan "el contenido de esta dirección":

```
mov eax, 0x401000      ; EAX = el número 0x401000
mov eax, [0x401000]    ; EAX = lo que hay en la dirección 0x401000
```

Esa diferencia es la que separa un puntero de un valor. El capítulo 02 la explica al mover un valor desde memoria a `EAX`: los bytes `36 00 00 38` en la dirección se convierten en `0x38000036` en el registro (`Reversing desde cero, cap. 02`, 00:07:00–00:08:00).

Transferencias

`MOV`

```
mov eax, 5          ; inmediato -> registro
mov eax, ebx        ; registro -> registro
mov eax, [0x403000] ; memoria -> registro
mov [0x403000], eax ; registro -> memoria
```

No modifica flags. No se puede `mov` de memoria a memoria: hay que pasar por un registro.

`LEA`

`LEA` (*load effective address*) calcula una dirección y la guarda, **sin leer memoria**:

```
lea eax, [ebx + ecx*4 + 8]
```

Si `EBX = 0x1000` y `ECX = 2`, `EAX = 0x1000 + 8 + 8 = 0x1010`. No importa qué haya en `0x1010`: no se lee. Se usa para aritmética de punteros y, de paso, como una multiplicación rápida. El capítulo 12 lo muestra en la práctica (`Reversing desde cero, cap. 12`, 00:08:07).

`MOVSX` y `MOVZX`

Copian un valor más chico a un registro más grande, rellenando los bits altos:

```
; AX = 0x8384
movsx eax, ax   ; EAX = 0xFFFF8384  (rellena con el bit de signo)
movzx eax, ax   ; EAX = 0x00008384  (rellena con ceros)
```

`MOVSX` interpreta el origen como con signo; `MOVZX`, como sin signo. En el capítulo 12, `0x8384` es mayor que `0x7FFF`, así que como `WORD` con signo es negativo y `MOVSX` rellena con `FF` (00:07:21–00:08:00).

`XCHG`

Intercambia los dos operandos. No hay destino ni fuente: los dos cambian.

```
; EAX = 0x20, EBX = 0x40
xchg eax, ebx
; EAX = 0x40, EBX = 0x20
```

> **En el Curso IDA: editar una instrucción (y qué pasa después)**
>
> El Curso IDA edita estas instrucciones desde **Edit → Patch Program → Assemble** sobre el crackme de Cruehead. Un detalle útil: si la instrucción nueva ocupa **menos** bytes que la vieja, el byte que sobra queda mostrado como **dato** (`db 0xC0`) y la función aparece "corrupta"; se lo reemplaza con un `NOP` (`0x90` , relleno que no hace nada) para que vuelva a verse como código. `LEA` , acá arriba, el curso la usa justamente para obtener la dirección de variables y argumentos (Curso IDA · tute 4, Transferencia y tute 5, `LEA`).

Aritmética

`ADD` y `SUB`

```
add eax, ebx    ; EAX = EAX + EBX
sub eax, 4      ; EAX = EAX - 4
```

Actualizan `ZF`, `SF`, `CF` y `OF`. El capítulo 02 las practica con registros y memoria (00:10:00–00:18:00).

`INC` y `DEC`

Suman o restan 1. Son más cortas que `add eax, 1` y **no modifican** `CF` (sí `ZF`, `SF`, `OF`). En el capítulo 06 se ve `INC EAX` con `EAX = 0xFFFFFFFF` (pasa a 0) y `DEC EAX` con `EAX = 1` (queda 0 y activa `ZF`) (`Reversing desde cero, cap. 06`, 00:20:03–00:21:00).

`MUL`, `IMUL`, `DIV`, `IDIV`

Ya las vimos en registros. Para 32 bits: `MUL` usa `EDX:EAX`; `DIV` divide `EDX:EAX`. `IMUL` / `IDIV` son las versiones con signo. En 64 bits, `RDX:RAX`.

Lógica de bits

Instrucción	Qué hace	Ejemplo (8 bits)
`AND`	1 solo si ambos bits son 1	`0x11 & 0x0F = 0x01`
`OR`	1 si alguno es 1	`0x11 OR 0x0F = 0x1F`
`XOR`	1 si son distintos	`0x11 ^ 0x0F = 0x1E`
`NOT`	Invierte todos los bits	`~0x11 = 0xEE` (8 bits) o `0xFFFFFFEE` (32)

El capítulo 08 las explica con tablas de verdad y las verifica en el debugger (`Reversing desde cero, cap. 08`, 00:07:00–00:17:00).

`XOR EAX, EAX` es la forma idiomática de poner un registro en cero: ocupa 2 bytes y activa `ZF`. `MOV EAX, 0` ocupa 5 bytes y no toca flags. Esa diferencia de tamaño es la que hace que los compiladores prefieran `XOR`.

Desplazamientos

```
shl eax, 1    ; desplaza 1 bit a la izquierda = multiplicar por 2
shr eax, 1    ; desplaza 1 bit a la derecha  = dividir por 2 (sin signo)
sar eax, 1    ; desplazamiento aritmético (conserva el signo)
rol eax, 4    ; rota a la izquierda: el bit que sale entra por la derecha
ror eax, 4    ; rota a la derecha
```

Con `EAX = 0x11`:

```
shl eax, 2   -> 0x44     (0x11 * 4)
shr eax, 2   -> 0x04     (0x11 / 4, parte entera)
```

`SHL` / `SHR` **pierden** los bits que salen; `ROL` / `ROR` los reinyectan por el otro extremo. El capítulo 08 lo muestra con `0x11 << 2 = 0x44` y `0x44 >> 2 = 0x11` (00:24:33–00:28:11).

Comparar sin restar: `CMP` y `TEST`

`CMP a, b` hace `a - b`, actualiza flags y **descarta el resultado**. `TEST a, b` hace `a AND b`, actualiza flags y también descarta el resultado.

```
cmp eax, 4      ; ¿EAX == 4?  -> ZF=1 si son iguales
test eax, eax   ; ¿EAX == 0?  -> ZF=1 si es cero
```

`TEST EAX, EAX` es la forma habitual de preguntar "¿este registro es cero?" sin modificar nada. El capítulo 09 lo explica y lo usa con `JZ` (`Reversing desde cero, cap. 09`, 00:31:32–00:33:00).

Pila y cadenas

```
push eax        ; ESP -= 4 ; [ESP] = EAX
pop  ebx        ; EBX = [ESP] ; ESP += 4
pushad          ; guarda EAX, ECX, EDX, EBX, ESP, EBP, ESI, EDI (32 bytes)
popad           ; los restaura
```

`PUSHAD` / `POPAD` se usan para preservar registros alrededor de una llamada. Dato fino: `POPAD` **no** restaura `ESP`, aunque `PUSHAD` lo haya guardado (cap. 12, 00:21:07).

Las instrucciones de cadena operan sobre `ESI` (origen) y `EDI` (destino):

```
rep movsb      ; copia ECX bytes de [ESI] a [EDI]
repe cmpsb     ; compara [ESI] y [EDI] mientras sean iguales
lodsb          ; AL = [ESI]; ESI++
stosb          ; [EDI] = AL; EDI++
repne scasb    ; busca AL en [EDI] mientras no lo encuentre
```

El capítulo 12 las recorre con un ejemplo concreto: `ESI` apunta a `"core"` (`0x402012`), `EDI` a `"core security"` (`0x402031`) y `REPNE SCASB` busca el byte `0x65` (`'e'`) (00:37:11–00:38:30).

Ejemplo paso a paso: un bloque completo

```
mov  eax, 0x11
mov  ecx, 0x0F
xor  eax, ecx
and  eax, 0x1F
shl  eax, 2
mov  ebx, eax
sub  ebx, 0x44
```

Paso	Instrucción	EAX	EBX	Flags relevantes
1	`mov eax, 0x11`	`0x11`	?	sin cambio
2	`mov ecx, 0x0F`	`0x11`	?	sin cambio
3	`xor eax, ecx`	`0x1E`	?	`ZF=0`, `SF=0`
4	`and eax, 0x1F`	`0x1E`	?	`ZF=0`
5	`shl eax, 2`	`0x78`	?	`CF` = último bit que salió
6	`mov ebx, eax`	`0x78`	`0x78`	sin cambio
7	`sub ebx, 0x44`	`0x78`	`0x34`	`ZF=0`, `SF=0`, `CF=0`

Comprobación: `0x11 ^ 0x0F = 0x1E`; `0x1E & 0x1F = 0x1E`; `0x1E << 2 = 0x78`; `0x78 − 0x44 = 0x34`. Todos los valores entran en 8 bits, así que no hay acarreo.

Práctica

1. Ejecutá el bloque anterior en x64dbg y completá la tabla con los valores reales.
2. Cambiá `mov eax, 0x11` por `mov eax, 0xFFFFFFFF` y predecí qué pasa en cada paso.
3. Escribí una secuencia de tres instrucciones que deje `EAX = 0` sin usar `mov`.
4. ¿Qué diferencia hay entre `mov eax, [ebx]` y `lea eax, [ebx]`?
5. Traducí a assembly la comprobación de `if (x == 0) return;` con `x` en `EAX`. *(Pista: usá `test eax, eax` o `cmp eax, 0` para encender el flag `ZF=1` cuando `EAX` vale cero; en las lecciones 2.4 y 2.5 verás cómo combinarlo con el salto condicional `jz` / `jne` y la instrucción `ret`.)*
6. Explicá por qué `INC` no sirve para detectar acarreo sin signo.

Preguntas de repaso

1. ¿Qué significa que el resultado se guarda en el primer operando? Que en `add eax, ebx` el resultado queda en `EAX`, no en `EBX`. El segundo operando se lee pero no se modifica.

2. ¿`LEA` lee memoria? No. Calcula la dirección efectiva de la expresión entre corchetes y la guarda. Es aritmética de punteros.

3. ¿Qué hace `XOR EAX, EAX` y por qué se usa? Pone `EAX` en cero, ocupa menos bytes que `MOV EAX, 0` y actualiza flags. Es la forma idiomática de limpiar un registro.

4. ¿Cuál es la diferencia entre `SHL` y `ROL`? `SHL` desplaza y descarta el bit que sale; `ROL` lo reinyecta por el otro extremo. `SHL` equivale a multiplicar por 2; `ROL` no es una multiplicación.

5. ¿`CMP` guarda el resultado de la resta? No. Solo actualiza flags. Los operandos quedan intactos.

6. ¿Por qué `TEST EAX, EAX` es útil? Porque activa `ZF` si `EAX` es cero, sin modificar el registro. Es la forma estándar de comprobar un valor nulo.

Antes de avanzar

Tenés que poder:

- Leer un bloque de 10 instrucciones y calcular el valor final de los registros.
- Distinguir `mov` de `lea` y `mov` de `movsx` / `movzx`.
- Explicar qué flags cambian `add`, `sub`, `inc`, `and`, `shl` y `cmp`.
- Reconocer las instrucciones de cadena y su relación con `ESI` / `EDI`.

> **Puerta de salida**
>
> Si podés resolver el bloque del ejemplo sin ejecutarlo y después verificarlo en el debugger, estás listo para el flujo de control.

Fuentes de esta lección

- `Reversing desde cero, cap. 02` — `MOV`, `XCHG`, `ADD`, `SUB`, memoria y corchetes.
- `Reversing desde cero, cap. 04` — flags, `MUL` y `DIV`.
- `Reversing desde cero, cap. 06` — `CALL`, `RET`, `JMP`, `PUSH` / `POP`, `INC` / `DEC`.
- `Reversing desde cero, cap. 08` — lógicas y desplazamientos.
- `Reversing desde cero, cap. 09` — `CMP`, `TEST`, `LEAVE`.
- `Reversing desde cero, cap. 12` — práctica con `MOVSX` / `MOVZX`, `LEA`, `LOOP` y cadenas.
- Curso IDA · tutes 4–6 — las mismas instrucciones en IDA: editar con *Assemble*, el byte sobrante como `db` y el relleno `NOP`; `LEA` para direcciones; regla de `ADD` (no se permiten dos operandos en memoria a la vez).

> **Síntesis de estudio de esta lección**
>
> **Núcleo conceptual:** los valores de los ejemplos (`0x11`, `0x0F`, `0x44`, `0x8384`, `"core"`) y la secuencia de instrucciones de los capítulos 02 a 12. **Conocimiento externo:** la tabla de operaciones y la semántica precisa de cada flag. **Recomendación editorial:** el bloque integrador y los ejercicios.

Punto flotante y SSE: los registros xmm

> **De dónde sale esta lección**
>
> Este tema es indispensable en binarios modernos de 64 bits y forma parte del núcleo técnico avanzado de esta obra. El curso de reversing trabaja casi todo con enteros, así que el punto flotante quedaba sin cubrir. Lo incluimos porque aparece en cualquier binario de Windows real y, sin él, te trabás la primera vez que ves un registro `xmm`.

> **Objetivo de la lección**
>
> Al terminar vas a poder reconocer una operación con `float` / `double` en un listado, leer las instrucciones SSE escalares más comunes (`movss` , `movsd` , `addsd` , `cvtsi2sd` , `ucomisd`) y entender por qué una comparación de flotantes se lee distinto de una de enteros.

Prerrequisitos: instrucciones, flujo y registros (Etapa 2). **Duración sugerida:** 3 horas.

Por qué vas a chocarte con esto

Apenas analices un binario que haga una cuenta con decimales —un precio, un porcentaje, una coordenada, un promedio— vas a ver registros `xmm0` , `xmm1` … e instrucciones terminadas en `ss` , `sd` , `ps` o `pd` que no estaban en la tabla de enteros. No es un tema avanzado: es el día a día de cualquier programa compilado hoy. Además, muchos `memcpy` / `memset` optimizados usan registros `xmm` para mover 16 bytes de una.

Dos mundos: x87 y SSE

Hay dos formas de hacer punto flotante en x86, y conviene distinguirlas porque se ven muy distinto:

	x87 (FPU)	**SSE / SSE2**
Época	Clásico; común en código de 32 bits	Estándar desde ~2003; **obligatorio en x64**
Registros	Pila `st(0)` … `st(7)`	`xmm0` … `xmm7` (32 bits) / `xmm0` … `xmm15` (64 bits)
Estilo	Basado en pila (como una calculadora RPN)	Basado en registros (como los enteros)
Instrucciones	`fld` , `fstp` , `faddp` , `fmul` , `fcomip`	`movss` , `movsd` , `addsd` , `mulsd` , `ucomisd`

En 64 bits el compilador usa **casi siempre SSE**; vas a ver x87 sobre todo en binarios viejos de 32 bits.

IEEE 754 en una cucharada

- Un `float` ocupa 4 bytes; un `double` , 8. No son enteros: tienen signo, exponente y mantisa.
- Por eso las constantes flotantes **no se ven como el número**. `1.0` en `double` es `0x3FF0000000000000` ; `0.5` es `0x3FE0000000000000` . Suelen cargarse desde `.rdata` .

- Si en un `movsd xmm0, [0x...]` el valor cargado es `0x4059000000000000`, no es "un número raro": es `100.0` en `double`. Cualquier conversor IEEE 754 te lo traduce; no hace falta hacerlo a mano.

Las instrucciones SSE que vas a ver

La clave está en el **sufijo**:

- `ss` = *scalar single* → un `float` (4 bytes).
- `sd` = *scalar double* → un `double` (8 bytes).
- `ps` / `pd` = *packed* → varios valores a la vez (vectorización; ver la lección de optimizaciones).

Instrucción	Qué hace
`movss` / `movsd`	mueve un float / un double
`movaps` / `movups`	mueve 16 bytes (alineado / sin alinear)
`addsd`, `subsd`, `mulsd`, `divsd`	aritmética con double
`cvtsi2sd`	entero → double
`cvttsd2si`	double → entero (trunca)
`cvtss2sd` / `cvtsd2ss`	float ↔ double
`comiss` / `ucomisd`	**comparar** flotantes (actualiza EFLAGS)
`xorps xmm0, xmm0`	pone el registro en 0 (truco idiomático)
`pxor`, `andps`, `orps`	lógicas de 128 bits (a veces para el signo)

> **`xorps xmm, xmm` no es "una lógica rara"**
>
> Igual que `xor eax, eax` pone `EAX` en cero, `xorps xmm0, xmm0` pone `xmm0` en cero. Es el modismo del compilador para inicializar un flotante en 0.0.

> **`movaps` exige alineación a 16 bytes (el crash nº 1 al parchear en x64)**
>
> `movaps` requiere que la dirección sea múltiplo de 16 (`addr % 16 == 0`); `movups` no. Si al parchear o inyectar una llamada en x64 dejás `RSP` **desalineado** (típico: 8 bytes de más), el programa muere con `0xC0000005` (`STATUS_ACCESS_VIOLATION`) dentro de `ntdll` / `msvcrt` sobre un `movaps xmmword ptr [rsp+...], xmm0`, aunque la memoria sea perfectamente escribible. Por eso la ABI de Windows x64 exige el stack alineado a 16 bytes antes de cada `call`.

Ejemplo paso a paso: `double` en x64

```
double promedio(double a, double b) {
    return (a + b) / 2.0;
}
```

En x64 de Windows, los `double` llegan en `xmm0` y `xmm1`, y el resultado vuelve en `xmm0`:

```
promedio:
    addsd   xmm0, xmm1              ; xmm0 = a + b
    movsd   xmm1, [rel dos]         ; xmm1 = 2.0  (0x4000000000000000)
    divsd   xmm0, xmm1              ; xmm0 = (a+b) / 2.0
    ret
dos: dq 0x4000000000000000          ; 2.0 en .rdata
```

Lo que tenés que reconocer: `xmm0` / `xmm1` traen los argumentos, `addsd` es la suma, la constante `2.0` se carga de datos y el resultado queda en `xmm0`. No hay `push` / `pop` de flotantes: viajan por registros `xmm`.

Comparar flotantes es distinto

Una comparación de enteros usa `cmp` + `je/jg/jl`. Una de flotantes usa `ucomisd` / `comiss`, que actualiza `ZF`, `PF` y `CF` (no `SF` / `OF`), y después un salto **sin signo** (`ja`, `jae`, `jb`, `jbe`, `je`):

```
ucomisd xmm0, xmm1
jae     es_mayor_o_igual     ; usa CF/ZF, como una comparación sin signo
```

Además aparece el **flag de paridad (`PF`)**: se activa si alguno de los operandos es `NaN` (*not a number*). Por eso a veces ves un `jp` / `jnp` antes del salto real: el compilador está manejando el caso "no es un número". Si ves `ucomisd` seguido de `jp`, no te asustes: es el chequeo de NaN, no la condición principal.

x87 en binarios viejos (lo mínimo)

En código de 32 bits podés encontrarte con la pila del FPU:

```
fld  qword [a]     ; empuja a en st(0)
fadd qword [b]     ; st(0) = a + b
fstp qword [res]   ; guarda st(0) en res y lo saca de la pila
```

Se lee como una calculadora de pila: `fld` empuja, las operaciones trabajan sobre `st(0)`, `fstp` guarda y saca. No necesitás dominarlo; alcanza con reconocer que **`f...` = punto flotante x87** y seguir el valor por la pila `st`.

Práctica

1. Compilá `promedio.c` con `gcc -O0` (x64) y mirá el listado: ubicá `xmm0`, `xmm1`, `addsd`, `divsd` y la constante `2.0` en `.rdata`.
2. Tomá la constante que carga tu binario y convertila con un conversor IEEE 754. ¿Coincide con `2.0`?
3. Agregá una comparación `if (a > b)` y mirá cómo aparece `ucomisd` + salto sin signo (y quizás un `jp`).
4. Compilá una versión que use el **x87** y compará (deberías ver `fld` / `faddp` en vez de `xmm`). **Ojo en Windows x64:** `-mfpmath=387` sin `-m32` se **ignora** (x86-64 exige SSE2 por ABI) y `-m32` falla si tu MinGW no es *multilib*. Comandos que sí funcionan: MSVC de 32 bits (*x86 Native Tools Command Prompt*) `cl /Od /arch:IA32 promedio.c`; o MinGW de 32 bits `i686-w64-mingw32-gcc -O0 -mfpmath=387 -mno-sse -o promedio32.exe promedio.c`.

Preguntas de repaso

<details>
<summary>¿Qué significan los sufijos <code>ss</code>, <code>sd</code>, <code>ps</code> y <code>pd</code>?</summary>

<code>ss</code> = un <code>float</code> (scalar single); <code>sd</code> = un <code>double</code> (scalar double); <code>ps</code> / <code>pd</code> = varios valores a la vez (packed, vectorizado).
</details>

<details>
<summary>Ves <code>xorps xmm0, xmm0</code>. ¿Qué hace?</summary>

Pone <code>xmm0</code> en 0.0. Es el modismo para inicializar un flotante en cero, igual que <code>xor eax, eax</code> con enteros.
</details>

<details>
<summary>¿Por qué después de <code>ucomisd</code> aparece a veces un <code>jp</code>?</summary>

Porque la comparación de flotantes activa el flag de paridad si hay un <code>NaN</code>. El <code>jp</code> / <code>jnp</code> maneja ese caso antes del salto principal.
</details>

<details>
<summary>Un <code>movsd xmm0, [rel k]</code> carga <code>0x3FF0000000000000</code>. ¿Qué número es?</summary>

<code>1.0</code> en <code>double</code>. Las constantes flotantes se ven como su codificación IEEE 754, no como el número decimal.
</details>

Antes de avanzar

Tenés que poder:

- Reconocer una operación con `float` / `double` por los registros `xmm` y los sufijos `ss` / `sd`.
- Leer una comparación de flotantes (`ucomisd` + salto sin signo + chequeo de NaN).
- Traducir una constante IEEE 754 con una herramienta.
- Distinguir x87 (`f...`, pila `st`) de SSE (`xmm`).

Puerta de salida

Si podés leer `promedio` en assembly y explicar cómo viajan los argumentos y el resultado en `xmm`, ya no te va a frenar el punto flotante.

Fuentes de esta lección

- *Intel 64 and IA-32 Architectures Software Developer's Manual*, vol. 1 (FPU x87) y vol. 2 (instrucciones SSE/SSE2). Referencia primaria del repertorio de instrucciones.
- *Microsoft x64 calling convention*: paso de `float` / `double` en registros `xmm`.
- Estándar **IEEE 754** para la codificación de `float` y `double`.

Síntesis de estudio de esta lección

Conocimiento externo (agregado por esta edición): todo el contenido (x87, SSE, IEEE 754, convención de `xmm`). Integrado para cubrir código real de 64 bits. **Recomendación editorial:** los ejemplos `promedio` y la práctica con conversor IEEE 754.

Flujo: comparar, saltar y repetir

> **Objetivo de la lección**
>
> Al terminar vas a poder reconstruir un `if/else`, un bucle y un `switch` a partir de un listado de assembly, eligiendo el salto correcto según el dato sea con signo o sin signo.

Prerrequisitos: instrucciones y flags. **Duración sugerida:** 4 horas. Estimación aproximada.

La idea central

El procesador no tiene `if`. Tiene instrucciones que comparan y actualizan flags, y saltos que se toman o no según esos flags. Un `if` en C se convierte en:

```
cmp  eax, 4
jne  else_1      ; si no es igual, salta al else
; bloque del if
jmp  fin
else_1:
; bloque del else
fin:
```

Leer flujo es reconstruir ese patrón: **comparación → salto condicional → bloques → salto incondicional al final**.

`CMP` y `TEST` en detalle

`CMP a, b` calcula `a - b` y descarta el resultado. Los flags quedan como si hubieras restado:

Caso	`ZF`	`CF`	`SF`	`OF`
`a == b`	1	0	según resultado 0	0
`a > b` (sin signo)	0	0	—	—
`a < b` (sin signo)	0	1	—	—
`a > b` (con signo)	0	—	0	0
`a < b` (con signo)	0	—	1	0

`TEST a, b` hace `a AND b` y descarta el resultado. `TEST EAX, EAX` activa `ZF` si `EAX` es cero; es la forma estándar de preguntar "¿es nulo?".

El capítulo 09 lo muestra con `CMP EAX=4, EBX=4` (activa `ZF` y deja los registros intactos) y con `TEST EAX, EAX` seguido de `JZ` (`Reversing desde cero, cap. 09`, 00:01:05–00:06:00 y 00:31:32–00:33:00).

La tabla de saltos condicionales

Instrucción	Salta si...	Uso típico
`JE` / `JZ`	`ZF = 1`	igual / cero
`JNE` / `JNZ`	`ZF = 0`	distinto / no cero
`JA` / `JNBE`	`CF = 0` y `ZF = 0`	mayor, **sin signo**
`JAE` / `JNB` / `JNC`	`CF = 0`	mayor o igual, sin signo
`JB` / `JNAE` / `JC`	`CF = 1`	menor, sin signo
`JBE` / `JNA`	`CF = 1` o `ZF = 1`	menor o igual, sin signo
`JG` / `JNLE`	`ZF = 0` y `SF = OF`	mayor, **con signo**
`JGE` / `JNL`	`SF = OF`	mayor o igual, con signo
`JL` / `JNGE`	`SF ≠ OF`	menor, con signo
`JLE` / `JNG`	`ZF = 1` o `SF ≠ OF`	menor o igual, con signo
`JS` / `JNS`	`SF = 1` / `SF = 0`	signo
`JO` / `JNO`	`OF = 1` / `OF = 0`	desborde con signo

No hace falta memorizarla: alcanza con saber que `J` **+ letra** indica la condición (`E` igual, `N` no, `A` arriba, `B` abajo, `G` mayor, `L` menor) y que las variantes `A/B` son sin signo y `G/L` con signo.

Cómo se codifica un salto (y hasta dónde llega)

Hasta acá vimos *qué* hace cada salto; falta *cómo se escribe en bytes*, porque al parchear —o al leer un salto en el editor hexadecimal— trabajás con esos bytes. El **Curso IDA** lo muestra byte a byte sobre un `CRACKME` propio (Curso IDA · tute 7, Control de flujo):

- **Salto corto (`JMP SHORT`)** → opcode `EB` + **1 byte** de desplazamiento. El destino se calcula como **inicio de la instrucción + 2 + desplazamiento** (el 2 es el largo del salto corto). El alcance es chico: `0x7F` es el **máximo hacia adelante** y `0x80` el **máximo hacia atrás**.
- Un desplazamiento `FE` (−2) hace que el salto caiga **sobre sí mismo**: es el **loop infinito** clásico (`EB FE`).
- **Salto largo (`JMP` / `Jcc` near)** → opcode `E9` + un **dword** de desplazamiento, para alcanzar direcciones lejanas. La distancia se calcula como **destino − inicio − 5** (5 = largo del salto largo).

> **Probarlo sin romper la base de datos**
>
> El tute edita esos bytes con **Patch Byte** y **KeyPatch** dentro de IDA, y antes hace un **snapshot de la IDB** (*View → Database snapshot manager*) para volver atrás con *Restore* si el cambio rompe una función. Hacé lo mismo antes de experimentar: cambiás un `0x05` por `0x7F` o `0x80`, ves el efecto y, si algo se rompe, restaurás.

Ejemplo paso a paso: el mismo dato, dos interpretaciones

```
mov eax, 0xFFFFFFFF
cmp eax, 2
jg  mayor_firmado
ja  mayor_sin_signo
```

- `CMP` calcula `0xFFFFFFFF − 2`. Como número **sin signo**, `0xFFFFFFFF` es 4.294.967.295, mucho mayor que 2: `CF = 0`, `ZF = 0`, así que `JA` **salta**.
- Como número **con signo**, `0xFFFFFFFF` es −1, menor que 2: `SF = 1`, `OF = 0`, `SF ≠ OF`, así que `JG` **no salta**; el que saltaría es `JL`.

Es el ejemplo que usa el capítulo 09 (00:24:30–00:25:30). La lección: **el mismo `cmp` puede producir saltos opuestos según la instrucción que lo sigue**. Si el código usa `JA/JB`, el autor pensó el dato como sin signo; si usa `JG/JL`, como con signo. Ese detalle cambia por completo la reconstrucción de una comparación de contraseñas o de un índice.

Bucles

`while` / `for`

```
while (i < 10) { ...; i++; }
```

Se compila a:

```
mov ecx, 0
loop_start:
cmp ecx, 10
jge loop_end      ; si ecx >= 10, salir
; cuerpo
inc ecx
jmp loop_start
loop_end:
```

El salto de salida es la **negación** de la condición: el cuerpo se ejecuta mientras se cumple, así que se sale cuando deja de cumplirse.

`do/while`

La condición va al final, después del cuerpo. Se ejecuta al menos una vez.

`LOOP`

`LOOP etiqueta` decrementa `ECX` y salta si `ECX ≠ 0`. Es una instrucción compacta pero más lenta que `dec ecx; jnz`. En el capítulo 12 se ve `LOOP` con `ECX` como contador (00:24:06).

Bucles con cadenas

`REP MOVSB`, `REPE CMPSB` y `REPNE SCASB` repiten una operación mientras se cumple una condición y `ECX` no llegue a cero. Son bucles "de hardware" que el procesador ejecuta sin un `jmp` explícito.

`switch`: la tabla de saltos

Un `switch` con casos consecutivos se compila a una **tabla de direcciones**:

```
cmp eax, 3
ja  default
jmp [tabla + eax*4]
```

La tabla contiene las direcciones de cada `case`; `jmp [tabla + eax*4]` salta a la que corresponde al índice. En el desensamblador se ve como un `jmp` con un operando indexado, seguido de una zona de datos con direcciones (a veces en `.rdata`).

Para reconstruirlo: identificá la comparación de rango (`cmp` + `ja default`), la tabla y el orden de las entradas. Si los casos no son consecutivos, el compilador puede usar una cadena de comparaciones en lugar de una tabla.

Decisiones sin salto: `CMOVcc`

No toda decisión produce un salto. La familia `CMOVcc` (*conditional move*) **mueve un valor solo si se cumple una condición**, mirando los mismos flags que los `Jcc`. Por ejemplo, `CMOVZ eax, ebx` copia `EBX` en `EAX` únicamente si `ZF = 1`; si no, `EAX` queda como estaba.

```
cmp  byte ptr [esi], 0x41   ; ¿el primer carácter es 'A'?
mov  ecx, 0x31
cmovz esi_byte, ecx         ; si era 'A' (ZF=1), lo reemplaza por 0x31; si no, lo deja
```

La consecuencia para el reversing es importante: con `CMOVcc` **no hay dos bloques** (uno para el camino bueno y otro para el malo) ni un salto visible. La comparación y la elección ocurren en el mismo bloque, así que no hay un "salto para parchear" evidente. El capítulo 16 usa justamente esto para que un crackme no tenga bloques separados de "chico bueno" y "chico malo": decide con un `CMOVZ` y no con un `Jcc` (`Reversing desde cero, cap. 16`).

Cuando no encuentres el salto que decide, buscá un `CMOVcc`, un `SETcc` (que deja 0 o 1 en un byte según la condición) o una operación aritmética que use el flag: la decisión puede estar ahí, sin bifurcación.

Ejemplo integrador: el `if` del capítulo 15

El programa de práctica del capítulo 15 pide dos números y aplica:

```
if (a > 50 || b < 50) { error; }
if ((a + b) * 4 == 480) { correcto; } else { incorrecto; }
```

En assembly, la primera condición se parte en dos comparaciones con saltos:

```
cmp eax, 50
jg  error          ; a > 50 -> error
cmp ebx, 50
jl  error          ; b < 50 -> error
```

La segunda usa una suma, un desplazamiento y una comparación:

```
add eax, ebx
shl eax, 2         ; (a+b) * 4
cmp eax, 480
jne incorrecto
```

El capítulo lo reconstruye leyendo los saltos y los comentarios que va agregando en IDA, y luego lo confirma en el debugger con `40` y `80` (`Reversing desde cero, cap. 15`, 00:10:00–00:19:00).

> **El error de leer el salto al revés**
>
> Un salto "hacia el bloque malo" no significa que la condición sea verdadera: puede ser la negación. Antes de escribir el `if`, preguntate **qué camino toma el programa cuando el salto se produce** y **qué camino cuando no**. El capítulo 09 insiste en esto al explicar cada `Jcc` por su condición (00:07:00–00:12:00).

Práctica

1. Traducí a C:

```
cmp eax, 10
jl  menor
mov ebx, 1
jmp fin
menor:
mov ebx, 0
fin:
```

2. Escribí en C un `for` que sume los números del 1 al 10 y compilalo con `-O0`. Identificá el contador, la comparación y el salto de salida.
3. ¿Qué diferencia hay entre `JA` y `JG`? Dame un ejemplo con `0xFFFFFFFF`.

4. Reconstruí un `switch` de tres casos a partir de una tabla de saltos inventada por vos. Dibujá la tabla y el `jmp` indexado.
5. En el programa del capítulo 15, cambiá `40 80` por `50 70`. ¿Sigue dando 480? ¿Qué condición falla?

Preguntas de repaso

1. ¿`CMP` modifica los operandos? No. Solo actualiza flags. Es una resta "de prueba".

2. ¿Cómo se sale de un `while`? Con un salto condicional que se toma cuando la condición deja de cumplirse, o con un `jmp` al final si el cuerpo tiene un `break`.

3. ¿Qué indica `JA` que no indica `JG`? `JA` compara sin signo; `JG`, con signo. Con `0xFFFFFFFF` y `2`, `JA` salta y `JG` no.

4. ¿Cómo se reconoce una tabla de salto? Por un `jmp` con operando indexado (`[tabla + registro*4]`) y una zona de datos con direcciones consecutivas, generalmente precedida por una comparación de rango.

5. ¿`LOOP` usa qué registro? `ECX`. Decrementa y salta si no llegó a cero.

Antes de avanzar

Tenés que poder:

- Reconstruir un `if/else` a partir de `cmp` + `jcc` + `jmp`.
- Elegir `JA/JB` o `JG/JL` según el tipo del dato.
- Reconocer un bucle `while`, un `do/while` y un `for`.
- Explicar una tabla de saltos.

> **Puerta de salida**
>
> Si podés mirar un bloque con dos saltos y decir qué camino toma con un valor concreto, estás listo para las llamadas a funciones.

Fuentes de esta lección

- `Reversing desde cero, cap. 09` — `CMP`, `TEST`, `JE/JNE/JA/JB/JG/JL`, `LEAVE`.
- `Reversing desde cero, cap. 16` — `CMOVcc`: decidir sin salto, un crackme sin bloques separados de bueno/malo.
- `Reversing desde cero, cap. 12` — `LOOP` y comparaciones en un binario real.
- `Reversing desde cero, cap. 15` — reconstrucción de condiciones.
- Curso IDA · tute 7, Control de flujo — codificación de los saltos (`EB` / `E9`), alcance ± `0x7F` / `0x80`, `EB FE` = loop infinito, y cómo parchearlos en IDA con snapshot de la IDB.
- `Videos y material adicional, Video 29` — `IF/ELSE` compilado a `CMP` + saltos.
- `Videos y material adicional, Video 30` — bucles y pila.

Síntesis de estudio de esta lección

Núcleo conceptual: la tabla de saltos del capítulo 09, el ejemplo `0xFFFFFFFF` contra `2`, el `LOOP` del capítulo 12 y el `if` del capítulo 15. **Conocimiento externo:** la tabla completa de `Jcc` y el patrón de tabla de saltos. **Recomendación editorial:** los ejercicios y el ejemplo integrador.

Llamadas, convenciones y marcos de función

> **Objetivo de la lección**
>
> Al terminar vas a poder seguir un `call` hasta su destino, calcular la dirección de retorno, identificar los argumentos de una función en 32 y 64 bits y decir quién limpia el stack.

Prerrequisitos: flujo, stack y registros. **Duración sugerida:** 4 horas. Estimación aproximada.

`CALL` y `RET`

`call destino` hace dos cosas:

1. Empuja en el stack la dirección de la instrucción **siguiente** (el *return address*).
2. Salta a `destino`.

`ret` hace lo inverso: saca el return address del stack y salta ahí.

Ejemplo del capítulo 06: un `call` en `0x00401013` (5 bytes) deja `0x00401018` en el tope del stack; la rutina hace `nop; ret` y vuelve a `0x00401018` (`Reversing desde cero, cap. 06`, 00:00:53–00:03:10).

```
0x401013: call 0x40102C     ; push 0x401018 ; jmp 0x40102C
0x401018: ...               ; acá vuelve
```

`ret n` (por ejemplo `ret 0x10`) hace lo mismo y además suma `n` a `ESP` después de sacar el return address. Se usa en la convención `stdcall` para que la función llamada limpie los argumentos. En el ejemplo del capítulo 06, `ret 0x10` mueve el stack `0x10 + 4 = 0x14` (00:08:38–00:09:30).

`call` directo e indirecto

```
call 0x401000          ; directo: el destino está en el opcode (E8 + desplazamiento)
call dword ptr [0x403000]  ; indirecto: el destino está en la memoria (FF15)
```

El indirecto es el que se usa para llamar APIs a través de la IAT. En el capítulo 06, `call dword ptr [0x402018]` no salta a `0x402018` sino a lo que hay allí; el capítulo 05 lo muestra con una API de `kernel32` (`Reversing desde cero, cap. 05`, 00:13:00–00:18:00).

Convenciones de llamada

Una **convención de llamada** es el acuerdo sobre tres cosas: dónde van los argumentos, quién limpia el stack y qué registros se preservan.

32 bits

Convención	Argumentos	Limpia el stack	Registros volátiles
`cdecl`	En el stack, de derecha a izquierda	El llamador	`EAX`, `ECX`, `EDX`
`stdcall`	En el stack, de derecha a izquierda	La función (`ret n`)	`EAX`, `ECX`, `EDX`
`fastcall`	`ECX`, `EDX` y el resto en el stack	La función	`EAX`, `ECX`, `EDX`
`thiscall`	`ECX` = `this`; resto en el stack	La función	`EAX`, `ECX`, `EDX`

`cdecl` es la convención por defecto de C; por eso las funciones variádicas (`printf`) la usan: solo el llamador sabe cuántos argumentos pasó. `stdcall` es la de la API de Windows (`MessageBoxA`, `CreateThread`). `thiscall` es la de los métodos de clase en C++.

Cómo reconocerlas en el listado:

- Si después del `call` hay un `add esp, N`, es `cdecl`: el llamador limpia.
- Si la función termina en `ret N`, es `stdcall` o `fastcall`: la función limpia.
- Si el primer argumento llega en `ECX`, es `fastcall` o `thiscall`.

El video 28 del curso de OllyDbg muestra el mismo concepto con procedimientos propios: `CALL` / `RET`, variables locales y argumentos en la pila (`Videos y material adicional, Video 28`, 00:00:00–00:05:00).

64 bits

En x64 hay una sola convención para el código de Windows: los primeros cuatro argumentos enteros van en `RCX`, `RDX`, `R8` y `R9`; el resto, en el stack. El llamador reserva **32 bytes de shadow space** antes del `call` y limpia el stack después. Los registros `RAX`, `RCX`, `RDX`, `R8`–`R11` son volátiles; `RBX`, `RBP`, `RDI`, `RSI`, `R12`–`R15` se preservan.

El capítulo 18 lo muestra con `main`: los tres argumentos llegan en `RCX`, `RDX` y `R8`, y las variables locales se referencian como `[rsp+0xNN]` (`Reversing desde cero, cap. 18`, 00:13:00–00:20:00).

> **En 64 bits los argumentos no se cargan "en orden"**
>
> Como los argumentos no se apilan uno tras otro con `push`, el compilador puede setear los registros en **cualquier orden**: podés ver que carga `R8` y recién una línea más abajo `RCX`. No deduzcas el primer argumento por su posición en el listado; identificalo por el **registro** (`RCX`, `RDX`, `R8`, `R9`) y por el rol. El capítulo 19 lo muestra con `fgets(buffer, maxcount, stream)`: `RCX` es el buffer, `RDX` el máximo y `R8` el stream, aunque en el código `RCX` aparezca seteado después de `R8` (`Reversing desde cero, cap. 19`).

x64: el resto de la ABI

El cap. 18 muestra lo básico; la ABI de x64 de Windows tiene cuatro detalles más que conviene conocer (**conocimiento externo**; convención x64 de Microsoft):

- **Alineación de la pila.** En el punto del `call`, `RSP` debe estar alineado a **16 bytes**. Como el `call` empuja 8 bytes (el return address), al entrar a la función `RSP` queda "desalineado en 8"; por eso los prólogos restan una cantidad que deja todo en múltiplo de 16 (y el **shadow space** de 32 bytes también respeta eso). Si ves un `sub rsp, 0x28` típico, es shadow space (0x20) + alineación.
- **Argumentos de punto flotante.** `float` / `double` van en `XMM0` – `XMM3` (misma posición que los enteros: el 1.º en `XMM0` o `RCX`, no ambos). `XMM0` – `XMM5` son **volátiles**; `XMM6` – `XMM15` se **preservan**.
- **Funciones variádicas.** En una variádica (`printf`), cada argumento flotante se pasa **a la vez** en el registro XMM y en el GPR entero correspondiente, y `AL` indica cuántos registros vectoriales se usaron. Por eso una variádica se ve distinta de una función normal con los mismos tipos.
- **Unwind / `.pdata`.** No hay cadena SEH en el stack como en 32 bits: la info de desenrollado (qué registros salva el prólogo, dónde está el handler) vive en tablas `.pdata` / `.xdata` (lo ves en PE fino 2.14). Sirve para **delimitar funciones** sin símbolos y para entender el prólogo/epílogo.

Ejemplo paso a paso: `suma(a, b)` en 32 bits

```
int suma(int a, int b) { return a + b; }
int main(void) { return suma(3, 4); }
```

Compilado con `cdecl`, la llamada se ve así:

```
push 4          ; segundo argumento
push 3          ; primer argumento
call suma
add  esp, 8     ; el llamador limpia los dos argumentos
```

Y la función:

```
suma:
push ebp
mov  ebp, esp
mov  eax, [ebp+8]    ; a
add  eax, [ebp+12]   ; b
pop  ebp
ret
```

Recorrido del stack:

```
antes del call:      [esp] = 3, [esp+4] = 4
después del call:    [esp] = return address
                     [esp+4] = 3
                     [esp+8] = 4
después del prólogo: [ebp] = ebp viejo
                     [ebp+4] = return address
                     [ebp+8] = 3
                     [ebp+12] = 4
```

Si la función fuera `stdcall`, terminaría en `ret 8` y no habría `add esp, 8` después del `call`. Ese es el detalle que te dice quién limpia.

> **En el Curso IDA: leer una función basada en EBP (ejercicios IDA1/IDA2)**
>
> El Curso IDA trae dos ejercicios muy didácticos de funciones basadas en `EBP`: `IDA1.exe` e `IDA2.exe`. El método es el de siempre: se llega al `main` **por la string** ("You are a winner man je"), se usan los **xrefs** (`X`) para ver dónde se tocan las variables y se las **renombra** (en `IDA2` hay dos "cookies" que hay que comparar para caer en la zona del "chico bueno"). Un detalle real a tener en cuenta: `IDA1.exe` está compilado con **DevC++**, que arma las llamadas a las APIs de una forma distinta a la habitual y puede marear al principio (Curso IDA · tute 23, IDA1.exe y tute 24, IDA2 (cookies)).

Recursión

Una función recursiva se llama a sí misma. Cada llamada crea un **nuevo marco** en el stack, con sus propios argumentos y locales. Por eso una recursión sin caso base termina en *stack overflow*: el stack se agota.

```
int fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }
```

En el debugger vas a ver el mismo `call` repetido con valores decrecientes de `n`, y el call stack creciendo hacia arriba. El panel **Call Stack** te muestra la cadena de marcos; es la forma más rápida de entender una recursión.

Llamadas indirectas y funciones virtuales

```
call [eax]        ; llama a la función apuntada por EAX
call [eax+4]      ; llama a la segunda entrada de una tabla
```

En C++, un objeto con funciones virtuales guarda al principio un puntero a la **vtable**: una tabla de punteros a métodos. `call [eax+4]` puede ser "llamar al segundo método virtual del objeto apuntado por EAX". El capítulo 37 lo desarrolla con una clase de `0x58` bytes y una vtable de cuatro punteros (`Reversing desde cero, cap. 37`, 00:07:38–00:10:48). Es un tema de la rama de C++; en el recorrido base alcanza con reconocer el patrón.

Práctica

1. Compilá `suma.c` con `gcc -O0 -m32` (o Visual Studio en 32 bits) y mirá el listado en x64dbg. Identificá los `push`, el `call`, el `add esp` y el `ret`.
2. Hacé lo mismo con `__stdcall` (en GCC, `__attribute__((stdcall))`) y comprobá si aparece `ret 8`.
3. Compilá la versión de 64 bits y verificá en qué registros llegan `a` y `b`.
4. Escribí una función recursiva de factorial y mirá el Call Stack en el debugger con `n = 3`.
5. En el programa del capítulo 15, identificá si `printf` / `scanf` son `cdecl` o `stdcall` y justificá con el `add esp` posterior.

Preguntas de repaso

1. ¿Qué guarda `call` en el stack? La dirección de la instrucción siguiente al `call`, para que `ret` sepa dónde volver.

2. ¿Quién limpia los argumentos en `cdecl`? ¿Y en `stdcall`? En `cdecl`, el llamador (con `add esp, N` después del `call`). En `stdcall`, la función llamada (con `ret N`).

3. ¿Cómo se pasa `this` en un método de C++ de 32 bits? En `ECX`, según la convención `thiscall`.

4. ¿Cuántos argumentos van en registros en x64? Los primeros cuatro enteros: `RCX`, `RDX`, `R8`, `R9`. El resto va al stack, después de 32 bytes de shadow space.

5. ¿Por qué una recursión infinita produce un stack overflow? Porque cada llamada crea un marco nuevo y el stack tiene un tamaño limitado. Al agotarse, el sistema termina el proceso.

Antes de avanzar

Tenés que poder:

- Calcular la dirección de retorno de un `call`.
- Distinguir `call` directo de indirecto.
- Reconocer `cdecl`, `stdcall` y la convención x64 en un listado.
- Dibujar el marco de una función con dos argumentos.

> **Puerta de salida**
>
> Si podés explicar quién limpia el stack mirando el `ret` o el `add esp`, estás listo para usar el debugger con criterio.

Fuentes de esta lección

- `Reversing desde cero, cap. 06` — `CALL` directo/indirecto, `RET`, `RET n`.
- `Reversing desde cero, cap. 18` — marcos, prólogo/epílogo, convención x64.

- `Reversing desde cero, cap. 19` — argumentos x64 cargados fuera de orden; identificar por registro y rol (`fgets`).
- `Videos y material adicional, Video 28` — procedimientos y variables locales en MASM32.
- `Reversing desde cero, cap. 37` — vtables y llamadas virtuales.
- Curso IDA · tutes 23 y 24 — funciones basadas en EBP (`IDA1.exe` / `IDA2.exe`), el quirk de DevC++ al llamar APIs y el método strings→ `main` →xrefs→renombrar.

> **Síntesis de estudio de esta lección**
>
> **Núcleo conceptual:** el ejemplo del `call` en `0x00401013`, `ret 0x10`, la convención x64 del capítulo 18 y la vtable del capítulo 37. **Conocimiento externo:** tabla de convenciones `cdecl` / `stdcall` / `fastcall` / `thiscall`. **Recomendación editorial:** el ejemplo `suma` y los ejercicios.

El debugger: x64dbg de punta a punta

> **Objetivo de la lección**
>
> Al terminar vas a poder abrir un programa propio en x64dbg, moverte por el código, ejecutar con F7/F8/F9, poner breakpoints por software, por hardware y de memoria, condicionarlos y dejar un registro de lo observado.

Prerrequisitos: registros, instrucciones, flujo y llamadas. **Duración sugerida:** 5 horas. Estimación aproximada.

Análisis dinámico y estático

Hay dos formas de mirar un programa:

- **Dinámico:** lo ejecutás dentro de un debugger y observás qué hace mientras corre. Te muestra la verdad de la ejecución, pero solo del camino que tomaste.
- **Estático:** lo analizás sin ejecutarlo, con un desensamblador como IDA o Ghidra. Te muestra todo el código, pero no los valores concretos.

El capítulo 11 lo plantea como una decisión de método: el dinámico es más visual y rápido para problemas chicos; el estático permite analizar funciones que no sabés cómo alcanzar y es obligatorio cuando no podés ejecutar (`Reversing desde cero, cap. 11`, 00:00:05–00:01:00).

En la práctica se usan los dos: primero el estático para orientarte, después el dinámico para confirmar.

> **En el Curso IDA: IDA también debuggea, y se practica con crackmes propios**
>
> El Curso IDA refuerza esta misma combinación de estático y dinámico, pero sin salir de IDA: recordá que **IDA no solo desensambla, también soporta varios debuggers**, así que podés estudiar el loader y después ejecutar sobre el mismo programa sin cambiar de herramienta. Lo practica con crackmes chiquitos hechos por el propio autor (`HOLA_REVERSER.exe` , `TEST_REVERSER.exe`) (Curso IDA · tute 10, Loader + debugger; prácticas en tute 9 y tute 12).

La ventana de x64dbg

Al abrir un programa, x64dbg se detiene en el **entry point**: la primera instrucción que ejecuta el código del programa. No es necesariamente `main`; es el punto donde el cargador le pasa el control (cap. 01, 00:02:08).

Las pestañas principales:

Pestaña	Qué muestra
CPU	Desensamblado, registros, flags, stack y dump
Log	Mensajes del debugger y de los breakpoints con log
Notes	Notas que escribís vos
Breakpoints	Lista de breakpoints con condición y estado
Memory Map	Regiones del espacio de direcciones
Call Stack	Cadena de llamadas actual
SEH	Cadena de manejadores de excepciones (32 bits)
Symbols	Módulos cargados y sus funciones
Threads	Hilos del proceso
Handles	Recursos abiertos
Trace	Resultado del trazado
PE Viewer	Cabecera PE del módulo (plugin)

El capítulo 01 hace un recorrido por todas ellas y aclara que al principio no hace falta entenderlas todas (00:04:00–00:07:00).

Ejecutar paso a paso

Tecla	Acción	Qué hace
`F9`	Run	Ejecuta hasta el próximo breakpoint o hasta que el programa termine
`F8`	Step over	Ejecuta la instrucción; si es un `call`, ejecuta la función completa sin entrar
`F7`	Step into	Ejecuta la instrucción; si es un `call`, entra a la función
`F2`	Toggle breakpoint	Pone o saca un breakpoint por software
`Ctrl+F2`	Restart	Reinicia el proceso
`*`	Follow EIP/RIP	Vuelve a la instrucción actual

La diferencia entre F7 y F8 es la más importante al empezar: con F8 pasás "por arriba" de una función; con F7 entrás. El capítulo 01 lo demuestra entrando a `main` con F7 y saliendo con F8 (00:15:16–00:20:00).

> **El "step over" no saltea la función**
>
> `F8` **ejecuta** la función completa, no la ignora. La diferencia es que no te detiene instrucción por instrucción dentro de ella. Si la función tiene efectos (escribe un archivo, muestra un mensaje), esos efectos ocurren igual.

Los tres tipos de breakpoint

Por software

Es el clásico. El debugger reemplaza el primer byte de la instrucción por `0xCC` (la instrucción `INT3`). Cuando el procesador la ejecuta, genera una excepción que el debugger intercepta, muestra el estado y restaura el byte original.

- Se pone con `F2` o con doble clic en la columna de opcodes.
- No hay un límite práctico: podés poner todos los que quieras.
- Es fácil de detectar por un programa: si compara el byte en memoria con el del archivo, ve `0xCC` donde debería haber otra cosa.

Por hardware

Usa los registros de depuración `DR0` a `DR3` del procesador. Hay **cuatro por hilo** (no por proceso: cada hilo tiene su propio contexto de CPU). En x64dbg se aplican al hilo actual, salvo que configures lo contrario.

- Se ponen con clic derecho → *Breakpoint* → *Set hardware on execution*.
- Se marcan con un punto rojo, no con la línea entera.
- No modifican el código, así que son más difíciles de detectar.
- También permiten *on access* (lectura/escritura) sobre 1, 2 o 4 bytes.

El capítulo 07 los muestra y aclara que el hardware "para una instrucción después" del acceso (`Reversing desde cero, cap. 07`, 00:02:34–00:10:00). Es una imprecisión útil de conocer: el *debug exception* se genera después de ejecutar la instrucción que accedió a la dirección vigilada.

> **Los breakpoints de hardware son por hilo (no por proceso)**
>
> A veces se menciona por error que hay "cuatro breakpoints de hardware por proceso" o que "funcionan solamente en el thread principal" (así se dice en un pasaje del capítulo 07). En realidad, los registros `DR0` – `DR3` forman parte del contexto de CPU de **cada hilo**, por lo que el límite de cuatro es por hilo. En la práctica, x64dbg te muestra y aplica los breakpoints de hardware sobre el hilo que tengas activo en ese momento; si no los ves dispararse, revisá en la pestaña *Threads* qué hilo está seleccionado.

De memoria

Vigila una región (no solo 1, 2 o 4 bytes) y se dispara cuando el programa la lee o la escribe. Internamente usa protecciones de página: el debugger marca la página como no accesible y recibe la excepción cuando alguien la toca.

- Se pone seleccionando un rango en el dump → clic derecho → *Breakpoint* → *Memory on access/read/write*.
- Opciones: *single shot* (se deshabilita después del primer impacto) o *restore on hit* (sigue activo).
- Es la herramienta para responder "¿quién lee este buffer?".

El capítulo 07 lo prueba con el buffer donde `GetDlgItemText` escribe el nombre: pone un breakpoint de memoria sobre los primeros bytes y el programa se detiene cuando los lee (00:10:00–00:14:00).

Breakpoints condicionales y con log

Un breakpoint puede tener una **condición** y una **acción**:

- **Condición** (`breakif`): solo se detiene si se cumple. Ejemplo: `EAX == 8`.
- **Log**: escribe un texto en el panel Log cada vez que pasa, sin detenerse. Ejemplo: `valor de EAX = {EAX}`.

En el capítulo 07, Ricardo usa un breakpoint en una función que devuelve la longitud del texto ingresado. Primero lo configura para loguear `{EAX}` y ve `5` cuando escribe `"solid"`. Después le pone la condición `EAX == 8` y comprueba que solo se detiene cuando el nombre tiene 8 caracteres (00:15:00–00:20:00).

```
breakif: EAX == 8
log:     "longitud = {EAX}"
```

Ese patrón —"no me interesa parar, me interesa saber qué valor toma"— es el que vas a usar para entender bucles y comparaciones sin detenerte cien veces.

Etiquetas, comentarios y navegación

- **Etiqueta** (`:`): renombra una dirección para no perderte. En el capítulo 11, Ricardo etiqueta el bloque del mensaje "acceso denegado" como `chico_malo` y el otro como `chico_bueno`.
- **Comentario** (`;`): agrega una nota a una línea.
- **Follow in dump**: muestra en el panel de memoria el contenido de una dirección.
- **Follow in disassembler**: salta a la dirección apuntada.
- **Ctrl+G**: ir a una dirección o expresión.
- **Ctrl+Shift+G**: ir a un file offset (útil para comparar con el archivo).

Las etiquetas y comentarios no cambian el programa: son tu mapa de lectura. El capítulo 11 muestra cómo el análisis visual se apoya en ellas (00:02:16–00:06:00).

Encontrar el código que maneja un botón o un mensaje

En un programa con ventana no hay un `main` lineal: Windows le **envía mensajes** (se hizo clic, se escribió en un campo, se apretó un botón) y el programa los atiende en una función despachadora. Encontrar esa función es el primer paso para analizar una interfaz.

Lo que hay que saber:

- La función despachadora es el **WinProc** (si la ventana se creó con `CreateWindow`) o la **Dialog procedure** (si se creó con `DialogBoxParam`). Las dos hacen lo mismo: reciben el mensaje y lo reparten con un `switch`.
- El **mensaje** es el **segundo argumento** de esa función. Un `WM_COMMAND` vale `0x111` y es el que se dispara, en general, al apretar un botón; un `WM_LBUTTONDOWN` vale `0x201`.
- El **Control ID** identifica qué botón se tocó. Se puede ver pasando el mouse por el control en una herramienta de inspección de ventanas.

Tres maneras de llegar al handler, de más simple a más robusta:

1. **Buscar el Control ID en estático.** Si el botón tiene ID `0x13A`, buscás ese inmediato en IDA (ver la lección de IDA) y caés dentro del `WinProc`. Falla cuando el ID es un número bajo y común (`1`, `5`, `10`): hay demasiadas ocurrencias.
2. **Breakpoint condicional sobre el mensaje.** Poné un breakpoint donde la función carga el mensaje en un registro y condicionalo a ese mensaje. Podés además loguear los dos argumentos leyéndolos del stack:

```
log:     "msg={2}  id={3}"      ; con getdword(sp+8) y getdword(sp+C)
breakif: getdword(sp+8) == 0x111 && getdword(sp+0xC) == 0x13A
```

3. **Breakpoint en `SendMessage` + breakpoint de ejecución en el código.** Es el método genérico que sirve aunque no sepas nada del programa. En Windows, los mensajes llegan vía `SendMessageW` / `SendMessageA` de `user32.dll` (ahí el mensaje también es el segundo argumento). Poné un breakpoint ahí, condicionado a tu mensaje; cuando pare, poné un **breakpoint de hardware por ejecución sobre toda la sección de código del ejecutable** (su dirección de inicio y su tamaño, que ves en el *Memory Map* o en *Segments*). Al continuar, el programa se detiene en la primera instrucción **de tu código** que corre tras recibir el mensaje: esa es la entrada del WinProc. Después borrás ese breakpoint para seguir.

> **Cómo se registra el WinProc**
>
> Con `CreateWindow`, antes hay un `RegisterClassA` que recibe una estructura `WNDCLASS`; dentro de esa estructura está el puntero al WinProc. Con `DialogBoxParam`, el procedimiento se pasa directamente como uno de sus argumentos. Encontrar `RegisterClassA` o `DialogBoxParam` te lleva, por cualquiera de los dos caminos, a la función que maneja los mensajes.

El capítulo 28 ("Mensajes de Windows") muestra las tres formas sobre varios crackmes propios, incluida la carga de los símbolos de `user32.dll` para poner el breakpoint en `SendMessageW` (`Reversing desde cero`,

cap. 28). El capítulo 24b agrega que el `switch` de mensajes se lee mejor traduciendo los números a sus nombres `WM_*` con la tecla `M` en IDA.

Parchear (solo programas propios)

x64dbg permite modificar instrucciones en memoria y guardar los cambios en un archivo:

1. Seleccioná la instrucción.
2. Clic derecho → *Binary* → *Fill with NOP* (o *Edit* para escribir bytes).
3. *Patches* → *Patch file* para guardar una copia con los cambios.

En el capítulo 11, Ricardo reemplaza un salto condicional por dos `NOP` y guarda una copia del programa que acepta cualquier clave (00:20:00–00:25:00). Es una demostración didáctica sobre un crackme de práctica.

> **El límite legal y ético**
>
> Parchear un programa **no es una técnica neutral**. En esta guía se practica únicamente con programas propios, software abierto o crackmes diseñados para aprender, y se explica el efecto de un parche para entender el flujo. No se parchea software comercial para saltar licencias, DRM, autenticación ni controles de acceso. Si el programa no es tuyo o no tiene licencia para analizarlo, no lo toques.

Anti-debug: qué cambia cuando hay un debugger

Un programa puede detectar que está siendo depurado. Los chequeos más comunes en Windows de 32 bits:

Chequeo	Dónde mira	Qué delata
`IsDebuggerPresent`	`PEB+2`	Byte en 1 si hay debugger
`NtGlobalFlag`	`PEB+0x68`	Flags de heap activados por el debugger
Lista de procesos	`CreateToolhelp32Snapshot`	Encuentra `x64dbg.exe`, `ida.exe`, etc.
`ParentPID`	PID del proceso padre	El padre no es `explorer.exe`
`CloseHandle` inválido	Excepción `0xC0000008`	Comportamiento distinto bajo debugger
`INT3` en memoria	Byte `0xCC`	Breakpoint por software
`ContextRecord` tras una excepción	`DR0`–`DR3` del contexto	Breakpoints de **hardware**

El capítulo 21 muestra el último caso: un programa que provoca una excepción a propósito puede, en su manejador, leer el `ContextRecord` y mirar los registros de depuración `DR0`–`DR3`; si no están en cero, hay breakpoints de hardware puestos y el programa lo detecta (`Reversing desde cero, cap. 21`). Es un motivo para conocer cómo funcionan las excepciones (ver la rama de vulnerabilidades).

El capítulo 22 explica los dos primeros con el PEB y el comando `hide` de x64dbg (`Reversing desde cero, cap. 22`, 00:02:35–00:07:00). El capítulo 24 muestra un crackme propio con cuatro chequeos y un mensaje que dice cuál lo detectó (`Reversing desde cero, cap. 24`, 00:01:17–00:03:31).

Para esta guía, el objetivo es **entender** esos mecanismos y reconocerlos en un binario, no enseñar a evadirlos. Si un programa se cierra solo bajo el debugger, la pregunta correcta es "¿qué chequeo está fallando?", no "¿qué truco le aplico?".

Práctica

1. Compilá tu programa `check.exe` (Etapa 1) y abrilo en x64dbg.
2. Recorré con F8 hasta `main`. Anotá `EIP`, `ESP` y `EBP`.
3. Poné un breakpoint por software en la comparación `(a+b)*4 == 480`, ejecutá con `40` y `80`, y verificá los valores en el stack.
4. Poné un breakpoint de hardware *on access* sobre la variable `b` y observá cuándo se lee.
5. Poné un breakpoint de memoria sobre el buffer de `scanf` y observá cuándo se escribe.
6. Agregá un breakpoint con log `{EAX}` en la comparación y ejecutá varias veces con valores distintos. Guardá el log.
7. Etiquetá el bloque "Correcto" y el bloque "Incorrecto". Sacá una captura para tu bitácora.

Preguntas de repaso

1. ¿Qué diferencia hay entre F7 y F8? F7 entra a las funciones; F8 las ejecuta sin detenerse dentro. Las dos ejecutan la instrucción actual.

2. ¿Qué byte usa un breakpoint por software? `0xCC`, la instrucción `INT3`. El debugger lo restaura al detenerse.

3. ¿Cuántos breakpoints de hardware hay y de quién son? Cuatro (`DR0` – `DR3`) por hilo. Se aplican al contexto del hilo; si no aparecen, revisá qué hilo está seleccionado.

4. ¿Para qué sirve un breakpoint con `log` y `breakif 0`? Para registrar valores sin detener la ejecución. Es útil en bucles donde el programa pasa muchas veces.

5. ¿Un breakpoint de memoria vigila bytes o páginas? Vigila un rango de bytes, pero internamente trabaja con protecciones de página. Por eso puede detenerse ante un acceso cercano al rango.

6. ¿Qué es el entry point? La primera instrucción del código del programa, donde el cargador le pasa el control. No es necesariamente `main`.

Antes de avanzar

Tenés que poder:

- Abrir un programa, moverte y reconocer las pestañas principales.
- Usar F7, F8 y F9 con intención.
- Poner los tres tipos de breakpoint y explicar en qué se diferencian.
- Configurar un breakpoint condicional con log.
- Guardar un parche en un programa propio y explicar por qué no lo harías en software ajeno.

Puerta de salida

Si podés responder "¿quién escribe este buffer?" con un breakpoint de memoria y "¿cuántas veces pasa por acá y con qué valor?" con uno condicional, ya usás el debugger como herramienta, no como curiosidad.

Fuentes de esta lección

- `Reversing desde cero, cap. 01` — interfaz, ventanas, F7/F8/F9, entry point.
- `Reversing desde cero, cap. 07` — breakpoints por software, hardware, memoria, condicionales y log.
- `Reversing desde cero, cap. 11` — etiquetas, análisis visual y parcheo didáctico.
- `Reversing desde cero, cap. 22` — PEB y detección de debugger.
- `Reversing desde cero, cap. 24` — cuatro chequeos anti-debug en un crackme propio.
- `Reversing desde cero, cap. 28` — WinProc y Dialog procedure, `WM_COMMAND`/Control ID, breakpoint condicional sobre el mensaje y el método `SendMessage` + breakpoint de ejecución en la sección de código.
- `Reversing desde cero, cap. 21` — lectura de `DR0`–`DR3` desde el `ContextRecord` como detección de breakpoints de hardware.
- Curso IDA · tutes 9, 10 y 12 — IDA como debugger (soporta varios) y práctica dinámica sobre crackmes propios (`HOLA_REVERSER`, `TEST_REVERSER`).

Síntesis de estudio de esta lección

Núcleo conceptual: el recorrido de la interfaz del capítulo 01, los tres breakpoints del capítulo 07, el log `{EAX}` y las etiquetas del capítulo 11, y los chequeos de los capítulos 22 y 24. **Conocimiento externo:** el mecanismo `INT3`, los registros `DR` y las protecciones de página. **Recomendación editorial:** el límite sobre parcheo y la corrección sobre "cuatro por proceso".

Ampliación editorial basada en fuentes externas — Flujo operativo de trinchera en x64dbg y extracción masiva de cadenas cifradas

El uso ágil del depurador dinámico y la desofuscación rápida de cadenas constituyen dos habilidades prácticas decisivas (x64dbg Documentation):

1. Hoja de Ruta Táctica de Operaciones en x64dbg

- **Atajos de Navegación Esenciales:** `F2` (Poner/quitar breakpoint de software), `F7` (Paso a paso dentro de la función), `F8` (Paso a paso sobre la función), `Ctrl+F9` (Ejecutar hasta el `ret` de la función actual), `F9` (Reanudar ejecución continua) y `Ctrl+G` (Saltar a una dirección o expresión).
- **Follow in Dump:** Clic derecho sobre cualquier registro en el panel de CPU o sobre una dirección en el Stack -> *Follow in Dump*: proyecta inmediatamente el contenido de la memoria apuntada en la vista hexadecimal.
- **Búsqueda de Referencias Intermodulares:** Clic derecho en el desensamblado -> *Search for* -> *Current Module* -> *Intermodular calls*: lista todas las llamadas a APIs importadas por el binario. Permite seleccionar múltiples funciones (ej. `RegOpenKeyEx`, `CreateFileW`, `InternetOpenA`) y presionar `F2` para establecer breakpoints simultáneos en todas ellas.
- **Memory Map (`Alt+M`):** Visualiza todas las páginas de memoria del proceso. Presionar `Ctrl+B` permite buscar cadenas o secuencias de bytes en toda la memoria dinámica. Facilita la detección de bloques con protección `PAGE_EXECUTE_READWRITE` (`ERW`), indicio clásico de payloads inyectados o shellcode.
- **Reconstrucción con Scylla:** Menú *Plugins* -> *Scylla* integrado en x64dbg: automatiza la detección del OEP (*IAT Autosearch*), lectura de importaciones (*Get Imports*), volcado a disco (*Dump*) y corrección de la tabla de importaciones (*Fix Dump*).

2. Los 3 Métodos Estándar para Extraer Cadenas Cifradas en Masa

1. **Método 1 — Breakpoint Condicional con Log en el Retorno del Decodificador (x64dbg):** Cuando un binario tiene decenas de llamadas a una rutina decodificadora XOR o RC4, no se debe revertir una por una a mano. Se localiza la instrucción `ret` de la función decodificadora y se configura un breakpoint:

   ```
   Log text:        "Cadena descifrada: {utf8@rax}"   ; (o {utf16@rax} según corresponda)
   Break condition: 0                                 ; Condición 0: no pausa la ejecución
   ```

   Al presionar `F9`, el programa se ejecuta a velocidad normal y la pestaña *Log* de x64dbg vuelca automáticamente todas las cadenas descifradas secuencialmente.
2. **Método 2 — Scripting Estático en IDAPython / Ghidra:** Si la clave está embebida y el algoritmo es determinista, se escribe un script que localice las referencias cruzadas (XREFs) al buffer cifrado y aplique la operación directamente en la base de datos de IDA:

```
import ida_bytes
def decode_xor(ea, length, key):
    raw = ida_bytes.get_bytes(ea, length)
    return bytes([b ^ key for b in raw]).decode('latin1', errors='ignore')
```

3. **Método 3 — Micro-emulación con Unicorn Engine:** Permite aislar únicamente los 30 o 40 bytes de la rutina decodificadora, inicializar los registros de la CPU virtual con los punteros y longitudes deseadas, y ejecutar el bloque emulado sin interactuar con dependencias del sistema operativo ni requerir un entorno Windows activo.

Ampliación editorial basada en fuentes externas — Patrón de doble breakpoint encadenado para funciones in-place y plugins tácticos

En el flujo de extracción masiva de cadenas cifradas, existe un caso frecuente donde el truco estándar de leer `{utf8@rax}` en el `ret` no funciona:

1. Funciones Decodificadoras void con Buffer In-Place

Cuando la función es `void decodificar(char* buffer, size_t len)`, el buffer se descifra sobre sí mismo (*in-place*) y la función no devuelve ningún puntero en `RAX`. En x64, el puntero al buffer llega en `RCX`, pero `RCX` es un registro volátil (*scratch*) que el compilador suele reutilizar como contador de bucle o índice, perdiéndose su valor antes de llegar al `ret`.

Solución: Técnica de Doble Breakpoint Encadenado con Variable de Usuario ($ptr):

1. **En el prólogo de la función decodificadora (dirección inicial):**

```
Break condition: 0
Command text:    $ptr = rcx    ; Guarda el puntero original antes de que el bucle lo
pise
```

2. **En la instrucción `ret` de la función:**

```
Break condition: 0
Log text:        "Cadena descifrada: {utf8@$ptr}"    ; Lee el buffer ya descifrado en
memoria
```

Al reanudar con `F9`, la consola de Log vuelca todas las cadenas limpias sin detener nunca la ejecución.

2. Ecosistema de Plugins Tácticos para x64dbg

- **ScyllaHide:** Plugin indispensable que intercepta a nivel de Ring 3 las APIs de anti-debug más comunes (`PEB.BeingDebugged` , `NtGlobalFlag` , `CheckRemoteDebuggerPresent` , `ProcessDebugPort` y temporizadores `RDTSC` / `GetTickCount`) con un solo clic.
- **xAnalyzer:** Analizador heurístico estático para x64dbg que reconoce automáticamente los prototipos de funciones de Windows y anota los nombres de los argumentos en el desensamblado, emulando la clásica vista contextual de OllyDbg en binarios modernos de 64 bits.
- **OllyDumpEx / SwissArmyKnife:** Permite realizar volcados selectivos de regiones de memoria específicas o secciones individuales directamente desde el *Memory Map* sin necesidad de reconstruir el binario completo.

Nota de alcance arquitectónico — Expresión getdword(sp+8) en WindowProc (x86 vs. x64)

La expresión `getdword(sp+8) == 0x111 && getdword(sp+0xC) == 0x13A` de esta lección aplica a binarios de **32 bits (x86)** (donde `__stdcall` apila `hWnd` en `[esp+4]`, `uMsg` en `[esp+8]` y `wParam` en `[esp+0xC]`) y asume que la palabra alta de `wParam` (código de notificación, p. ej. `BN_CLICKED = 0`) vale cero.

Ampliación editorial basada en fuentes externas — Condiciones para WndProc en x64 y función arg.get() en x64dbg

Según la especificación oficial de WindowProc y WM_COMMAND en Microsoft Learn *(Hecho confirmado)* y la documentación de Expressions & Functions de x64dbg *(Hecho confirmado)*:

- **En binarios de 64 bits (x64):** Los cuatro argumentos llegan en `RCX` (`hWnd`), `RDX` (`uMsg`), `R8` (`wParam`) y `R9` (`lParam`), y el ID del control ocupa solo los 16 bits bajos (`LOWORD(wParam)`):

```
breakif: rdx == 0x111 && (r8 & 0xFFFF) == 0x13A
log:     "WM_COMMAND: notif={r8>>16} id={r8&0xFFFF} hwnd={rcx}"
```

- **Expresión universal (x86 y x64) en el prólogo con `arg.get(n)`:**

```
breakif: arg.get(1) == 0x111 && (arg.get(2) & 0xFFFF) == 0x13A
log:     "uMsg={arg.get(1)} control_id={arg.get(2)&0xFFFF}"
```

IDA y el análisis estático

> **Objetivo de la lección**
>
> Al terminar vas a poder abrir un ejecutable en IDA (o Ghidra), encontrar una cadena, llegar al código que la usa, leer el grafo de una función, renombrar variables y comentar el flujo, y sincronizar direcciones con el debugger.

Prerrequisitos: PE, instrucciones, flujo y llamadas. **Duración sugerida:** 5 horas. Estimación aproximada.

Para qué sirve un desensamblador

Un debugger te muestra el programa **ejecutándose**. Un desensamblador te muestra el programa **completo, sin ejecutarlo**. Esa diferencia importa por tres razones:

1. Hay código que no podés ejecutar: malware, un binario de otra arquitectura, una función a la que no sabés cómo llegar.
2. Hay funciones que se llaman una vez cada mil ejecuciones y no querés esperar a que pasen.
3. El análisis estático te deja **leer todo** el código, no solo el camino que tomaste.

El capítulo 14 lo dice con un ejemplo claro: si estás analizando un virus, no lo corrés para ver qué hace; primero lo mirás estático. Y si estás buscando un bug en una función que no sabés cómo se alcanza, la podés analizar igual (`Reversing desde cero, cap. 14`, 00:03:57–00:05:30).

Abrir el archivo

1. **File** → **Open** y elegí el `.exe`.
2. IDA detecta el formato: `Portable executable for 80386 (PE)` indica un PE de 32 bits; si fuera de 64, lo diría.
3. En el diálogo de carga podés aceptar los valores por defecto. *Manual load* sirve si querés cargar secciones que IDA no trae por defecto (por ejemplo, el header).
4. Si el binario tiene símbolos PDB, IDA te los pide. Si no, cancelá: IDA hará su mejor esfuerzo.
5. Esperá a que termine el autoanálisis (el indicador deja de girar).

El capítulo 14 muestra este recorrido y aclara que la versión gratuita (IDA Free) tiene limitaciones: menos procesadores, un solo decompilador y, según la versión, solo IDC en lugar de Python (00:00:40–00:02:30).

El tutorial escrito de `descargas_sueltas` agrega un dato práctico: para saber si un PE es de 32 o 64 bits antes de abrirlo, mirá los bytes que siguen a la firma `PE`. En el documento, un binario de 64 bits muestra `PE..d†` y uno de 32 bits `PE..L`; la diferencia está en la cabecera opcional (PE32+ vs PE32) (`Introducción al reversing con IDA Pro desde cero, 2016`, sección "Lo primero es lo primero").

> **Sobre las versiones**
>
> El curso usa IDA de 32 bits de una versión de escritorio. La interfaz cambió bastante en las versiones 8 y 9, y las capacidades de IDA Free también. Los conceptos —strings, xrefs, grafo, renombrado, rebase — se mantienen; los botones y atajos pueden variar. Si un atajo no funciona, buscalo en el menú.

Las ventanas que vas a usar

Ventana	Cómo abrirla	Para qué
IDA View	`View → Open subviews → Disassembly`	Código desensamblado
Hex View	`View → Open subviews → Hex`	Bytes del archivo
Strings	`Shift+F12`	Cadenas del binario
Imports	`View → Open subviews → Imports`	Funciones que usa
Exports	`View → Open subviews → Exports`	Funciones que ofrece
Functions	`View → Open subviews → Functions`	Lista de funciones detectadas
Names	`View → Open subviews → Names`	Símbolos y etiquetas
Structures	`View → Open subviews → Structures`	Tipos y structs
Graph	`Espacio` sobre una función	Grafo de bloques

Ejemplo paso a paso: de una cadena al código

Este es el flujo que vas a repetir siempre. Lo hacemos con el programa de práctica del capítulo 15, que muestra el mensaje "el primer número debe ser menor a ese valor".

Paso 1. Buscar la cadena. `Shift+F12` abre la ventana de strings. Con `Ctrl+F` filtrás por texto. También podés usar `View → Open subviews → Strings`.

Paso 2. Ir a la cadena. Doble clic sobre ella: IDA te lleva a la sección `.rdata`, donde está almacenada.

Paso 3. Ver quién la usa. Con el cursor sobre la cadena, apretá `X` (*cross references*). IDA te muestra las instrucciones que la referencian. Elegí una y te lleva al código.

Paso 4. Leer el grafo. Apretá `Espacio` para ver los bloques. Cada bloque es una secuencia de instrucciones sin saltos internos; las flechas verdes/rojas son los caminos. En el capítulo 15, Ricardo pinta los bloques: verde para el camino bueno, salmón para el malo, celeste para el recorrido (`Reversing desde cero, cap. 15`, 00:02:16–00:06:00).

Paso 5. Renombrar y comentar. `N` renombra una función, variable o etiqueta; `;` agrega un comentario; `Y` cambia el tipo. Renombrar `var_18` como `primer_valor` hace que todas las referencias se actualicen y el código se vuelva legible.

Paso 6. Decompilar. `F5` abre el decompilador (si la versión lo incluye). Te da una vista en C aproximada. No es el código original: es una reconstrucción. Sirve para entender la forma general, no para copiar y pegar.

Sincronizar IDA con el debugger

Hay dos problemas clásicos:

1. **La base cambia.** IDA muestra la imagen en su `ImageBase` preferida (por ejemplo `0x400000`), pero el debugger la carga en otra dirección por ASLR. En el capítulo 14, IDA muestra `0x401040` y el debugger `0x410040`; la diferencia es la base.
2. **Las direcciones absolutas no coinciden.** Aunque rebasees, un byte puede estar en una sección con otra alineación.

Soluciones:

- **Rebasar IDA** a la base que ves en el debugger: `Edit → Segments → Rebase program`. En el capítulo 14, Ricardo rebasea de `0x400000` a `0x40000` para que coincida con el debugger (00:05:30–00:08:00).
- **Comparar por file offset.** En el debugger, `Ctrl+Shift+G` te da el offset de una instrucción; en IDA, `Jump → Jump to file offset` te lleva al mismo lugar. En el ejemplo, el offset `0x440` coincide en ambos (00:08:00–00:09:00).
- **Desactivar ASLR** en una copia del binario si necesitás direcciones estables (es lo que hace el capítulo 26b para trabajar cómodo). Es una modificación del archivo: hacela sobre una copia, nunca sobre el original.

Buscar un valor inmediato y las constantes con nombre

A veces no tenés una cadena que te lleve al código, pero sí un número: el identificador de un botón, un código de error, un mensaje de Windows. IDA permite buscarlo.

- **Search → immediate value** (`search for immediate value`) encuentra todas las instrucciones donde aparece ese número. Si el botón "Check" tiene Control ID `0x7D0`, buscás `0x7D0` e IDA te lista cada lugar donde se usa; manteniendo `Shift` podés hacer doble clic para recorrerlos.
- **Constante con nombre** (tecla `M`, *standard symbolic constant*): sobre un número que es en realidad una constante de Windows, `M` te ofrece el nombre simbólico. Un `0x111` se vuelve `WM_COMMAND`; un `0x201`, `WM_LBUTTONDOWN`. Eso hace legible el `switch` que reparte los mensajes de una ventana.
- Si la opción `M` no aparece sobre un `case`, apretá `F5` (decompilador) y probá `M` ahí: suele reconocer la constante igual.

El capítulo 24b hace esto sobre un crackme propio: busca el Control ID con *search immediate value*, llega al `WinProc` y traduce los números de mensaje a `WM_COMMAND` y `WM_LBUTTONDOWN` con las constantes simbólicas (`Reversing desde cero, cap. 24b`).

Proximity View: ver y buscar caminos entre funciones

El **Proximity View** (menú del grafo, tecla `-` para alejarte) muestra las funciones como un árbol de llamadas en vez de instrucciones. Por defecto muestra dos niveles: lo que la función llama y lo que la llama.

- Los nodos con "corona" son llamadas a la **IAT** (APIs importadas); si una llamada no continúa hacia abajo, es una API que se usa y vuelve (por ejemplo `MessageBox`).
- Doble clic en un nodo lo **centra** y expande dos niveles desde ahí; así seguís solo la rama que te interesa (por ejemplo, de `WinMain` a `WinProc`).
- Clic derecho → *Collapse children / parents* deja una función sola, para no llenar la pantalla.

Lo más útil es **buscar un camino**: con `Add node by name` agregás la función (o variable) destino, y con clic derecho → **Find path** IDA te dibuja la ruta estática que conecta dos funciones, si existe. Sirve cuando el programa es grande y querés saber "¿desde el inicio, cómo se llega hasta la función que muestra *Correcto*?". No siempre hay camino estático (a veces el flujo se arma en tiempo de ejecución), pero cuando lo hay, ahorra horas.

Esto lo muestra el capítulo 24b, que usa *Add node by name* + *Find path* para conectar el inicio del programa con una cadena de la sección de datos, y recomienda activar *use name for addresses* para que el grafo se lea (`Reversing desde cero, cap. 24b`).

Cuando no hay símbolos

Si el binario no trae símbolos, IDA muestra `sub_401000` en vez de `printf`. Peor aún: a veces las DLL están **embebidas** en el `.exe` (compilación estática), así que ni siquiera ves una llamada a una API conocida. Cómo reconocer una función igual:

1. **Por sus argumentos.** Una función con un solo argumento que siempre es una cadena, o con una cadena y después un número, tiene pinta de `printf`. Mirá todas sus referencias: si el patrón de argumentos se repite, confirma la hipótesis.
2. **Siguiendo a una API conocida.** Aunque la DLL esté embebida, `printf` termina llamando a `WriteConsole`. Con el Proximity View podés dejar la función sospechosa sola, agregar `WriteConsoleW` y buscar el camino: si llega, era una envoltura de `printf`.
3. **Confirmando en dinámico.** Si sospechás que una función sin argumentos es `getchar`, ejecutá: si se queda esperando una tecla, era `getchar`.
4. **Con Lumina** (solo IDA con licencia): el servidor compara las funciones con una base y les pone nombre. La versión gratuita o sin licencia no accede al servidor, así que no cuentes con eso.

El capítulo 34 ("Reverseando sin símbolos") usa estas tres primeras técnicas para identificar `printf`, `getchar` y `strcat` en un binario sin símbolos (`Reversing desde cero, cap. 34`). El mismo capítulo resuelve además un keygen; esta guía toma solo las técnicas de **identificación de funciones**, no la generación de seriales.

Reconstruir estructuras y vtables

Cuando una función usa `[reg+offset]` muchas veces sobre la misma base, lo más probable es que esté recorriendo una **estructura**. IDA deja trabajarlas aunque no haya símbolos:

- **Aplicar una estructura a un operando:** parado sobre el offset, apretá `T` (*structure offset*) y elegí la estructura; IDA te dice a qué campo corresponde (por ejemplo, `+0x58` → `F1`). A partir de ahí el listado se lee con nombres en vez de números.

- **Crear una estructura vacía y llenarla:** en la ventana *Structures*, `Insert` crea una estructura; con `D` agregás un campo; con clic derecho → *Array* (sin la tilde *create as array*) la agrandás hasta el tamaño que viste que reserva el programa (por ejemplo, si hace `new` de `0x3C`, creás una de `0x3C`). Después vas nombrando cada campo a medida que descubrís qué guarda (el retorno de `fopen` → `FILE*`, etc.).
- **Padding:** recordá que el tamaño puede incluir relleno por alineación (ver la lección de memoria); no asumas que los campos son contiguos sin huecos.

Para un **objeto de C++**, el primer campo (offset 0) suele ser el puntero a la **vtable**. El cap. 33 arma dos estructuras —una para el objeto y otra para la vtable con los punteros a los métodos virtuales (`suma`, `operator+`, `entrada`, `chequeo`)— y las aplica con `T`. Así, `mov edx, [obj]; call [edx+8]` se lee como "llamar al tercer método virtual".

> **Las llamadas virtuales no dejan xref directa**
>
> Un método virtual se llama a través de la vtable, así que `X` sobre la función solo muestra la referencia **desde la vtable**, no desde el `call`. El cap. 33 resuelve esto agregando una **referencia cruzada manual** (`View → Open subviews → Cross references → Add cross reference`) desde el sitio del `call` hacia el método. Conviene expresarla como **offset** del módulo (`modulo+0x92`) y no como dirección fija, para que ASLR no la rompa. Hecho esto, `X` y el Proximity View ya muestran de dónde viene la llamada.

Dos atajos de navegación que ayudan con listados densos (cap. 20b): con un registro o campo resaltado, `Alt+↓` / `Alt+↑` salta entre sus apariciones; y en una lista de referencias o de strings, **`Shift`+doble clic** te lleva a cada una sin cerrar la ventana.

Con símbolos es más directo (cap. 23). Si el binario trae PDB, la estructura ya está definida en *Local Types* (después de sincronizarla con el IDB) y el **constructor** se reconoce por el patrón `Nombre::Nombre`. Aplicás la struct con `T` y cada `[reg+offset]` muestra su campo por nombre. Ojo con un caso frecuente: cuando un campo se pasa como **buffer** a una función (por ejemplo, el destino de `fread`), vas a ver `lea`/dirección `&campo`, no su contenido —es un puntero al campo, no su valor—. Reconocer las APIs por sus argumentos (`fopen` devuelve `FILE*`, `fread(buf,size,count,stream)`, `LoadLibrary`→`HMODULE`, `GetProcAddress(hmod,"printf")`) termina de dar sentido a la estructura (`Reversing desde cero, cap. 23`).

> **x64dbg también entiende estructuras**
>
> No es exclusivo de IDA. En x64dbg, la pestaña *Struct* permite **importar** una estructura con *parse header* (cargás un `.h` con su definición, bien formateado) y luego *Visit type* con el nombre del tipo y la dirección (por ejemplo, la que apunta a una `SYSTEMTIME` que llenó `GetSystemTime`). Es más áspero que en IDA, pero sirve para leer los campos en vivo mientras traceás (cap. 19).

> **En el Curso IDA: una serie entera dedicada a estructuras (25–28)**
>
> El Curso IDA le dedica cuatro partes seguidas al tema. Lo más útil para practicar: el ejercicio `IDA_STRUCT.7z` (ejecutable `ConsoleApplication4.exe`) **viene con su** `.pdb`, pero el autor propone resolverlo **primero sin símbolos** —borrando o renombrando el `.pdb`— porque "es más complejo, pero es más real", y recién después comparar con los símbolos puestos. Es la mejor forma de ejercitar lo de arriba: armar la estructura a mano con `T` / `D` y nombrar campos, contra tenerla servida en *Local Types* (Curso IDA · tutes 25–28, Estructuras).

Patrones de datos más allá de una estructura

Una estructura sola es el caso fácil. En binarios reales vas a ver **colecciones** de estructuras, y cada una deja un patrón propio en el assembly (**conocimiento externo**, salvo lo citado):

- **Arreglo de estructuras.** El acceso a un elemento tiene la forma `base + índice × tamaño + campo`. Lo vas a ver como `imul rax, rcx, 18h` seguido de `[rbx+rax+8]`, o como un índice escalado `[rbx+rcx*8+10h]`. El multiplicador te da el **tamaño del elemento** y el desplazamiento final, el **campo**. Con eso creás la estructura del tamaño correcto y la aplicás como arreglo.
- **Lista enlazada simple.** Un bucle que hace `mov rbx, [rbx+8]` hasta que `rbx` vale cero: el `+8` es el campo *siguiente*. El resto de los accesos sobre `rbx` dentro del bucle son los campos del nodo.
- **Lista doblemente enlazada de Windows (`LIST_ENTRY`).** Windows usa una estructura con dos punteros, `Flink` (siguiente) y `Blink` (anterior), **embebida** dentro de cada elemento, y listas **circulares** con una cabecera. Dos consecuencias para leerla: el puntero apunta al campo `LIST_ENTRY`, no al comienzo del elemento, así que verás un `sub` / `lea` que **resta** el offset del campo para llegar al inicio (lo que en C hace la macro `CONTAINING_RECORD`); y el recorrido termina cuando se vuelve a la **cabecera**, no cuando aparece un `NULL`. Es la misma forma que tiene la lista de módulos del PEB de 2.14.
- **Árboles.** Nodos con dos punteros a hijos y una comparación que decide por cuál bajar; los contenedores ordenados de C++ (`std::map`) se ven así (2.18).

*(Ejemplo **ilustrativo**: recorrido de una lista circular de Windows en x64.)*

```
lea   rdi, [rcx+10h]        ; rdi = &cabecera (LIST_ENTRY dentro de un objeto)
mov   rbx, [rdi]            ; rbx = cabecera.Flink
bucle:
cmp   rbx, rdi              ; ¿volvimos a la cabecera? -> fin (lista circular)
je    fin
lea   rsi, [rbx-20h]        ; inicio del elemento = &entry - offset del campo (CONTAINING_RECORD)
mov   eax, [rsi+8]          ; un campo del elemento
mov   rbx, [rbx]            ; siguiente: entry.Flink
jmp   bucle
fin:
```

Lectura: el campo `LIST_ENTRY` está a `+0x20` dentro de cada elemento (por el `lea rsi, [rbx-20h]`), y el elemento tiene al menos un campo de 4 bytes en `+8`. Con esos dos datos creás la estructura del elemento y la de la cabecera.

Cadenas: codificaciones y por qué no siempre aparecen

La ventana de strings es la puerta de entrada más rápida, pero solo muestra lo que el programa guarda **tal cual** en el archivo. Conviene saber qué se pierde:

- **ANSI frente a UTF-16.** Las funciones de Windows que manejan texto suelen tener una versión **A** (página de códigos, un byte por carácter) y una **W** (Unicode, en Windows UTF-16, dos bytes). En un volcado hexadecimal, un texto ASCII guardado como UTF-16LE se ve como `H.o.l.a.` con ceros intercalados. Si la búsqueda de cadenas está configurada solo para C/ANSI, esas cadenas **no aparecen**: en IDA se agregan los tipos Unicode desde la configuración de la ventana de strings; Ghidra los incluye en *Defined Strings*. Que el programa llame a `CreateFileW` en vez de `CreateFileA` ya te dice qué codificación buscar.
- **Cadenas con longitud.** Algunos lenguajes guardan cadenas como puntero + longitud, sin el `0` final (Go, Rust y otros; ver 2.13). El buscador de cadenas las puede mostrar pegadas unas con otras o cortadas.
- **Cadenas armadas en el stack.** El programa escribe el texto de a pedazos con instrucciones `mov` de inmediatos imprimibles (`mov dword ptr [rsp+20h], 6C6C6548h` = `"Hell"` en little-endian). No hay ninguna cadena en `.rdata`; la pista es una ráfaga de `mov` con constantes que, leídas como bytes, son caracteres.
- **Cadenas cifradas.** Se guardan transformadas y una rutina las descifra justo antes de usarlas (un bucle con `xor` es lo más simple; ver 2.10). Se recuperan siguiendo la rutina, ejecutando hasta después de que descifre o con herramientas que extraen cadenas armadas en el stack y decodificadas (por ejemplo, FLOSS de Mandiant, externa al curso).

Regla práctica: si un programa claramente muestra textos al usuario y la ventana de strings está casi vacía, alguna de estas cuatro situaciones está pasando, y eso ya es un dato del análisis.

Estrategias de lectura

No leas una función de arriba a abajo sin plan. Algunas tácticas que funcionan:

1. **Empezá por las strings.** En programas con interfaz, los mensajes ("acceso denegado", "ingrese la clave") te llevan directo a la lógica.
2. **Buscá las APIs conocidas.** `GetWindowTextA`, `strcmp`, `MessageBoxA`, `CreateFileA` son pistas. En Imports tenés la lista.
3. **Seguí las referencias.** `X` sobre una función te dice desde dónde se llama. A veces el `main` es el nodo con más referencias.
4. **Nombrá mientras leés.** Un binario sin símbolos se vuelve legible cuando renombrás `sub_401000` como `check_password` y `var_8` como `largo`.
5. **Anotá las hipótesis.** Escribí "si el primer valor es <= 50, sigue por acá" y verificalo en el debugger.

El capítulo 15 usa exactamente este método: busca la cadena, llega al código, identifica el `printf` / `scanf`, renombra las variables, deduce las condiciones y las confirma ejecutando (00:06:00–00:19:00).

> **En el Curso IDA: el mismo método, dicho por Ricardo**
>
> El Curso IDA plantea este enfoque desde el arranque: cuando todavía no tenés práctica con el análisis estático, **comentá y renombrá lo poco que vas entendiendo** y resolvé el resto en el debugger; con práctica vas necesitando el debugger cada vez menos, hasta reversear casi sin ejecutar. Y algo clave para no agobiarte: **no se reversea el programa entero**, sino la o las funciones de la zona que necesitás para tu objetivo (Curso IDA · tute 3, El loader y tute 8, Trabajar en el loader).

IDA Free, Ghidra y alternativas

Herramienta	Fortalezas	Límites
IDA Free	La misma base de IDA; buena para empezar	Menos procesadores, decompilador limitado, sin plugins de pago
IDA Pro	Estándar de la industria; decompilador maduro	Licencia cara
Ghidra	Gratis y open source; decompilador integrado; scripting en Java/Python	Interfaz más pesada; consume más recursos
Binary Ninja	Interfaz moderna; buena API	De pago
radare2 / Cutter	Gratis, scriptable, multiplataforma	Curva de aprendizaje alta
x64dbg	Debugger, no desensamblador estático	No reemplaza a IDA para análisis estático

Para este curso, **IDA Free o Ghidra** son suficientes. El capítulo 14 recomienda IDA Free si no se tiene Pro; esta guía agrega Ghidra como alternativa gratuita y activa.

> **Una recomendación del audio y del documento que no se copia**
>
> En el capítulo 14, el autor sugiere buscar una copia "por internet" de IDA Pro si no se puede pagar (00:01:00–00:02:00). El documento de `descargas_sueltas` va más lejos: indica buscar "IDA PRO 6.8 + HEXRAYS" en Google y usar la contraseña de `install_pass.txt`. Esta guía **no** reproduce esas instrucciones ni distribuye claves. Usá IDA Free, Ghidra o una licencia educativa.

Práctica

1. Abrí tu `check.exe` en IDA Free o Ghidra.
2. Buscá la cadena "Correcto" con `Shift+F12`, llegá al código con `X` y leé el grafo.
3. Renombrá la función `main` si no está nombrada, y las variables de los dos números.
4. Agregá comentarios en las dos comparaciones.
5. Si tu versión tiene decompilador, compará la vista `F5` con el assembly. Marcá tres diferencias.
6. Rebaseá IDA a la base que muestra x64dbg y comprobá que una dirección coincide. Si no coincide, usá el file offset.

7. Escribí en tu bitácora: qué string usaste, qué xref elegiste, qué función renombraste y qué hipótesis confirmaste.
8. Agregá a tu programa un mensaje con `MessageBoxW` y otro armado carácter por carácter en un arreglo local. Comprobá cuál aparece en la ventana de strings con la configuración por defecto, cuál solo al activar Unicode y cuál no aparece nunca.
9. Agregá un arreglo de estructuras y recorrelo en un bucle. En el listado, identificá el tamaño del elemento por el multiplicador y creá la estructura correspondiente.

Preguntas de repaso

1. ¿Cuál es la diferencia principal entre IDA y x64dbg? IDA es un desensamblador estático: analiza el archivo sin ejecutarlo. x64dbg es un debugger: ejecuta el programa y muestra su estado.

2. ¿Para qué sirve `X` sobre una cadena? Para ver las referencias cruzadas: las instrucciones que usan esa cadena. Es el camino más rápido de la cadena al código.

3. ¿Qué hace `N`? Renombra un símbolo (función, variable, etiqueta). Todas las referencias se actualizan.

4. ¿Por qué las direcciones de IDA y del debugger no coinciden? Porque ASLR carga la imagen en una base distinta de la preferida. Se soluciona rebaseando o comparando por file offset.

5. ¿El decompilador devuelve el código original? No. Reconstruye una aproximación en C a partir del assembly. Sirve para entender, no para copiar.

6. ¿Qué mirada conviene primero, la estática o la dinámica? Depende. Para un programa con interfaz y strings claras, la estática orienta rápido. Para confirmar un valor concreto, la dinámica es más directa. Lo habitual es alternar.

7. El programa muestra mensajes pero la ventana de strings está casi vacía. ¿Qué cuatro causas revisás? Cadenas UTF-16 no incluidas en la búsqueda, cadenas con longitud sin `0` final, cadenas armadas en el stack con `mov` de inmediatos y cadenas cifradas que se descifran en ejecución.

8. Ves `[rbx+rcx*8+10h]` dentro de un bucle. ¿Qué te dice? Que probablemente es un arreglo de elementos de 8 bytes indexado por `rcx`, y que `+10h` es el desplazamiento del arreglo o de un campo respecto de la base.

9. ¿Por qué en una lista `LIST_ENTRY` de Windows ves una resta antes de leer los campos? Porque el puntero apunta al campo `LIST_ENTRY` embebido, no al inicio del elemento; la resta del offset del campo recupera el inicio (lo que hace `CONTAINING_RECORD`).

Antes de avanzar

Tenés que poder:

- Abrir un binario, buscar una cadena y llegar al código que la usa.
- Leer un grafo y distinguir el camino bueno del malo.
- Renombrar y comentar para que el listado se entienda.
- Sincronizar IDA con el debugger por rebase o por offset.

- Explicar por qué no usás una copia ilegal de IDA.
- Reconocer cadenas UTF-16, armadas en el stack o cifradas, y saber por qué no aparecen en la ventana de strings.
- Reconocer un arreglo de estructuras, una lista enlazada y una lista `LIST_ENTRY` por su patrón en el assembly.

> **Puerta de salida de la Etapa 2**
>
> Si podés analizar tu propio programa de punta a punta —estático y dinámico— y escribir un informe con las direcciones y los valores observados, terminaste la introducción al reversing. La Etapa 3 es una victoria rápida con .NET.

Fuentes de esta lección

- `Reversing desde cero, cap. 14` — interfaz de IDA, rebase, file offset, símbolos, limitaciones de IDA Free.
- `Reversing desde cero, cap. 15` — strings, xrefs, grafo, renombrado, decompilación y verificación dinámica.
- `Reversing desde cero, cap. 24b` — Proximity View, Find Path, search immediate value, constantes simbólicas (`WM_*`) y decompilador para confirmar un `case`.
- `Reversing desde cero, cap. 34` — identificar funciones sin símbolos por argumentos, por el camino a una API conocida y confirmando en dinámico (se omite la parte de keygen).
- `Reversing desde cero, cap. 20b` — aplicar y reconstruir estructuras con `T`, y navegación con `Alt+↑/↓` y `Shift`+doble clic.
- `Reversing desde cero, cap. 33` — objeto con vtable, reconstruir las dos estructuras y agregar cross-references manuales para las llamadas virtuales (se omite la parte de keygen).
- `Reversing desde cero, cap. 23` — estructura con símbolos, constructor `Nombre::Nombre`, `&campo` como buffer y reconocer `fopen` / `fread` / `LoadLibrary` / `GetProcAddress` (se omite la parte de keygen).
- `Reversing desde cero, cap. 19` — importar y visitar estructuras (`SYSTEMTIME`) en x64dbg con parse header / visit type.
- `Videos y material adicional, Video 33` — mapa de memoria y secciones como primer paso del análisis.
- `Videos y material adicional, Video 39` — ExeInfoPE y detección de código nativo/empaquetado.
- Curso IDA desde cero · 69 tutes ilustrados — texto y 3.278 capturas extraídos e indexados por tema (índice en ida-tutes); el método de lectura estática —comentar, renombrar y reversear solo lo necesario— es de los tutes 3 y 8.
- Curso IDA · tutes 25–28, Estructuras — serie dedicada a estructuras en IDA; el ejercicio `IDA_STRUCT` / `ConsoleApplication4.exe` se resuelve primero sin símbolos (lo más real) y después con el `.pdb`.
- `Introducción al reversing con IDA Pro desde cero (docx, 2016)` — tutorial escrito de la parte 1: IDA vs OllyDbg, PE32/PE32+, proximity view, strings, imports, hex dump.
- Microsoft Learn (conocimiento externo): Unicode in the Windows API, Working with Strings, LIST_ENTRY, Singly and doubly linked lists. Herramienta externa: FLOSS.

Síntesis de estudio de esta lección

Núcleo conceptual: el recorrido del capítulo 14, el ejemplo de `0x440`, las técnicas del capítulo 15 y la advertencia sobre IDA Free. **Conocimiento externo:** comparación con Ghidra y otras herramientas, atajos actuales, patrones de colecciones de datos (arreglos, listas, `LIST_ENTRY`, árboles) y codificaciones y formas de ocultar cadenas. **Ejemplo ilustrativo:** el recorrido de una lista circular. **Recomendación editorial:** la estrategia de lectura y el límite legal sobre copias de IDA Pro.

Cómo cambia el código con optimizaciones

De dónde sale esta lección

Este tema es indispensable en binarios modernos de 64 bits y forma parte del núcleo técnico avanzado de esta obra. El curso de reversing usa ejemplos compilados en modo depuración (`-O0`/Debug), donde el assembly sigue de cerca al código fuente. Los binarios reales vienen optimizados (`-O2`/Release) y se ven muy distintos. Esta lección cubre ese salto, que es donde más se traba quien recién sale de los ejercicios.

Objetivo de la lección

Al terminar vas a poder reconocer los patrones de optimización más comunes —inlining, *strength reduction*, división por constante, vectorización, *tail call*— y no confundirlos con "código raro" o con una protección.

Prerrequisitos: instrucciones, flujo, llamadas (Etapa 2). **Duración sugerida:** 3 horas.

El problema

En `-O0`, cada variable vive en el stack, cada operación es una instrucción y las funciones se llaman tal como las escribiste. En `-O2`, el compilador reescribe todo para que corra más rápido: mete funciones una dentro de otra, reemplaza multiplicaciones por sumas, borra código que "no hace falta" y reordena instrucciones. El resultado es correcto pero **no se parece** al fuente. Reconocer estos patrones es lo que separa leer un ejercicio de leer un programa real.

Strength reduction: multiplicar sin `mul`

El compilador cambia operaciones caras por otras baratas. La más común: multiplicar por una constante usando `lea` y shifts.

```
; x * 2
add eax, eax            ; o:  shl eax, 1
; x * 8
shl eax, 3
; x * 5   ->  x + x*4
lea eax, [rax + rax*4]
; x * 9   ->  x + x*8
lea eax, [rax + rax*8]
```

Si ves un `lea` haciendo cuentas (no calculando una dirección), es casi seguro una multiplicación por constante disfrazada. `lea eax, [rax+rax*4]` es `x*5`, no "un acceso a memoria".

División por constante: el número mágico

Dividir es carísimo, así que, en código optimizado, el compilador **casi nunca** usa `div` para dividir por una constante: lo habitual es que multiplique por un "número mágico" y desplace. Depende del compilador, la arquitectura y las opciones (sin optimizar, o con un divisor que no conoce en tiempo de compilación, sí vas a ver `div` / `idiv`).

```
; unsigned  x / 3
mov   eax, 0xAAAAAAAB     ; número mágico
mul   edx                 ; producto de 64 bits
shr   edx, 1              ; EDX = x / 3
```

Cuando veas una multiplicación por una constante aparentemente arbitraria (`0xAAAAAAAB`, `0x55555556`, `0xCCCCCCCD`…) seguida de un `shr`, es una **división por constante**, no un algoritmo secreto. Hay tablas y calculadoras para recuperar el divisor; lo importante es reconocer el patrón y no perder tiempo buscando sentido a la constante.

Inlining: la función que desaparece

Con `-O2`, una función corta que llamás una vez puede **desaparecer**: el compilador copia su cuerpo donde se usaba (*inlining*). Entonces:

- No vas a encontrar un `call` a esa función.
- El código de la función aparece **mezclado** dentro de quien la llamaba.
- Varias funciones del fuente pueden colapsar en un solo bloque grande.

La consecuencia para el análisis: no asumas "una función del fuente = una función en el binario". Buscá el comportamiento, no el nombre.

Vectorización: SSE donde esperabas un bucle

Un bucle que suma o copia un arreglo puede convertirse en instrucciones **packed** (`addps`, `movdqu`, `paddd`) que procesan varios elementos por vuelta. En vez de un bucle de 100 iteraciones de a 1, ves un bucle de 25 de a 4. Si un bucle "no tiene sentido" porque usa `xmm` y avanza de a 16 bytes, está vectorizado (ver la lección de punto flotante y SSE).

Tail call: un `jmp` donde esperabas `call`

Si lo último que hace una función es llamar a otra y devolver su resultado, el compilador reemplaza `call f; ret` por un simple `jmp f` (*tail call*). Ahorra un marco de stack. El efecto al analizar: una función "termina" saltando a otra con `jmp`, sin `ret` propio. No es un salto sospechoso ni ofuscación: es una llamada final optimizada.

Otros patrones frecuentes

Patrón	Cómo se ve	Qué es
`xor eax, eax`	poner 0	modismo para `eax = 0`
`test eax, eax` + `je`	en vez de `cmp eax, 0`	"¿es cero?"
`cmovcc` / `setcc`	decisión sin salto	el `if` se hizo sin bifurcar (ver lección de flujo)
`lea` con cuentas	aritmética	suma/multiplicación, no acceso a memoria
bloques reordenados	el camino "frío" (error) va al final	mejora la predicción de saltos
constantes plegadas	`mov eax, 480` donde el fuente decía `(a+b)*4`	*constant folding* en tiempo de compilación

Cómo convivir con esto

1. **Aprendé con `-O0`, analizá con `-O2`.** Compilá tu ejemplo en los dos modos y poné los listados lado a lado. Ver la misma lógica en ambas versiones entrena el ojo.
2. **Usá el decompilador.** Ghidra o Hex-Rays deshacen muchos de estos patrones: te muestran `x / 3` en vez del número mágico, y la función inlineada como una sola expresión. No es mágico, pero ahorra tiempo.
3. **Pensá en comportamiento, no en forma.** La pregunta no es "¿por qué hay un `lea` acá?", sino "¿qué valor queda en este registro al final?".

Práctica

1. Compilá una función `int f(int x){ return x*5 + x/3; }` con `-O0` y con `-O2`. Encontrá el `lea [rax+rax*4]` (el `*5`) y el número mágico (el `/3`).
2. Escribí dos funciones chicas donde una llame a la otra y compilá con `-O2`. ¿Desapareció la llamada (inlining)?
3. Hacé un bucle que sume un arreglo de 1000 `int` y compilá con `-O3 -mavx2`. ¿Ves registros `xmm` / `ymm` y avance de a varios elementos?
4. Abrí el `-O2` en Ghidra y compará el decompilado con tu fuente. ¿Recuperó la división por 3?

Preguntas de repaso

▼ **Ves `lea eax, [rax+rax*4]`. ¿Qué es?**

Una multiplicación por 5 (`x + x*4`). `lea` se usa acá para aritmética, no para calcular una dirección.

▼ **Aparece `mov eax, 0xAAAAAAAB` seguido de `mul` y `shr`. ¿Qué está pasando?**

Es una división por constante (por 3) implementada con un número mágico. El compilador evita `div`.

▼ **Una función termina con `jmp otra_funcion` en vez de `call` + `ret`. ¿Es sospechoso?**

> No. Es una *tail call*: la última acción era llamar y devolver, y el compilador la optimizó a un salto.

▼ **No encontrás el `call` a una función que sí está en el fuente. ¿Qué pudo pasar?**

> Inlining: el compilador copió su cuerpo dentro de quien la llamaba, así que no hay llamada separada.

Antes de avanzar

Tenés que poder:

- Reconocer *strength reduction* (`lea`/shifts) y división por constante (número mágico).
- Explicar por qué una función del fuente puede no existir como función en el binario.
- Identificar un bucle vectorizado y una *tail call*.
- Usar un decompilador para deshacer estos patrones.

> **Puerta de salida**
>
> Si podés mirar un listado `-O2` y decir "esto es `x*5`, esto es `x/3`, esta función está inlineada" sin frustrarte, estás listo para binarios reales.

Fuentes de esta lección

- Agner Fog, *Optimizing subroutines in assembly language* y *Instruction tables* (patrones de código generado y costos).
- Documentación de **GCC** (`-O1/-O2/-O3`) y **LLVM/Clang** sobre pases de optimización.
- *Hacker's Delight* (Warren) para división por constantes y trucos aritméticos.

> **Síntesis de estudio de esta lección**
>
> **Conocimiento externo (agregado por esta edición):** todos los patrones de optimización. El corpus tradicional usa binarios sin optimizar, así que este salto no estaba cubierto. **Recomendación editorial:** la estrategia "aprendé con Debug, analizá con Release" y los ejercicios.

Herramientas: del OllyDbg histórico al taller actual

> **Objetivo de la lección**
>
> Al terminar vas a poder elegir la herramienta adecuada para identificar, desensamblar, depurar o inspeccionar un binario, justificar por qué algunas del curso son históricas y registrar versiones y hashes de tu taller.

Prerrequisitos: debugger e IDA. **Duración sugerida:** 2 horas. Estimación aproximada.

Por qué hablar de herramientas

El conocimiento de reversing —registros, stack, saltos, PE— envejece poco. Las herramientas envejecen rápido. El curso de reversing mezcla material de 2007, 2016 y 2021, y usa OllyDbg en una parte y x64dbg en otra. Si no distinguís qué es concepto y qué es botón, vas a estudiar una interfaz muerta creyendo que estudias reversing.

El taller mínimo

Tarea	Herramienta	Estado	Nota
Depurar 32 y 64 bits	**x64dbg**	Activa	Gratis, open source, Windows
Desensamblar	**IDA Free**	Activa (versión gratuita)	Limitada pero suficiente para empezar
Desensamblar y decompilar	**Ghidra**	Activa	Gratis, open source, multiplataforma
Editar bytes	**HxD**	Activa	Gratis para uso personal; verificar licencia
Ver cabecera PE	**PE-bear**, **CFF Explorer**, **Detect It Easy**	Activas	Detect It Easy también identifica empaquetado
Ver procesos y handles	**Process Explorer** (Sysinternals)	Activa	Útil para el mapa de memoria y handles
Ver actividad de archivos/registro	**Process Monitor**	Activa	Para análisis dinámico controlado
Ver red	**TCPView**, **Wireshark**	Activas	Solo en laboratorio aislado
Automatizar	**Python** + `pefile`, `capstone`	Activas	Opcional
Detección	**YARA**	Activa	Para reglas defensivas

No necesitás instalar todo el primer día. x64dbg, un editor hexadecimal y un visor de PE alcanzan para las etapas 1 a 4.

El taller actual (2026): más allá de lo mínimo

> **Actualización externa**
>
> La literatura clásica no cubre estas herramientas (son más modernas). Las agregamos como estándar indispensable para que el taller esté al día. Todas corren o se usan sobre **Windows**; no son obligatorias para el recorrido base, pero sí para análisis real.
>
> Verificá siempre la versión y descargá desde el sitio oficial.

Cuando el taller mínimo se te queda corto, estas categorías son las que usa hoy un analista de Windows:

Categoría	Herramienta	Para qué
Instrumentación dinámica	**Frida**	Enganchar (*hook*) funciones de un proceso en vivo, leer/alterar argumentos y resultados con scripts. Potente para ver qué pasa sin trazar a mano.
Time-travel debugging	**WinDbg** + **TTD**	Grabar una ejecución completa y recorrerla **hacia atrás**. Ideal para "¿quién puso este valor?" sin reiniciar mil veces.
Diffing de binarios	**BinDiff**, **Diaphora**	Comparar dos versiones de un binario (por ejemplo, antes y después de un parche) para ver qué cambió. Clave en análisis de parches.
Extracción de strings	**FLOSS** (Mandiant)	Saca strings normales **y** las ofuscadas/apiladas que `strings` no ve.
Triage y capacidades	**CAPA**, **PEStudio**	Identifican qué "sabe hacer" un binario (red, cripto, persistencia) por patrones, sin ejecutarlo.
Clasificación/detección	**YARA**	Reglas para reconocer familias y rasgos. Defensivo.
Emulación y análisis simbólico	**Unicorn**, **Qiling**, **angr**, **Triton**	Ejecutar un fragmento sin el programa completo, o explorar caminos de forma simbólica (p. ej. "¿qué entrada llega a este bloque?").
Desensamblado programable	**Capstone** (leer) + **Keystone** (ensamblar)	Librerías para construir tus propias herramientas en Python.

> **Asistencia con IA (con cautela)**
>
> Hay plugins que usan modelos de lenguaje para **sugerir** nombres de funciones y variables o explicar un bloque decompilado (por ejemplo, asistentes para Ghidra/IDA). Pueden acelerar la lectura, pero **alucinan**: un nombre sugerido es una hipótesis, no un hecho. Usalos como punto de partida y **verificá** cada afirmación contra el código, igual que harías con el renombrado automático de un script.

Elegí por tarea, no por moda: para "¿quién escribe este buffer?" quizás te alcance un breakpoint de memoria en x64dbg; TTD y angr son para cuando el problema lo justifica.

> **En el Curso IDA: plugins y cuidado con la versión**
>
> El Curso IDA dedica una parte a los **plugins** que amplían IDA, con una advertencia que sigue valiendo: algunos no andan en la 6.8 pero sí en la 7.x, así que si no los podés instalar, al menos leé para qué sirven. Y muestra un **bug de la versión 6.8** (corregido en la 6.9) y cómo reconocerlo. Moraleja: **fijate qué versión de IDA tenés** antes de pelearte con un plugin que "no funciona" (Curso IDA · tute 13, Plugins y tute 11, Bug de la 6.8).

OllyDbg: herramienta histórica, no obsoleta como concepto

OllyDbg es un debugger de **ring 3 para binarios de 32 bits**, muy popular en los 2000. El curso de Cayetano de Juan Úbeda lo usa como herramienta principal en los videos 23 a 40, con una máquina virtual Windows 7 de 32 bits (`Videos y material adicional, Video 24`, 00:00:00 y 00:31:47).

Lo que OllyDbg enseña sigue siendo válido:

- Ver el estado de los registros y la pila.
- Poner breakpoints y trazar instrucción por instrucción.
- Leer el flujo de un `if` compilado.
- Identificar una comparación de clave.

Lo que cambió:

- No maneja binarios de 64 bits.
- No se actualiza desde hace años.
- Su interfaz y sus atajos no son los de x64dbg.
- Algunos plugins y técnicas de la época ya no funcionan en Windows modernos.

Conclusión: mirá los videos de OllyDbg por el método y los conceptos, no para aprender la interfaz. Después repetí el mismo ejercicio en x64dbg.

El flujo de trabajo con herramientas

```
1. Identificar      -> Detect It Easy / ExeInfoPE / PE-bear
2. Entender formato -> cabecera PE, secciones, imports
3. Análisis estático-> IDA Free / Ghidra
4. Análisis dinámico-> x64dbg (solo en laboratorio)
5. Documentar       -> bitácora + informe
```

Cada paso tiene una pregunta:

Paso	Pregunta que responde
Identificar	¿Qué tipo de archivo es? ¿Está empacado? ¿Es .NET?
Formato	¿Dónde está el código? ¿Qué imports usa?
Estático	¿Qué hace esta función? ¿Qué cadenas y APIs aparecen?
Dinámico	¿Qué valores toma en esta ejecución? ¿Qué camino sigue?
Documentar	¿Puede otra persona reproducir lo que hice?

El video 33 insiste en que el **mapa de memoria** es lo primero que abre al analizar un binario (`Videos y material adicional, Video 33`, 00:02:00–00:03:00). El video 39 muestra ExeInfoPE mirando cabecera, linker, secciones y entry point (`Videos y material adicional, Video 39`, 00:00:32–00:02:09).

Lo que no cambia

Estos conceptos valen igual en OllyDbg, x64dbg, IDA y Ghidra:

- La diferencia entre dirección virtual, RVA y offset.
- El stack y las convenciones de llamada.
- El significado de `cmp`, `test` y los saltos.
- La IAT y las llamadas indirectas.
- La diferencia entre análisis estático y dinámico.

Si entendés eso, cambiar de herramienta es aprender una interfaz nueva, no empezar de cero.

Práctica: inventario del taller

1. Instalá x64dbg, un editor hexadecimal y un visor de PE. Anotá versión y fecha de descarga.
2. Calculá el SHA-256 de cada instalador y guardalo en tu bitácora.
3. Abrí tu `check.exe` con el visor de PE, con x64dbg y con IDA Free. Completá:

Herramienta	Qué te mostró que las otras no
Visor de PE	
x64dbg	
IDA Free	

4. Buscá en el binario una cadena tuya ("Correcto") con las tres herramientas. ¿Cuál fue más rápida?
5. Escribí en tu bitácora qué herramienta usarías para: (a) saber si un binario está empacado, (b) encontrar quién escribe un buffer, (c) leer una función que nunca se ejecuta.

Preguntas de repaso

1. ¿Por qué OllyDbg se considera histórico? Porque es un debugger de 32 bits que ya no se actualiza. Sigue siendo útil para estudiar el método de la época, pero no es la herramienta recomendada para empezar hoy.

2. ¿x64dbg reemplaza a IDA? No. x64dbg es un debugger; IDA es un desensamblador estático. Se complementan.

3. ¿Qué herramienta usarías para saber si un binario está empacado? Detect It Easy, ExeInfoPE o un visor de PE: miran secciones, imports y entry point. La señal orienta, no concluye.

4. ¿Qué significa que una herramienta esté "activa"? Que recibe mantenimiento y funciona en sistemas actuales. No significa que sea obligatoria ni que sea la mejor para todo.

5. ¿Por qué registrar la versión de las herramientas? Porque un cambio de versión puede cambiar la interfaz o el comportamiento. Un informe reproducible dice con qué versión se hizo.

Antes de avanzar

Tenés que poder:

- Elegir herramienta para identificar, desensamblar, depurar y editar.
- Explicar por qué OllyDbg es histórico y qué se puede aprender de él.
- Registrar versiones y hashes de tu taller.
- Decir qué conceptos no dependen de la herramienta.

> **Puerta de salida**
>
> Si tu bitácora tiene el inventario del taller con versiones y hashes, estás listo para .NET.

Fuentes de esta lección

- `Videos y material adicional, Video 23` — introducción al reversing y limitaciones del análisis.
- `Videos y material adicional, Video 24` — OllyDbg y la VM Windows 7 de 32 bits.
- `Videos y material adicional, Video 33` — PE, secciones e imports.
- `Videos y material adicional, Video 39` — ExeInfoPE.
- `Reversing desde cero, cap. 14` — IDA y sus límites.
- Curso IDA · tutes 11 y 13 — plugins de IDA y el bug de la versión 6.8 (atención a la versión).

Síntesis de estudio de esta lección

Núcleo conceptual: el uso de OllyDbg en los videos 23–40, la VM Windows 7 y el recorrido de ExeInfoPE. **Conocimiento externo (agregado por esta edición):** el estado actual de las herramientas y todo el "taller actual 2026" (Frida, WinDbg/TTD, BinDiff/Diaphora, FLOSS, CAPA, angr/Unicorn, asistencia con IA). **Recomendación editorial:** el taller mínimo, la tabla de decisión y la cautela con la IA.

Ofuscación nativa: leer lo que no quiere ser leído

> **Objetivo de la lección**
>
> Reconocer las señales de que un binario nativo está **ofuscado** y tener un método para llegar al código real, sin romperlo. **Prerrequisitos:** PE y secciones (1.4), assembly y flujo (2.3–2.4), IDA y debugger (2.6–2.7).

Esto es **reconocimiento defensivo**: entender cómo se ve el código ofuscado para poder analizarlo. No se enseña a ofuscar ni a desempaquetar protectores comerciales.

Qué es la ofuscación nativa

Ofuscar es hacer que el programa **siga funcionando igual** pero sea difícil de leer para una persona y confuso para el desensamblador. A diferencia de un packer, el código puede estar a la vista: lo que cambia es la *forma*. Las técnicas típicas (**conocimiento externo**):

- **Secciones reordenadas o con nombres falsos**, y el *entry point* lejos de lo esperado.
- **Basura entre bloques**: bytes que no se ejecutan pero que el desensamblador intenta interpretar como instrucciones.
- **Saltos al medio de una instrucción**: un `jmp` / `jnz` cae a un byte que no es el inicio real, así el listado lineal muestra una instrucción que *no es* la que se ejecuta.
- **Código muerto y opaco**: `test ebp, ebp` seguido de `jnz` cuando `ebp` nunca es cero — un salto que *siempre* se toma, disfrazado de condicional.
- **Strings ocultas**: con un `00` adelante, armadas en el stack byte a byte, o cifradas y descifradas en runtime.

Enfoque clásico en desensambladores

El capítulo 26 analiza el ejercicio «choteado» y muestra el síntoma clásico: al abrirlo, **IDA lo muestra "todo roto"** y al forzarlo a función queda "todo feo"; en x64dbg tampoco se ve claro. Al mirar el *memory map* aparece la causa: **las secciones están cambiadas** —la primera sección está vacía y el código ejecutable recién está en la segunda—, y eso **engaña al desensamblador**. Con los símbolos cargados se empieza a navegar; sin ellos, la dificultad es la misma pero hay que apoyarse en la traza (`Reversing desde cero, cap. 26`).

El capítulo 30 resuelve la segunda parte y agrega un paso de preparación muy útil para analizar: cargar **todas las secciones** en IDA y, para trabajar con direcciones estables, **poner a cero el** `DLL characteristics` (el bit de `DYNAMICBASE`/ASLR) con *Patch Program → Change byte* y *Apply patches to input file*, de modo que el módulo cargue siempre en su base (aquí `0x400000`) (`Reversing desde cero, cap. 30`). Es un parche **sobre el crackme didáctico** del ejercicio (hecho para practicar), para poder estudiarlo con direcciones estables; no sobre software ajeno.

El método: encontrar el bloque real

1. **Sospechá temprano.** Si IDA no arma funciones, si hay flechas que cruzan todo o si el *memory map* no coincide con lo que esperás, asumí ofuscación.
2. **Mirá las secciones primero** (memory map / segments): dónde está de verdad el código ejecutable y cuál es la base.
3. **Dejá las direcciones quietas**: en tu laboratorio, desactivá ASLR del ejecutable propio para que las direcciones no cambien entre corridas.
4. **Seguí la ejecución, no el listado.** Ante un salto sospechoso, poné un breakpoint y mirá a dónde salta *de verdad*; el desensamblado lineal puede estar mostrando bytes que no se ejecutan.
5. **No NOPees a ciegas.** Reemplazar bytes sin entender rompe el programa y borra la pista. Comentá y renombrá; parchá solo cuando sabés qué hace el byte.

Ejemplo paso a paso: un salto al medio de una instrucción

*(Ejemplo **ilustrativo**: bytes construidos para mostrar la técnica, no un volcado de un binario puntual.)*

Supongamos estos bytes a partir de `0x401000`:

```
401000:  B8 EB FE 90 90        ; así lo decodifica el desensamblador LINEAL
         mov eax, 0x9090FEEB

pero el código que viene antes hace:  jmp 0x401001  (salta UN byte adelante)

401001:  EB FE                 ; reinterpretado desde 0x401001
         jmp short 0x401001    ; EB = jmp corto, FE = −2  → salta a sí mismo
```

Qué pasó: el byte `B8` (opcode de `mov eax, imm32`) hizo que IDA se "comiera" los 4 bytes siguientes como el inmediato. Pero el flujo real nunca ejecuta el `B8`: entra en `0x401001`, donde `EB FE` es un `jmp` a sí mismo (lo que viste en 2.4: desplazamiento `FE` = −2 sobre el final de la instrucción). El listado lineal mostraba una instrucción que **no se ejecuta**; la real aparece solo si decodificás desde el offset correcto. Por eso se sigue la **ejecución** (poné un breakpoint en `0x401001`) y no el listado.

Cadenas ocultas: cómo reconocer la rutina que las descifra

En 2.7 viste por qué una cadena puede no aparecer en la ventana de strings. Cuando la causa es que está **cifrada**, el patrón en el código suele ser reconocible aunque el algoritmo cambie (**conocimiento externo**):

- un **bucle corto** que recorre un buffer con un índice o puntero que avanza de a uno (o de a dos, si es UTF-16);
- una **operación reversible** por byte: `xor` con una constante o con un byte de una clave que se repite (`índice mod largo`), o `add` / `sub` / `rol`;
- un **destino** que después se pasa a una API que espera texto (`CreateFileW`, `RegOpenKeyExW`, `MessageBoxW`, `LoadLibraryW`).

*(Ejemplo **ilustrativo**: descifrado con `xor` y clave de un byte, x64.)*

```
lea   rsi, [rip+datos_cifrados]   ; fuente en .rdata (bytes ilegibles)
lea   rdi, [rsp+40h]              ; destino en el stack
xor   ecx, ecx
bucle:
movzx eax, byte ptr [rsi+rcx]
xor   al, 5Ah                     ; clave de un byte
mov   [rdi+rcx], al
inc   rcx
cmp   rcx, 0Ch                    ; largo fijo: 12 bytes
jb    bucle
mov   byte ptr [rdi+rcx], 0       ; terminador
lea   rcx, [rsp+40h]
call  cs:LoadLibraryA             ; el texto descifrado es un nombre de DLL
```

Cómo se lee: el `xor al, 5Ah` dentro del bucle es la transformación; `0Ch` es el largo; el destino termina como argumento de `LoadLibraryA`, así que el texto es un nombre de módulo. Para recuperarlo hay dos caminos, y conviene hacer los dos: **dinámico**, con un breakpoint en la llamada a `LoadLibraryA` y leyendo `RCX`; **estático**, aplicando la misma operación a los 12 bytes con un script (en IDA, IDAPython; en Ghidra, un script de 2.16). Si coinciden, anotás la cadena en un comentario junto al bucle y renombrás el buffer.

Lo que **no** conviene: ejecutar el programa entero para "ver qué cadenas aparecen" si no sabés qué hace, ni copiar la rutina sin entender de dónde sale la clave (puede depender de un dato del entorno y dar otro resultado en otra máquina).

Aplanamiento del flujo de control

Otra técnica frecuente en ofuscadores de código nativo es el **aplanamiento del flujo de control** (*control-flow flattening*): los bloques básicos de una función se sacan de su orden natural y se ponen **todos al mismo nivel**, dentro de un bucle infinito con un `switch` que elige qué bloque ejecutar según una **variable de estado**. Cada bloque, al terminar, escribe en esa variable el número del bloque siguiente (**conocimiento externo**; el ofuscador de investigación Obfuscator-LLVM lo describe así).

Cómo se ve en el grafo de IDA o Ghidra:

- un **despachador** central (comparaciones o una tabla de saltos sobre la misma variable) al que vuelven todos los bloques;
- muchos bloques "hermanos" sin relación aparente entre sí, todos con una flecha de vuelta al despachador;
- cada bloque termina con un `mov [var_estado], constante` (o con un cálculo que da una constante) antes de volver.

El método para leerlo no es mirar el grafo, que ya no dice nada, sino **reconstruir la secuencia**: para cada bloque, anotá qué valor de estado lo selecciona y qué valor deja al salir. Con esa tabla (estado → bloque → estado siguiente) se recupera el orden real. Los saltos condicionales originales aparecen como bloques que dejan **uno de dos valores** según una comparación. Hacerlo a mano sirve para funciones chicas; para funciones grandes existen complementos que lo automatizan, fuera del alcance de esta lección.

Señal	Qué sospechar	Cómo confirmarlo
Todos los bloques vuelven a un mismo punto	Aplanamiento	Ubicar la variable que compara el despachador
`jnz` tras `test` con registro siempre no nulo	Predicado opaco	Seguir el valor del registro en el debugger
Bucle corto con `xor` sobre un buffer que va a una API	Cadena cifrada	Breakpoint en la API y leer el argumento
Ráfaga de `mov` con constantes imprimibles	Cadena armada en el stack	Leer las constantes como bytes en little-endian
El listado no arma funciones	Basura o saltos al medio	Seguir la ejecución, no el listado

Desactivar ASLR sin romper la cabecera (ni depender de *Manual Load*)

El campo `DllCharacteristics` (2 bytes, WORD) está en el offset **+0x46** del Optional Header (`e_lfanew + 0x5E`), igual en PE32 y PE32+. El bit de ASLR es `IMAGE_DLLCHARACTERISTICS_DYNAMIC_BASE = 0x0040` (en x64 suele venir con `HIGH_ENTROPY_VA = 0x0020`). **No pongas todo el WORD en cero**: pisarías `NXCOMPAT (0x0100)`, `GUARD_CF (0x4000)`, etc. Y ojo: **IDA no carga la cabecera PE** salvo que elijas *Manual Load*, así que ahí no vas a encontrar el campo. Hacelo en PE-bear/CFF Explorer (Optional Header → DllCharacteristics → desmarcá *Dynamic Base*) o con Python sobre una copia:

```
import pefile
pe = pefile.PE("copia_lab.exe")
pe.OPTIONAL_HEADER.DllCharacteristics &= ~0x0060   # limpia DYNAMIC_BASE (0x40) y
HIGH_ENTROPY_VA (0x20)
pe.write("copia_lab_noaslr.exe")
```

Práctica

En tu VM, sobre un **ejercicio propio o un crackme didáctico** (hay varios en el Banco de prácticas):

1. Abrilo en IDA y en x64dbg y anotá tres señales de que algo no se lee normal.
2. Mirá el *memory map*: ¿la sección de código es la primera? ¿La base es la esperada?
3. Encontrá un salto condicional que en realidad sea incondicional (p. ej. `test reg,reg` + `jnz` con el registro siempre distinto de cero) y explicá por qué.
4. Llegá al primer bloque "real" de lógica siguiendo la ejecución, no el listado.
5. En un programa **propio**, guardá un nombre de DLL cifrado con `xor` de un byte y descifralo antes de usarlo. Recuperalo de las dos formas (breakpoint en la API y script sobre los bytes) y compará.
6. Escribí a mano, en C, una función chica con un `if` y un bucle, y una segunda versión "aplanada" con `while (1) { switch (estado) { ... } }`. Compilá ambas y compará los grafos; armá la tabla estado → bloque → estado siguiente de la versión aplanada.

Preguntas de repaso

▼ **¿Por qué un listado lineal puede mostrar una instrucción que no es la que se ejecuta?**

> Porque un salto puede caer **al medio** de lo que el desensamblador tomó como una instrucción; a partir de ese offset los bytes se decodifican distinto. Lo que se ejecuta es la secuencia que sigue el flujo real, no la que el desensamblador alineó de arriba a abajo.

▼ **Viste `test ebp, ebp` seguido de `jnz`. ¿Por qué puede ser ofuscación?**

> Si en ese punto `ebp` nunca vale cero, `ZF` nunca se activa y el `jnz` **se toma siempre**: es un salto incondicional disfrazado de condicional, para que parezca que hay dos caminos cuando hay uno.

▼ **En el «choteado», ¿qué pista da el *memory map*?**

> Que las secciones están **reordenadas**: la primera está vacía y el código ejecutable está en la segunda. Eso explica por qué el desensamblador no arma bien las funciones.

▼ **¿Por qué el capítulo 30 pone a cero el `DLL characteristics`?**

> Para **desactivar ASLR** del ejecutable y que cargue siempre en la misma base (`0x400000`), así las direcciones son estables mientras se lo analiza. Es un parche sobre un binario propio de práctica.

▼ **¿Por qué "no NOPear a ciegas"?**

> Porque reemplazar bytes sin entenderlos rompe la lógica y borra la evidencia de qué hacía el código. Primero se entiende (comentando, trazando); recién después, si hace falta y es un binario propio, se parchea.

▼ **¿Qué tres elementos delatan una rutina que descifra una cadena?**

> Un bucle corto sobre un buffer, una operación reversible por byte (`xor`, `add`, `rol`) y un destino que después se pasa a una API que espera texto.

▼ **¿Por qué conviene recuperar una cadena cifrada de forma estática y dinámica?**

> Porque se controlan entre sí: si el script sobre los bytes y el argumento leído en el breakpoint coinciden, la lectura de la rutina es correcta; si no, la clave puede depender del entorno o la rutina tiene un paso que no viste.

▼ **En un grafo aplanado, ¿qué tabla reconstruís para recuperar el orden?**

> Estado → bloque → estado siguiente: qué valor de la variable selecciona cada bloque y qué valor deja al salir. Los bloques que dejan uno de dos valores según una comparación son los condicionales originales.

Puerta de salida

Podés: (1) nombrar tres señales de ofuscación en un binario; (2) explicar un salto al medio de una instrucción y un salto condicional que siempre se toma; (3) usar el *memory map* para ubicar el código real; (4) llegar al primer bloque de lógica siguiendo la ejecución; (5) reconocer una rutina de descifrado de cadenas y recuperar el texto de dos formas; (6) reconocer un despachador de flujo aplanado y reconstruir el orden de una función chica.

Lo que sigue en el Bloque 2E (y lo que queda fuera)

La desofuscación simbólica y las expresiones mixtas booleano-aritméticas (MBA) se desarrollan con `angr` y `Z3` en la lección **2.25 · Ejecución simbólica y concólica**; la ofuscación por máquina virtual (dispatcher, VIP, handlers y mini-VM propia) se estudia en **2.26 · Ofuscación por máquina virtual**, y el reconocimiento de protectores multicapa en **2.27**. Lo que sí queda fuera por el límite ético del curso es desempaquetar o desvirtualizar protectores comerciales sobre software de terceros.

Fuentes de esta lección

- `Reversing desde cero, cap. 26` — el «choteado» con símbolos: IDA/x64dbg confundidos, secciones invertidas en el memory map.
- `Reversing desde cero, cap. 30` — cargar todas las secciones y poner a cero `DLL characteristics` (ASLR) para direcciones estables.
- Curso IDA · tutes 14–18 — contexto de binarios que "no se leen de una".
- Conocimiento externo: Obfuscator-LLVM, Control Flow Flattening — descripción del aplanamiento con variable de estado y `switch`; FLOSS — tipos de cadenas que extrae (estáticas, armadas en el stack, decodificadas).

Síntesis de estudio de esta lección

Núcleo conceptual: el «choteado» de los capítulos 26 y 30, las secciones invertidas y el parche de `DLL characteristics`. **Conocimiento externo:** la taxonomía de técnicas (basura, saltos al medio, código opaco, strings ocultas), el patrón de las rutinas de descifrado y el aplanamiento del flujo de control. **Ejemplo ilustrativo:** los bytes del salto al medio y el bucle de descifrado con `xor`. **Recomendación editorial:** el método de cinco pasos, la regla de no NOPear a ciegas, la tabla de señales y la recuperación doble (estática y dinámica).

Ampliación editorial basada en fuentes externas — Criterio de triage estático: Aplanamiento de Flujo (CFF) vs. Máquina Virtual de bytecode (VM)

Aunque desofuscar una máquina virtual queda fuera del alcance del curso, en el inventario estático (Etapa 4) tanto CFF (Obfuscator-LLVM) como un intérprete de VM (Rolles, USENIX WOOT '09) producen un grafo con un gran bucle y un `switch` central *(Hecho confirmado)*. Esta tabla permite distinguirlos en pocos minutos:

Criterio observable	Aplanamiento del Flujo de Control (CFF)	Ofuscación por Máquina Virtual (VM de bytecode)
Contenido de los bloques del switch	Operaciones reales de la función original (llamadas a APIs, acceso a variables locales y comparaciones del negocio).	Micro-operaciones genéricas (*handlers* del CPU virtual: `vADD`, `vPUSH`, `vMOV`, `vNOR`) sin lógica de negocio directa.
Selección del siguiente bloque	Cada bloque asigna una constante fija a una variable local de estado (`mov [rbp-8], 0x3A1F...`) y vuelve al encabezado del `switch`.	Un registro físico actúa como puntero de bytecode (`VIP` / `VPC`), lee el siguiente opcode de una tabla de datos (`lodsb` / `movzx eax, byte ptr [rsi] ; inc rsi`) y salta vía tabla de handlers (`jmp [rbx+rax*8]`).
Reutilización entre funciones	Cada función aplanada tiene su propio bucle y sus propios bloques básicos.	Varias funciones protegidas empujan un offset de bytecode distinto y saltan al mismo intérprete (`VM_Entry`).

TLS: código que corre antes de `main`

> **Objetivo de la lección**
>
> Entender qué es el **Thread Local Storage (TLS)** y, sobre todo, los **TLS callbacks**: código que Windows ejecuta **antes del entry point**, un lugar clásico para esconder lógica. Saber encontrarlo en el PE y en el debugger. **Prerrequisitos:** PE y directorios (1.4), procesos e hilos (1.5), debugger (2.6).

Dos cosas distintas con el mismo nombre

El capítulo 38 es un apéndice teórico sobre TLS y aclara que en un mismo ejercicio (`EjemploTLS`) conviven **dos formas** de guardar datos por hilo (`Reversing desde cero, cap. 38`):

1. **Por API:** `TlsAlloc`, `TlsSetValue`, `TlsGetValue`, `TlsFree`. El programa pide un índice y guarda/lee un valor propio de cada hilo. Es explícito y se ve en los imports.
2. **Por variable `__declspec(thread)`:** el compilador reserva el dato en la sección `.tls` y lo resuelve con ayuda del loader. No hay llamadas visibles: el acceso aparece como lectura de una zona por hilo.

Y hay un **tercer mecanismo, aparte**: los **TLS callbacks**. No dependen de cómo declares las variables —`__declspec(thread)` usa `.tls` pero **no** crea callbacks por sí solo—; los callbacks se registran en el **directorio TLS del PE** y son lo que más importa para el reversing, porque corren antes de `main`.

El directorio TLS y los callbacks

El PE tiene un **directorio TLS** (`IMAGE_TLS_DIRECTORY`) entre sus *data directories*. Su campo clave para el análisis es `AddressOfCallBacks`: un puntero a un arreglo de funciones terminado en cero. Esas funciones son los **TLS callbacks**, y el loader de Windows **las ejecuta antes de que corra el entry point** (antes de `main`), en cada carga y en cada creación/destrucción de hilo.

La consecuencia para quien analiza (**conocimiento externo**, confirmado por el rol de TLS que explica el cap. 38): si ponés el breakpoint solo en el entry point, **te perdés todo lo que pasó en el callback**. Es el escondite preferido para anti-debug temprano o para inicializaciones que no querés que se vean.

Cómo encontrarlo

- **En el PE** (PE-bear, CFF Explorer o IDA): abrí el *data directory* **TLS**; si `AddressOfCallBacks` no es cero, seguilo y vas a ver la lista de callbacks.
- **En IDA:** aparecen como `TlsCallback_0`, `TlsCallback_1`… y suelen quedar fuera del flujo del `main`; búscalos por nombre o desde el directorio TLS.
- **En x64dbg:** activá *Options → Preferences → Events →* ***TLS Callbacks*** para que el debugger **pare en el primer callback**, antes del entry point. Si no, arrancás ya pasado el punto interesante.

Ejemplo paso a paso: leer el directorio TLS

*(Ejemplo **ilustrativo**: valores construidos; en un binario real cambian las direcciones.)*

En PE-bear, el *data directory* número 9 (TLS) apunta a la estructura, y adentro seguís `AddressOfCallBacks`:

```
DataDirectory[9] TLS → RVA 0x3000, size 0x18

IMAGE_TLS_DIRECTORY  (en .rdata)
  StartAddressOfRawData  0x00404000
  EndAddressOfRawData    0x00404008
  AddressOfIndex         0x00404100
  AddressOfCallBacks     0x00403020   ← seguí este puntero
  SizeOfZeroFill         0
  Characteristics        0

En 0x00403020 (arreglo de punteros, terminado en NULL):
  20 10 40 00   00 00 00 00
  → callback[0] = 0x00401020
  → callback[1] = NULL  (fin de la lista)
```

Vas a `0x00401020` y ahí está el `TlsCallback_0`: ese código corre **antes** del entry point. Si `AddressOfCallBacks` fuera `0`, no habría callbacks y bastaría el breakpoint en el entry point.

> **Tamaño del directorio según la arquitectura (x86 vs x64)**
>
> El ejemplo de arriba es de 32 bits. En **PE32**, `IMAGE_TLS_DIRECTORY32` ocupa **0x18 bytes** y `AddressOfCallBacks` está en el offset **+0x0C** (arreglo de DWORDs de 4B). En **PE32+**, `IMAGE_TLS_DIRECTORY64` ocupa **0x28 bytes** y `AddressOfCallBacks` se corre a **+0x18** (arreglo de QWORDs de 8B, terminado en 8 ceros). En ambos casos esos campos son **direcciones virtuales absolutas (VA)**, no RVAs: si hay ASLR, la tabla `.reloc` los ajusta.

Práctica

En tu VM, con un **programa propio**:

1. Compilá algo mínimo con un callback TLS. En MSVC, el código mínimo es:

```c
#include <windows.h>
#include <stdio.h>

volatile int orden = 0;   /* lo escribe el callback; lo lee main */

void NTAPI mi_cb(PVOID h, DWORD motivo, PVOID r) {
    /* OJO: nada de printf acá. El callback corre ANTES de que
       el CRT se inicialice, así que el runtime de C todavía no
       está listo. Solo tocamos una variable. */
    if (motivo == DLL_PROCESS_ATTACH) orden = 1;
}

/* Registrar el callback en .CRT$XLB. Con __declspec(allocate) el símbolo NO
   se elimina aunque compiles con /O1 /O2 /GL (a diferencia de 'const_seg +
   EXTERN_C const', que el linker puede descartar). Incluye variante MinGW. */
#ifdef _MSC_VER
  #ifdef _WIN64
    #pragma comment(linker, "/INCLUDE:_tls_used")    /* x64: un guion bajo */
  #else
    #pragma comment(linker, "/INCLUDE:__tls_used")   /* x86: dos guiones */
  #endif
  #pragma section(".CRT$XLB", long, read)
  __declspec(allocate(".CRT$XLB")) PIMAGE_TLS_CALLBACK p_cb = mi_cb;
#else  /* MinGW-w64 (GCC / Clang) */
  PIMAGE_TLS_CALLBACK p_cb __attribute__((used, section(".CRT$XLB"))) = mi_cb;
#endif

int main(void) {
    printf("orden al entrar a main = %d  (1 => el callback ya corrió)\n", orden);
    return 0;
}
```

2. Abrilo en PE-bear y confirmá que el directorio **TLS** no está vacío y que `AddressOfCallBacks` apunta a tu función.
3. En x64dbg, activá el breakpoint de TLS callbacks y verificá que **para antes de** `main`.
4. Confirmá el orden **sin** imprimir desde el callback: que el callback solo escriba `orden = 1` y que `main` imprima `orden`. Si al entrar a `main` vale 1, el callback corrió antes (imprimir desde el callback puede colgar: el CRT todavía no se inicializó).

Preguntas de repaso

▼ **¿Qué hace especial a un TLS callback frente a cualquier otra función?**

> Que el loader lo ejecuta **antes del entry point** (y en cada alta/baja de hilo), sin que el `main` lo llame. Corre "solo", por estar registrado en el directorio TLS.

▼ **¿Qué campo del directorio TLS mirás primero y por qué?**

> `AddressOfCallBacks`: si no es cero, hay callbacks. Apunta a un arreglo de punteros a función terminado en cero.

▼ **Si solo ponés un breakpoint en el entry point, ¿qué te perdés?**

> Todo lo que haya hecho el/los callbacks TLS, que ya corrieron. Por eso en x64dbg conviene habilitar el breakpoint de TLS callbacks.

▼ **¿En qué se diferencian las dos formas de TLS del `EjemploTLS`?**

> Una usa **APIs** (`TlsAlloc` / `TlsGetValue` / `TlsSetValue`), visible en imports; la otra usa `__declspec(thread)`, resuelta por el compilador y el loader vía la sección `.tls`, sin llamadas explícitas.

▼ **¿Por qué el TLS es un buen escondite para anti-debug?**

> Porque corre antes que el código "principal" y mucha gente empieza a mirar desde el entry point; un chequeo puesto en un callback se ejecuta y decide antes de que el analista llegue.

Puerta de salida

Podés: (1) distinguir TLS por API de `__declspec(thread)`; (2) ubicar el directorio TLS y `AddressOfCallBacks` en un PE; (3) hacer que x64dbg pare en el primer callback; (4) demostrar con tu propio binario que el callback corre antes de `main`.

Fuentes de esta lección

- `Reversing desde cero, cap. 38` — `EjemploTLS`: TLS por API (`TlsAlloc` / `TlsGetValue` / `TlsSetValue`) y por `__declspec(thread)`.

Síntesis de estudio de esta lección

Núcleo conceptual: las dos formas de TLS del `EjemploTLS` del capítulo 38. **Conocimiento externo:** la estructura `IMAGE_TLS_DIRECTORY`, `AddressOfCallBacks`, el orden de ejecución antes del entry point y el breakpoint de TLS en x64dbg. **Recomendación editorial:** el método de búsqueda y la práctica con binario propio.

MFC: llegar al handler cuando todo pasa por clases

> **Objetivo de la lección**
>
> Reconocer un ejecutable hecho con **MFC** y llegar al **handler** de un control (el botón de un diálogo) aunque el `WinProc` clásico no esté a la vista. **Prerrequisitos:** mensajes de Windows y debugger (2.6), estructuras y vtables (2.7).

Por qué MFC complica la lectura

En los ejercicios anteriores la ventana se hacía **con la API de Windows directa**: el programa llamaba a `RegisterClassA` (que registra la clase y apunta al `WinProc`) y luego a `CreateWindow`; desde el `WinProc` se llegaba fácil a la comparación con los números de mensaje (`WM_COMMAND`, etc.) y de ahí al código que maneja cada evento. Era sencillo.

MFC (Microsoft Foundation Classes) es un conjunto de clases con mucha herencia que envuelve esa misma API. Por debajo sigue pasando lo mismo, pero **por arriba no se ve el `WinProc`**: Visual Studio genera una clase de diálogo (`CDialog` / `CWnd`) y el ruteo de mensajes queda escondido en el framework. Por eso, abrir un binario MFC y buscar el `WinProc` a mano no alcanza (**observado**, capítulo 39: `Reversing desde cero, cap. 39`).

Señales de que es MFC

- Importa `mfc###u.dll` (p. ej. `mfc140u.dll`) y el runtime de VC — **si MFC está enlazado dinámico**. En **enlace estático** esa DLL no aparece: ahí te guiás por los símbolos (`CWnd`, `AfxWinInit`) y las strings.
- Aparecen `AfxWinMain` / `AfxWinInit` y nombres de clases MFC (`CWnd`, `CDialog`, `CWinApp`, `CString`).
- Las APIs de texto que te sirven como ancla siguen estando: `GetWindowTextW`, `SetWindowTextW`, `MessageBoxW`, `SendMessageW`.

El message map: dónde está el ruteo

MFC asocia cada mensaje/control a una función con un **message map**, una tabla que el compilador arma a partir de macros del estilo `BEGIN_MESSAGE_MAP` / `ON_BN_CLICKED` / `END_MESSAGE_MAP`. En el binario queda como un arreglo de entradas `AFX_MSGMAP_ENTRY`, y cada entrada tiene más que "ID y puntero": `nMessage` (el mensaje, p. ej. `WM_COMMAND`), `nCode` (código de notificación), `nID` y `nLastID` (rango de IDs de control), `nSig` (la firma de la función) y `pfn` (el puntero al handler). El handler de "se hizo clic en el botón 1" suele llamarse `OnBnClickedButton1` en el fuente.

Dos caminos para llegar, sin pelearte con la tabla (**recomendación editorial**, apoyada en el método de mensajes del cap. 28):

1. **Por string/API (el más rápido):** buscá la string de la ventana ("clave", "acceso", el mensaje de éxito) o poné un breakpoint en `GetWindowTextW` / `SendMessageW`; al dispararse, el *call stack* te deja dentro del handler del control.

2. **Por el message map:** localizá la tabla del diálogo (cerca del constructor de la clase) y seguí el puntero de la entrada cuyo ID coincide con el del botón.

Ejemplo paso a paso: del botón al handler

*(Ejemplo **ilustrativo**: direcciones y nombres construidos para mostrar la ruta.)*

Ponés un breakpoint en `GetWindowTextW`, tocás **Aceptar** en el diálogo y mirás el *call stack*:

```
USER32!GetWindowTextW
app!CWnd::GetWindowTextW(...)          ; wrapper de MFC
app!CMiDialog::OnBnClickedOk+0x3e      ← el handler (acá estás)
mfc140u!_AfxDispatchCmdMsg+0x52        ; el framework ruteó el mensaje
mfc140u!CWnd::OnCommand+0x...
```

Subís al frame del handler `OnBnClickedOk` y encontrás la decisión:

```
lea   rdx, [rsp+0x40]        ; buffer donde quedó el texto tipeado
lea   rcx, [rsi+8]           ; el control (this)
call  CWnd::GetWindowTextW
lea   rdx, aS3cr3t           ; "s3cr3t"  (la clave esperada)
lea   rcx, [rsp+0x40]
call  wcscmp                 ; la comparación que decide éxito/fracaso
test  eax, eax
jnz   mal                    ; != 0 → clave incorrecta
```

Sin el call stack habría que desenredar el message map; con el breakpoint en la API de texto caés **directo** en `OnBnClickedOk`.

Práctica

En tu VM:

1. Si tenés Visual Studio, creá una **app MFC basada en diálogo** con un botón y un `OnBnClickedButton1` que compare un texto; compilala. Si no, usá un **binario MFC de práctica**.
2. Confirmá que importa `mfc###u.dll` y ubicá `AfxWinMain`.
3. Poné un breakpoint en `GetWindowTextW` (o en `SendMessageW`), tocá el botón y, desde el *call stack*, llegá al handler.
4. Identificá en el handler la comparación que decide éxito/fracaso.

Preguntas de repaso

▼ **¿Por qué en un binario MFC no encontrás el `WinProc` como en los ejercicios con API directa?**

> Porque MFC **envuelve** la API: el ruteo de mensajes lo maneja el framework a través de clases y del message map, en vez de un `WinProc` propio que el programa registre con `RegisterClassA`.

▼ **¿Qué es el message map y qué guarda cada entrada?**

> Una tabla que mapea **mensaje/control → función**. Cada entrada tiene el ID del control, el código de notificación y el **puntero al handler**.

▼ **Nombrá dos señales de que un binario es MFC.**

> Importa `mfc###u.dll` (p. ej. `mfc140u.dll`) y aparecen `AfxWinInit` / `AfxWinMain` y clases como `CDialog` / `CWnd` / `CString`.

▼ **¿Cuál es la forma más rápida de llegar al handler de un botón?**

> Un breakpoint en `GetWindowTextW` / `SendMessageW` (o seguir la string del diálogo): al dispararse, el *call stack* te deja dentro del handler, sin recorrer la tabla a mano.

▼ **Por debajo, ¿MFC cambia lo que hace Windows con los mensajes?**

> No. Sigue siendo el mismo sistema de mensajes de Windows; MFC solo agrega una capa de clases por encima. Las APIs ancla (`GetWindowTextW`, `SendMessageW`) siguen ahí.

Puerta de salida

Podés: (1) reconocer un binario MFC por imports y símbolos; (2) explicar qué es el message map; (3) llegar al handler de un botón por breakpoint en una API de texto y el call stack; (4) señalar la comparación que decide el resultado.

Fuentes de esta lección

- `Reversing desde cero, cap. 39` — el ejercicio MFC «de CITO»: por qué el `WinProc` no está a la vista y qué cambia respecto de usar la API directa.
- `Reversing desde cero, cap. 28` — base: `RegisterClassA`, `WinProc`, `WM_COMMAND` y breakpoint sobre el mensaje.

Síntesis de estudio de esta lección

Núcleo conceptual: el contraste API-directa vs. MFC del capítulo 39 y el método de mensajes del 28. **Conocimiento externo:** los nombres del message map (`BEGIN_MESSAGE_MAP`, `ON_BN_CLICKED`, `OnBnClickedButton1`), las DLLs `mfc###u.dll` y `AfxWinInit`. **Recomendación editorial:** los dos caminos para llegar al handler y la práctica con binario propio.

Ampliación editorial basada en fuentes externas — Definición en C y offsets de AFX_MSGMAP y AFX_MSGMAP_ENTRY para análisis estático

Cuando un binario MFC está enlazado estáticamente o se analiza sin ejecutarlo en IDA (*Local Types*) o Ghidra (*Parse C Source*), las estructuras oficiales de `<afxwin.h>` (Microsoft Learn: TN006 Message Maps) tienen el siguiente layout en memoria *(Hecho confirmado)*:

```
struct AFX_MSGMAP_ENTRY {
    unsigned int nMessage;   // +0x00 (4B): Mensaje (p. ej. 0x0111 = WM_COMMAND)
    unsigned int nCode;      // +0x04 (4B): Código de notificación (0 = BN_CLICKED)
    unsigned int nID;        // +0x08 (4B): Control ID inicial
    unsigned int nLastID;    // +0x0C (4B): Control ID final (igual a nID si es único)
    uintptr_t    nSig;       // +0x10 (4B en x86 / 8B en x64): Firma AfxSig_* (0 =
AfxSig_end)
    void*        pfn;        // +0x14 en x86 (total 24B) / +0x18 en x64 (total 32B): Puntero
al handler
};
struct AFX_MSGMAP {
    const void* pfnGetBaseMap;                // +0x00: Puntero al mapa base
    const struct AFX_MSGMAP_ENTRY* lpEntries; // +0x04 (x86) / +0x08 (x64): Arreglo
terminado en {0,0,0,0,0,0}
};
```

Recomendación editorial: El método virtual `GetMessageMap()` en la vtable de cualquier clase derivada de `CWnd` devuelve la dirección de su `AFX_MSGMAP` en dos instrucciones (`lea rax, [rip + MiDialog_messageMap] ; ret`).

Huella de compilador: saber con qué se hizo

> **Objetivo de la lección**
>
> Antes de leer una sola instrucción, saber **con qué compilador/runtime** se armó el binario, porque eso cambia todo: dónde está la lógica, cómo se ven las llamadas, qué estructuras esperar. **Prerrequisitos:** PE e imports (1.4), IDA (2.7), MFC (2.12).

Por qué mirar esto primero

El mismo programa en C se ve **distinto** según el compilador, y un programa de Delphi o Visual Basic no se parece en nada a uno de C. Identificar la huella te ahorra horas: te dice si vas a encontrar un `main` clásico, un runtime que rutea todo, p-code en vez de nativo, o una maraña de clases. Es lo primero que hace un analista con experiencia (**recomendación editorial**).

Tabla de huellas

Conocimiento externo (pistas que se confirman en los imports, strings y secciones del PE):

Compilador / runtime	Pistas típicas
MSVC (Visual C++)	**Rich header** en la cabecera (ver 2.14); `__security_cookie`, `__scrt_common_main`, `__acrt_*`; ruta a un `.pdb`; runtime `vcruntime###.dll` / `ucrtbase.dll`
MinGW/GCC	`msvcrt.dll`, `__mingw_*`, `libgcc_s_*`, `___chkstk_ms`, sección `.eh_frame`; **sin** Rich header; strings de rutas tipo `/mingw/...`
Delphi	Strings `Sysutils`, `System`, `Classes`, `TForm`, `TObject`; símbolos con `@`; VMT (tabla de métodos) al inicio del objeto; secciones `.itext` / `.data` particulares
Visual Basic 6	Import de `MSVBVM60.DLL`; funciones `__vba*`; strings `Project1` / `Form1`; puede ser **p-code** (interpretado) o nativo
MFC (sobre MSVC)	`mfc###u.dll` (p. ej. `mfc140u.dll`), `AfxWinInit` / `AfxWinMain`, clases `CWnd` / `CDialog` / `CString`, message map (ver 2.12)
Go	Binario grande y enlazado estáticamente, con pocos imports; cadena `Go build ID`; nombres de paquetes y funciones como `main.main`, `runtime.` o `fmt.` visibles aunque no haya símbolos de depuración; sección o datos con la tabla `pclntab`
Rust	Rutas de código fuente en mensajes de pánico (`src/main.rs`, `library/core/src/...`); símbolos con mangling `_ZN...E` (antiguo) o `_R...` (v0); mensajes como «called `Option::unwrap()` on a `None` value»; con el target `-msvc` puede traer **Rich header** del linker de Microsoft

Regla práctica: **mirá primero los imports** (qué DLLs de runtime trae) y las **strings** (nombres de clases/módulos); con eso ya ubicás la familia.

Dos casos que cambian la estrategia

- **VB6 p-code vs. nativo.** Si es p-code, el código "real" no es x86: es bytecode que interpreta `MSVBVM60.DLL`. Leer el x86 directo no sirve; hay que usar herramientas específicas de VB. Reconocerlo temprano evita perder tiempo.
- **Delphi.** Las APIs no se llaman como en C: el objeto trae su **VMT** al inicio y los métodos se resuelven por tabla. Esperar un `main` con `CreateWindow` directo te confunde.

En recopilaciones dispersas esto suele figurar como material suelto, no como lección estructurada: en `Teorias_Numeradas` hay **decenas de títulos de Delphi** y **varios de Visual Basic/VB6, p-code y Java** (índice clasificado). Son conteos **por subcadena del título** (aproximados, sin abrir los contenedores): orientan para profundizar, no son prueba. El único runtime trabajado a fondo en el curso es **MSVC/C++** (y MFC en la 2.12, observado en el cap. 39: `Reversing desde cero, cap. 39`).

Lenguajes que no se leen como C: Go, Rust y Delphi

Los tres producen código nativo x86/x64, así que IDA y Ghidra los abren sin problema. Lo que cambia es **qué esperar** del listado. Todo este apartado es **conocimiento externo** al material de reversing; lo citado viene de la documentación de cada lenguaje.

Go. Un "hola mundo" en Go pesa varios megabytes porque trae enlazado el runtime completo (planificador de goroutines, recolector de basura). Para leerlo:

- **Los nombres de funciones sobreviven.** El runtime de Go necesita una tabla que relaciona direcciones con funciones y líneas (`pclntab`) para armar trazas de pila, perfilar y recolectar basura, así que está en el binario aunque se hayan quitado los símbolos de depuración. Herramientas e IDA/Ghidra recientes la usan para recuperar nombres como `main.main` o `main.verificarClave`. Empezá por las funciones del paquete `main`: el resto suele ser runtime y biblioteca estándar.
- **Las cadenas no terminan en `0`.** Un `string` de Go es puntero + longitud; el compilador guarda muchas cadenas pegadas en una sola zona y el buscador de strings las muestra como un bloque largo. Para saber dónde termina una, buscá la longitud que se carga junto al puntero.
- **Convención propia.** Go no usa la convención x64 de Windows de Llamadas y convenciones para sus funciones internas: tiene su propio ABI (en versiones recientes pasa argumentos en registros, en otro orden). Al comienzo de casi toda función vas a ver una **comprobación de pila** —una comparación del puntero de pila con un límite y un salto a `runtime.morestack`— que no es lógica del programa y se puede saltar con la vista.

Rust. También enlaza estáticamente su biblioteca estándar, pero suele usar el runtime de C de la plataforma.

- **Huella engañosa.** Con el target `x86_64-pc-windows-msvc` el binario lo enlaza el linker de Microsoft: puede tener Rich header e importar `vcruntime140.dll`. Ver esas pistas y concluir "C++ de MSVC" es el error típico; las rutas `.rs` y los mensajes de pánico lo desmienten.
- **Cadenas como puntero + longitud** (`&str`), igual que en Go: sin `0` final.
- **Nombres con mangling.** El esquema antiguo se parece al de C++ de Itanium (`_ZN...` con un hash al final) y el esquema **v0** empieza con `_R`; ambos se pueden demanglear con las herramientas de Rust.

- **Mucho código de comprobación.** Chequeos de límites de arreglos y desbordes que terminan en una función de pánico. Las ramas que van al pánico son el camino de error; la lógica está en el otro.

Delphi (amplía la fila de la tabla).

- **Formularios como recursos.** Los formularios (`TForm`) se guardan como recursos `RCDATA` que empiezan con la firma `TPF0`; ahí están los nombres de los controles y los **nombres de los manejadores de eventos** (por ejemplo `Button1Click`), que te llevan directo a la lógica.
- **Convención `register`.** En Delphi de 32 bits la convención por defecto pasa los tres primeros argumentos en `EAX`, `EDX` y `ECX`, no en el stack: un método recibe `Self` en `EAX`. Si leés el listado esperando `stdcall`, los argumentos "desaparecen".
- **Cadenas con cabecera.** Los `string` largos de Delphi guardan contador de referencias y longitud **antes** del primer carácter, así que el puntero apunta al texto pero hay datos en offsets negativos.
- **RTTI y VMT.** Los nombres de clases y la tabla de métodos virtuales permiten reconstruir jerarquías; existen herramientas específicas para Delphi que lo automatizan.

> **Pistas, no pruebas**
>
> Una huella es un **indicio**, no un certificado. La **ausencia de Rich header** sugiere que no es MSVC, pero no **demuestra** MinGW (podría ser otro linker o estar removido). Y una app **MFC enlazada estáticamente** puede **no** importar `mfc###u.dll`. Confirmá siempre con varias señales juntas.

Ejemplo paso a paso: el mismo `hola.c`, dos compiladores

*(Ejemplo **ilustrativo**: imports representativos; varían según versión.)*

El mismo programa compilado en MSVC y en MinGW deja huellas distintas:

```
MSVC (cl hola.c)                      MinGW (gcc hola.c)
────────────────────────              ──────────────────────────
Rich header: presente                 Rich header: AUSENTE
Imports:                              Imports:
  VCRUNTIME140.dll                      msvcrt.dll
    __C_specific_handler                  __getmainargs, __set_app_type
    memcpy, __current_exception           _onexit, __p__fmode
  api-ms-win-crt-stdio-l1-1-0.dll       KERNEL32.dll
    __stdio_common_vfprintf           Secciones: .text .data .rdata
  KERNEL32.dll                                   .eh_frame  ← GCC
Secciones: .text .rdata .data         Símbolos: __mingw_*, ___chkstk_ms
  (sin .eh_frame)
```

Con solo mirar imports y secciones ya decidís: Rich header + `api-ms-win-crt-*` + `__C_specific_handler` ⇒ **MSVC**; `msvcrt.dll` + `__mingw_*` + `.eh_frame` y sin Rich header ⇒ **MinGW** (recordá la salvedad de arriba: es indicio, no prueba).

Ampliación editorial basada en fuentes externas — Reversing de ejecutables empaquetados de Python (`PyInstaller`) y aplicaciones Electron (`ASAR`)

Un error frecuente de triage ocurre cuando el analista abre en IDA o Ghidra un `.exe` de varios megabytes y empieza a desensamblar funciones en C como `pyi_main` , `pyi_arch_open_archive` o llamadas a `uv_run` , creyendo que está viendo la lógica del programa, cuando en realidad está frente al *bootloader* nativo de un lenguaje interpretado (**Hecho confirmado**):

1. **Ejecutables de Python empaquetados con `PyInstaller` :**

PyInstaller empaqueta el intérprete (`python3XX.dll`), las extensiones `.pyd` y los scripts compilados a bytecode (`.pyc`) dentro de un archivo `CArchive` adjunto al final del PE (en el *overlay* o como recurso).

- **Huella estática inequívoca:**
- Strings en `.rdata` : `"PyInstaller: FormatMessageW failed."` , `"_MEIPASS2"` , `"python311.dll"` (o `310` , `312` , `313`).
- Magic de la estructura `COOKIE` buscando desde el final del archivo: los 8 bytes `4D 45 49 0C 0B 0A 0B 0E` (`MEI\x0c\x0b\x0a\x0b\x0e`).
- **Flujo de extracción y decompilación en laboratorio (sin ejecutar el binario):**

1. **Extraer el contenedor:** Ejecutar `pyinstxtractor.py` sobre una copia del `.exe` :

```
python pyinstxtractor.py muestra_pyinstaller.exe
```

Esto crea la carpeta `muestra_pyinstaller.exe_extracted/` , indica la versión exacta de Python (p. ej. `Python 3.11`) y señala los posibles puntos de entrada (`Possible entry points: main.pyc`).

2. **Estructura del encabezado `.pyc` (Python 3.7 a 3.13+):** Todo archivo `.pyc` válido en Python 3.7+ tiene una cabecera de **16 bytes** antes del objeto `code` serializado con `marshal` :

- `+0x00` (4B): *Magic number* de la versión de Python (p. ej. `A7 0D 0D 0A` en Python 3.11; `CB 0D 0D 0A` en 3.12).
- `+0x04` (4B): Bit field (`0x00000000` para timestamp).
- `+0x08` (4B): Timestamp o hash parcial.
- `+0x0C` (4B): Tamaño del archivo fuente `.py` original.

(Nota: versiones recientes de `pyinstxtractor.py` ya reconstruyen estos 16 bytes automáticamente al extraer).

3. **Decompilar o desensamblar el bytecode:** Con `pycdc` (*Decompyle++*) se recupera código fuente `.py` ; si el bytecode es de Python 3.11/3.12/3.13 y el decompilador falla en algún opcode nuevo, el módulo estándar `dis` + `marshal` corrido con **esa misma versión de Python** muestra el desensamblado exacto e infalible:

```
# volcar_pyc.py — ejecutar con la MISMA versión menor de Python que informó pyinstxtractor
import dis, marshal

with open("main.pyc", "rb") as f:
f.seek(16)  # salta los 16 bytes del encabezado .pyc (Python 3.7+)
code_obj = marshal.load(f)
dis.dis(code_obj)
```

2. **Aplicaciones empaquetadas con Electron / Node.js (`app.asar`):**

- **Huella:** Binario PE enorme (> 100 MB) que importa `ffmpeg.dll` , `d3dcompiler_47.dll` , contiene strings de `Chromium` / `Node.js` y va acompañado de una carpeta `resources/` con un archivo `app.asar` (magic header con longitud JSON prefijada).
- **Extracción estática:** La lógica no está en el `.exe` , sino en el JavaScript dentro de `resources/app.asar` . Se extrae en modo lectura con `npx @electron/asar extract resources/app.asar ./codigo_extraido` .

Práctica

En tu VM, con **programas propios**:

1. Compilá el **mismo** `hola.c` con **MSVC** (`cl`) y con **MinGW** (`gcc`). Abrí ambos en PE-bear/IDA.
2. Armá una tabla comparando: DLLs de runtime en imports, presencia de Rich header, strings de rutas, secciones (`.eh_frame` vs. no).
3. Si tenés cómo, compilá algo mínimo en **Delphi** o abrí un binario VB6 de práctica y buscá `MSVBVM60.DLL` o `TForm` / `TObject` .
4. Escribí en una línea, para cada binario, "esto es X porque veo Y".
5. Si tenés Go o Rust instalados, compilá un programa propio mínimo que compare una cadena ingresada con una fija. En el de Go, ubicá `main.main` y la comprobación de pila del inicio; en el de Rust, buscá las rutas `.rs` y mirá si hay Rich header. En ambos, encontrá la cadena fija y anotá dónde está su longitud.

Dónde profundizar cada lenguaje (Bloque 2E)

Esta lección enseña a identificar el compilador en el triage inicial. Para reconstruir tipos e interfaces (`itab` , `pclntab`) en Go y *trait objects* (`dyn Trait`), monomorfización y mangling `v0` en Rust, seguí a la lección **2.32 · Go y Rust en profundidad**. Para analizar `VBHeader` , llamadas `__vba*` y bytecode *p-code* en Visual Basic 6, consultá la lección **2.33 · VB6 y p-code**.

Preguntas de repaso

▼ **¿Por qué conviene identificar el compilador antes de leer el código?**

> Porque determina **dónde está la lógica y cómo se ve**: un binario Delphi, uno VB6 p-code y uno MSVC se analizan de formas distintas. Saberlo evita buscar un `main` clásico donde no lo hay.

▼ **¿Qué pista distingue MSVC de MinGW de un vistazo?**

> MSVC trae **Rich header** y runtime `vcruntime` / `ucrtbase` con `__security_cookie`; MinGW usa `msvcrt.dll`, símbolos `__mingw_*`, `.eh_frame` y **no** tiene Rich header.

▼ **Ves import de `MSVBVM60.DLL`. ¿Qué implica y qué riesgo hay?**

> Es **Visual Basic 6**. Riesgo: puede ser **p-code** (bytecode interpretado por esa DLL), así que leer el x86 directo no te muestra la lógica real; necesitás herramientas de VB.

▼ **¿Cómo reconocés Delphi?**

> Strings como `Sysutils` / `System` / `Classes` / `TForm` / `TObject`, símbolos con `@` y la **VMT** al inicio de los objetos; las llamadas no se ven como en C.

▼ **Un binario tiene Rich header e importa `vcruntime140.dll`, pero aparecen rutas `src/main.rs`. ¿Qué es?**

> Probablemente **Rust** compilado con el target `-msvc`: el linker de Microsoft deja el Rich header y el runtime de C, pero las rutas `.rs` y los mensajes de pánico delatan el lenguaje. Es el ejemplo de por qué una sola pista no alcanza.

▼ **¿Por qué en un binario de Go sin símbolos podés ver nombres como `main.main`?**

> Porque el runtime necesita la tabla `pclntab` (direcciones → funciones y líneas) para trazas de pila, perfilado y recolección de basura, y quitar los símbolos de depuración no la elimina.

▼ **En Delphi de 32 bits, ¿dónde llegan los primeros argumentos de un método?**

> En `EAX`, `EDX` y `ECX` (convención `register`); `Self` viene en `EAX`. Leerlo como `stdcall` hace que los argumentos parezcan faltar.

▼ **¿Qué parte de esto corresponde a patrones clásicos y qué a compiladores modernos?**

> MFC está observado (cap. 39) y hay **títulos** de Delphi/VB/Java en `Teorias_Numeradas` (no leídos). La tabla de huellas de MSVC/MinGW/Delphi/VB6/Go/Rust y el apartado de lenguajes no-C son **conocimiento externo**.

Puerta de salida

Podés: (1) decir con qué compilador/runtime se hizo un binario y por qué; (2) distinguir MSVC de MinGW por imports/Rich header/secciones; (3) reconocer VB6 (y el riesgo de p-code) y Delphi; (4) justificar cada conclusión con una pista concreta del PE; (5) reconocer un binario de Go o de Rust y explicar qué cambia al leerlo (nombres recuperables, cadenas con longitud, comprobaciones del runtime).

Fuentes de esta lección

- `Reversing desde cero, cap. 39` — MFC como capa sobre la API (huella de framework).
- Teorias_Numeradas · clasificación — títulos de Delphi, Visual Basic/VB6, p-code y Java (conteo aproximado por subcadena del título); material para profundizar, no leído.
- Go (conocimiento externo): paquete debug/gosym y diseño de la tabla de símbolos de Go 1.2 — la tabla `pclntab` y por qué la necesita el runtime; A Quick Guide to Go's Assembler.
- Rust (conocimiento externo): Symbol Mangling y esquema v0; tipo primitivo str.
- Delphi (conocimiento externo verificado en Embarcadero RAD Studio DocWiki): Program Control (convención register), Internal Data Formats (cabecera StrRec en offsets negativos) y Using Resource Files (recursos DFM/TPF0).

Síntesis de estudio de esta lección

Núcleo conceptual: la huella de MFC del capítulo 39 y los conteos por título de `Teorias_Numeradas`. **Conocimiento externo:** la tabla de huellas (Rich header, `MSVBVM60.DLL`, VMT de Delphi, `.eh_frame` de MinGW, Go y Rust), el caso p-code y las particularidades de lectura de Go, Rust y Delphi. **Recomendación editorial:** mirar imports y strings primero y la práctica comparativa MSVC/MinGW.

Ampliación editorial basada en fuentes externas — Metodología de reversing para binarios en Go (pclntab, strings y ABI) y Rust (mangling v0 y enums)

Dada la proliferación de binarios compilados en **Go** y **Rust** en software de sistema, herramientas defensivas y amenazas modernas, su análisis requiere técnicas específicas que difieren sustancialmente del código compilado en C/C++ clásico:

1. Reversing de Binarios en Go

- **Recuperación de símbolos mediante la tabla `pclntab`:** El compilador de Go (`gc`) enlaza una tabla en la sección `.gopclntab` o `.rodata` denominada *Program Counter Line Table* (Go Runtime: symtab.go). Esta estructura asocia cada dirección virtual con el nombre de su función, archivo fuente `.go` y número de línea. En binarios sin símbolos (*stripped*), `pclntab` **permanece intacta** porque el recolector de basura, el planificador y `runtime.panic` la necesitan para generar trazas de pila.
  Herramientas de recuperación: Plugins como `go_parser` (IDA Pro), el analizador integrado *Go Module Analyzer* (Ghidra 11+) o la utilidad `GoReSym` leen el número mágico al inicio de la tabla (`0xFFFFFFFB` en Go 1.2, `0xFFFFFFFA` en Go 1.16, `0xFFFFFFF0` en Go 1.18+ y `0xFFFFFFF1` en Go 1.20+) y renuevan en pocos segundos los nombres de las 2.000+ funciones (revelando de inmediato funciones críticas como `main.main` o `main.verificarLicencia`).
- **Estructura interna de cadenas (`string`):** Un `string` en Go es un par inmutable `struct { const char* str; ptrdiff_t len; }` (16 bytes en x64). No contiene terminador nulo `0x00` . En desensamblado x64 se reconoce el patrón invariable de carga del puntero seguido de su tamaño en un registro secundario:

```
lea rax, [rip + 0x2A150]   ; Puntero al inicio del buffer en .rodata
mov rbx, 0x18              ; Longitud exacta: 24 bytes (sin terminador nulo)
```

- **Convención de llamadas por registros (Go 1.17+ ABI):** A partir de Go 1.17, las funciones internas pasan argumentos y retornos en 9 registros enteros ordenados: `RAX` , `RBX` , `RCX` , `RDI` , `RSI` , `R8` , `R9` , `R10` , `R11` , pasando a la pila solo los excedentes.
- **Prólogo de comprobación de pila:** En el inicio de casi toda función Go aparece una comprobación con el puntero de la goroutine activa (almacenado en `R14` en x64):

```
cmp rsp, qword ptr [r14 + 0x10]   ; Comprueba límite de pila de la goroutine
jbe runtime.morestack             ; Aloca un segmento de pila nuevo si falta espacio
```

2. Reversing de Binarios en Rust

- **Name Mangling v0 (RFC 2603):** Desde Rust 1.59+, los símbolos usan el esquema estándar iniciado con el prefijo `_R` (Rust RFC 2603). Nombres crudos como `_RNvNtCs1234_7mi_app4auth14verificar_pass` se decodifican de inmediato con la herramienta oficial `rustfilt` a `mi_app::auth::verificar_pass` .
- **Tipos algebraicos `Result<T, E>` y `Option<T>` :** En memoria se compilan como estructuras con un byte discriminante (*tag*):

```
struct Option_u32 {
    uint8_t  tag;   // 0 = None, 1 = Some
    uint32_t valor; // Válido únicamente si tag == 1
};
```

En ensamblador se observa la comprobación del byte discriminante previa a operar con el dato (`cmp byte ptr [rsp+0x18], 1 ; jne fallo`).

- **Manejo de pánicos y cadenas de error:** Cuando una función aborta, Rust invoca a `core::panicking::panic_fmt` pasando una estructura que contiene el nombre del archivo fuente `.rs` y la línea exacta como cadenas ASCII legibles en `.rdata`.

Ampliación editorial basada en fuentes externas — Optimización de nicho (Niche Optimization) en Rust para tipos Option sin byte tag

Al revertir binarios compilados en Rust, es fundamental conocer la **optimización de nicho** (*Niche Optimization* o *Null-Pointer Optimization*) implementada por el compilador (The Rustonomicon: Alternative Representations):

- **El principio del nicho:** En tipos como `Option<u32>`, Rust debe reservar un byte extra como discriminante (*tag*) porque cualquier valor de 32 bits es un número válido. Sin embargo, para tipos que **garantizan que nunca pueden ser nulos** (como referencias `&T`, punteros inteligentes `Box<T>`, punteros a función `fn()` o enteros no nulos `core::num::NonZeroU32`), el compilador **elimina por completo el byte `tag`**.
- **Representación en memoria:**
  - `Some(&T)`: Se representa simplemente por la dirección de memoria válida del puntero (8 bytes en x64).
  - `None`: Se representa con el valor nulo exacto (`0x0000000000000000`).
- **Patrón en desensamblado:** Ante estos tipos, el analista **no debe buscar** un `cmp byte ptr [rsp+offset], 1`. En su lugar, el chequeo de `None` se compila de forma idéntica a una comprobación de puntero nulo en C:

```
test rax, rax     ; Comprueba si el Option<&T> o Option<Box<T>> es nulo
jz es_none        ; Si es cero, equivale a la rama None
; ... continúa en la rama Some(&T) usando RAX directamente como puntero
```

Ampliación editorial basada en fuentes externas — ABI register, cabecera StrRec en offsets negativos y firma TPF0 en Delphi

- **Convención `register` en 32 bits vs. x64 (Embarcadero DocWiki: Program Control):** En Delphi x86 (`DCC32`), los tres primeros argumentos de hasta 32 bits van en `EAX` (1.º, o `Self` en métodos), `EDX` (2.º) y `ECX` (3.º); los restantes se apilan de izquierda a derecha y la función llamada limpia el stack (`ret N`). En Delphi x64 (`DCC64`), el compilador usa la ABI estándar de Microsoft (`RCX` , `RDX` , `R8` , `R9`) *(Hecho confirmado)*.
- **Layout en memoria de cadenas largas (Embarcadero DocWiki: Internal Data Formats):** El puntero de cadena (Delphi 2009+) apunta al primer carácter (`offset 0`), precedido por una cabecera de 12 bytes en desplazamientos negativos: `[ptr - 12]` = `codePage` (2 B), `[ptr - 10]` = `elemSize` (2 B), `[ptr - 8]` = `refCnt` (4 B; `-1` en literales constantes) y `[ptr - 4]` = `length` en caracteres (4 B). Por eso `Length(s)` se traduce como `mov eax, [eax - 4]` *(Hecho confirmado)*.
- **Formularios en `.rsrc` (`TPF0`) (Embarcadero DocWiki: Using Resource Files):** Los formularios VCL/FMX compilados en recursos `RT_RCDATA` comienzan con los 4 bytes ASCII `54 50 46 30` (`"TPF0"`) generados por `TWriter` y asocian cada control con el nombre de su método de evento publicado *(Hecho confirmado)*.

PE fino: los directorios que cuentan la historia

> **Objetivo de la lección**
>
> Leer los **directorios del PE** que la lección 1.4 solo nombra, y entender qué pista da cada uno **antes** de llegar al entry point. **Prerrequisitos:** El formato PE (1.4), TLS (2.11), huella de compilador (2.13).

En 1.4 viste la cabecera, las secciones, los imports y la IAT. Acá vamos a los **otros** `DataDirectory`, que muchas veces resuelven el análisis sin ejecutar nada. Todo esto se lee con **PE-bear**, **CFF Explorer** o el navegador de IDA; el recorrido VA↔RVA↔offset es el del capítulo 05 (`Reversing desde cero, cap. 05`). El contenido de esta lección es **conocimiento externo** salvo donde se cite.

Rich header

Entre el `DOS stub` y la cabecera PE, los linkers de Microsoft dejan el **Rich header**: una lista cifrada (XOR con una clave que está ahí mismo) de qué herramientas y versiones tocaron el binario (`@comp.id`). Qué te dice:

- Es casi una **firma de MSVC**: si está, se armó con el toolchain de Microsoft; su **ausencia sugiere** otro compilador (MinGW u otro) o que lo removieron, pero **no lo prueba** (ver 2.13).
- Las versiones ayudan a **datar** el binario y a agrupar muestras de la misma máquina de compilación.

Recursos (`.rsrc`)

El **directorio de recursos** guarda íconos, menús, *dialogs*, *string tables*, *version info* y, a veces, **blobs** (otro ejecutable, una config, una clave). Qué mirar:

- *Version info*: nombre de producto, compañía, versión. Pistas de origen (y, en malware, mentiras).
- *Dialogs* y *string tables*: los textos de la interfaz, que te llevan a la lógica (como las strings del `.rdata`).
- Un recurso **grande o de tipo raro** (RCDATA) suele ser un payload embebido: a mirar con cuidado.

Imports diferidos (*delay-load*)

Además de la tabla de imports normal, el PE puede tener un directorio de **delay-load**: DLLs que **no** se cargan al inicio sino la **primera vez que se usan**. Consecuencia para el análisis: una dependencia puede **no aparecer** en la lista de imports "a primera vista" y cargarse recién en runtime vía `__delayLoadHelper2`. Si buscás una API y no está en los imports normales, revisá delay-load.

Excepciones y unwind (`.pdata`)

En x64, el manejo de excepciones **no** vive en el stack como el SEH de 32 bits: está en tablas `.pdata` / `.xdata` (ver la rama de vulnerabilidades, cap. 36). Para el análisis, esas tablas sirven para:

- **Delimitar funciones** aunque no haya símbolos (cada entrada cubre un rango de código).
- Entender el prólogo/epílogo (qué registros se salvan), útil al reconstruir marcos.

Debug directory y PDB

El **directorio de debug** puede traer la **ruta del** `.pdb` (p. ej. `D:\proyectos\cosa\Release\cosa.pdb`) y un GUID/age para emparejar símbolos. Qué te da:

- La **ruta** filtra información: nombre de proyecto, usuario, estructura de carpetas del que lo compiló.
- El GUID/age permite **cargar los símbolos** correctos (si los tenés o están en un symbol server), lo que vuelve legible un binario (como el caso con `.pdb` de la serie de estructuras del Curso IDA).

Load Config

El `Load Config` **directory** declara mitigaciones y metadatos que el loader usa:

- Puntero al `security cookie` (el `/GS` que viste en vulnerabilidades).
- Tabla **SEH segura** (SafeSEH) en 32 bits.
- Tablas de **CFG** (`GuardCFFunctionTable`): los destinos válidos de llamadas indirectas.

Leerlo te dice **qué defensas tiene** el binario sin ejecutarlo, y complementa la tabla de mitigaciones de la rama de vulnerabilidades.

Cuando los imports no cuentan todo: resolución dinámica de APIs

La tabla de imports solo lista lo que el programa enlazó **al compilar**. Un programa puede obtener la dirección de cualquier función **en ejecución**, y entonces esa API no aparece en los imports. Es una de las primeras cosas que se encuentran al analizar binarios reales, y hay tres formas típicas, de más visible a menos visible (**conocimiento externo** salvo lo citado):

1. `LoadLibrary` + `GetProcAddress`**.** El programa carga la DLL y pide la dirección por nombre. Señales: los dos imports, cadenas con nombres de API (`"VirtualAlloc"`, `"MessageBoxW"`) y, después, una llamada **indirecta** a través de un registro o una variable (`call rax`, `call [rbp-18h]`). La propia documentación de Microsoft usa este mecanismo para funciones sin biblioteca de importación, como las de `ntdll`.
2. **Recorrer el PEB y la tabla de exports.** Sin llamar a ninguna API: el código lee el PEB (`gs:[60h]` en x64, `fs:[30h]` en x86), sigue `Ldr` hasta la lista `InMemoryOrderModuleList` —una lista doblemente enlazada de entradas con la base y el nombre de cada módulo cargado— y, en el módulo elegido, recorre el **directorio de exports** del PE (nombres, ordinales y direcciones) hasta encontrar la función. Señales: accesos a esos offsets fijos, un bucle que compara nombres de módulo y otro que recorre `AddressOfNames`.
3. **Hashing de nombres de API.** Igual que el anterior, pero en vez de comparar el nombre con una cadena, calcula un **hash** de cada nombre exportado y lo compara con una **constante**. No queda ninguna cadena con el nombre de la API en el binario. Señales: un bucle corto que recorre los bytes de un nombre con rotaciones, `xor` o sumas, seguido de un `cmp` contra un valor de 32 bits. Como la misma función de hash produce la misma constante para el mismo nombre, esas constantes se pueden buscar en bases públicas de hashes conocidos para nombrar la API.

Qué registrar en el análisis: qué APIs resuelve dinámicamente (con evidencia: cadena, constante o traza), **dónde** guarda las direcciones (una tabla propia funciona como una IAT paralela) y desde dónde se llaman. En dinámico, un breakpoint en `GetProcAddress` o en el retorno del resolvedor te da la lista real.

Un detalle de lectura relacionado: en binarios modernos vas a ver imports de nombres como `api-ms-win-core-…`. Son **API sets**: nombres de **contratos**, no archivos; el cargador los resuelve contra la DLL que implementa cada función en esa versión de Windows, y puede no existir ningún archivo con ese nombre.

Firma Authenticode: integridad y procedencia

Un PE puede incluir una firma **Authenticode** en la tabla de certificados del directorio de datos; también puede tener firma de catálogo. La firma permite comprobar integridad según las reglas de hash de PE y validar la cadena del certificado, pero **no prueba que el programa sea benigno** ni que su editor sea confiable para cualquier uso. Una firma ausente tampoco demuestra intención maliciosa. La entrada de la tabla de certificados es una excepción importante al modelo general de directorios: su ubicación se expresa como **offset de archivo**, no como RVA.

Al revisar un binario, registrá el resultado de verificación, el sujeto del certificado, la cadena y el sello de tiempo cuando estén disponibles. Interpretá todo como evidencia de identidad e integridad, no como veredicto de seguridad.

Resolución efectiva de DLL

La IAT dice qué módulos y funciones necesita el binario; **no siempre revela qué archivo concreto se cargó**. Si el nombre no especifica ruta, la ubicación efectiva depende del orden de búsqueda de Windows y del contexto: directorio de la aplicación, módulos ya cargados, Known DLLs, API sets, redirecciones y configuración del proceso. Por eso, al analizar un problema de carga, distinguí el nombre importado de la ruta observada en ejecución. En programas propios, usar rutas explícitas y las opciones seguras del cargador reduce ambigüedades; en reversing, documentá la ruta efectiva y el contexto, sin inferirla solo del nombre de la DLL.

Aplicaciones empaquetadas, manifiestos y búsqueda segura

No hay un único orden que puedas aplicar sin identificar el tipo de aplicación y la forma de carga. En apps de escritorio sin paquete, las redirecciones side-by-side declaradas por manifiestos pueden intervenir. En apps empaquetadas participa el **package dependency graph**, cuyas dependencias se buscan según el manifiesto del paquete. También importan la lista de módulos ya cargados, Known DLLs, API sets, redirecciones y si el programa pidió una ruta completa o usó flags de búsqueda.

Para analizarlo, registra si el ejecutable está empaquetado, qué manifiesto y flags de carga observas, el nombre solicitado y el **path real** de la imagen cargada. Confírmalo en un proceso propio con debugger o Process Monitor; no infieras el archivo final por el nombre del import. En software propio, preferí rutas explícitas o LoadLibraryEx con flags de búsqueda seguros y dependencias conocidas.

La capa nativa: de kernel32 a ntdll y la llamada al sistema

Una llamada documentada de Win32 no es la última parada. En los sistemas actuales, buena parte de la implementación de las funciones de `kernel32.dll` vive en `kernelbase.dll` (conocimiento externo), y casi todo lo que necesita al kernel termina en una función **nativa** de `ntdll.dll` con prefijo `Nt` o `Zw`, que hace la **llamada al sistema** y entra en kernel mode. Microsoft documenta que, llamadas desde user mode, las

versiones `Nt` y `Zw` se comportan igual, y que la API nativa es en gran parte interna: funciones como `NtQueryInformationProcess` "pueden cambiar o no estar disponibles" en versiones futuras.

Cómo se ve al analizar:

- En una pila de llamadas, el último frame de user mode suele ser `ntdll!NtAlgo`, no la API que llamó el programa. Para entender una operación de archivo, por ejemplo, la cadena típica es `CreateFileW` → (kernelbase) → `NtCreateFile` → kernel.
- Un *stub* de `ntdll` en x64 es corto: carga el número de servicio en `EAX` y ejecuta `syscall`. Ese número **cambia entre builds** de Windows.
- **Llamadas al sistema directas.** Si encontrás una instrucción `syscall` (o `sysenter` / `int 2Eh` en x86) **dentro del código del propio programa**, fuera de `ntdll`, el programa está saltándose la capa de user mode: no pasa por las funciones de `ntdll` donde un depurador, una herramienta de monitoreo o un producto de seguridad suelen observar. Como los números cambian por build, ese código o los tiene fijos (y queda atado a una versión) o los lee de `ntdll` en ejecución. Es una señal de anti-análisis o evasión para **documentar**, no algo para replicar.

Ejemplo ilustrativo: un resolvedor sin cadenas

*(Listado **ilustrativo** de x64, construido para mostrar el patrón; los offsets siguen las estructuras documentadas `PEB`, `PEB_LDR_DATA` y `LDR_DATA_TABLE_ENTRY`; la constante de hash es inventada.)*

```
mov   rax, gs:[60h]          ; PEB
mov   rax, [rax+18h]         ; PEB->Ldr
mov   rsi, [rax+20h]         ; Ldr->InMemoryOrderModuleList.Flink
siguiente_modulo:
mov   rdi, [rsi+20h]         ; DllBase (offset relativo a InMemoryOrderLinks)
; ... ubicar el directorio de exports de ese módulo ...
siguiente_nombre:
xor   edx, edx               ; hash = 0
hash_loop:
movzx eax, byte ptr [rcx]    ; siguiente carácter del nombre exportado
test  al, al
jz    comparar
ror   edx, 0Dh               ; rotar ...
add   edx, eax               ; ... y sumar: una función de hash simple
inc   rcx
jmp   hash_loop
comparar:
cmp   edx, 1A2B3C4Dh         ; ¿es el hash de la API buscada? (valor ilustrativo)
jne   siguiente_nombre
```

Lectura: no hay `GetProcAddress` ni cadenas con nombres de API; la pista son `gs:[60h]`, la cadena de offsets hacia la lista de módulos, el bucle de hash y la constante. En el informe se anota "resolución dinámica por hash; API identificada como X porque la constante coincide con el hash de su nombre con esta función", con la evidencia de esa coincidencia.

Ejemplo paso a paso: un `Load Config` y un debug directory reales de leer

*(Ejemplo **ilustrativo**: valores construidos; mostrá los campos, no las direcciones exactas.)*

Compilado con `/GS /guard:cf`, PE-bear muestra:

```
Load Config Directory
  Size                          0x0118
  SecurityCookie                0x0040B120   → hay /GS (stack canary)
  SEHandlerTable                0x00000000   → x64: SEH va por .pdata, no acá
  GuardCFCheckFunctionPointer   0x0040B008
  GuardCFFunctionTable          0x0040A800
  GuardFlags                    0x00010500   → CFG activo (INSTRUMENTED)

Debug Directory  (type = CODEVIEW / RSDS)
  PDB: D:\dev\miapp\x64\Release\miapp.pdb
  GUID: 7F3A... age 1
```

Sin ejecutar nada ya sabés: tiene **/GS** y **CFG**, es **x64** (SEH por tablas), y el `.pdb` filtra la ruta de compilación (`D:\dev\miapp\...`: usuario, proyecto, configuración Release). Ese GUID + age es lo que usarías para cargar los símbolos correctos.

Ampliación editorial basada en fuentes externas — Anatomía de `DllMain`, `PEB->LoaderLock` y detección de *Export Forwarding / DLL Sideloading*

Cuando el binario analizado es una biblioteca dinámica (`.dll` con el flag `IMAGE_FILE_DLL = 0x2000` en `FileHeader.Characteristics`), el punto de entrada (`AddressOfEntryPoint`) no tiene la firma de `main`, sino la de `DllMain`, invocada por el cargador de Windows (`ntdll!LdrpCallInitRoutine`) bajo reglas estrictas de sincronización (*Microsoft Learn: Dynamic-Link Library Entry-Point Function y Best Practices for Creating DLLs*) (**Hecho confirmado**):

1. **Firma de `DllMain` y códigos `fdwReason`:**

```
BOOL WINAPI DllMain(
HINSTANCE hinstDLL,  // RCX (x64) / [esp+4] (x86): Dirección base (ImageBase) donde se mapeó
la DLL
DWORD     fdwReason, // RDX (x64) / [esp+8] (x86): Motivo de la llamada (0 a 3)
LPVOID    lpvReserved // R8  (x64) / [esp+0Ch]: NULL si es carga/descarga dinámica
(LoadLibrary/FreeLibrary)
);
```

- `DLL_PROCESS_ATTACH (1)` : La DLL se mapea por primera vez en el proceso. Si devuelve `FALSE (0)` , el cargador aborta o descarga la DLL.
- `DLL_THREAD_ATTACH (2)` y `DLL_THREAD_DETACH (3)` : Se crea o termina un hilo en el proceso (a menos que la DLL haya llamado a `DisableThreadLibraryCalls`).
- `DLL_PROCESS_DETACH (0)` : La DLL se descarga (`lpvReserved == NULL` vía `FreeLibrary` , o `lpvReserved != NULL` cuando el proceso está terminando).

2. **El *Loader Lock* (`PEB->LoaderLock`) y por qué importa en el análisis:**

Antes de ejecutar cualquier `DllMain` (o cualquier TLS callback), `ntdll.dll` adquiere la sección crítica global del cargador apuntada por `PEB->LoaderLock` (offset `+0x110` en x64; `+0xA0` en x86). Mientras ese candado está tomado, ninguna otra operación de carga o enumeración de módulos puede avanzar en otro hilo. Por eso:

- Llamar dentro de `DllMain` a `LoadLibraryEx` , `CoInitializeEx` , funciones de `user32.dll` / `shell32.dll` , o crear un hilo y esperarlo con `WaitForSingleObject` produce un interbloqueo (*deadlock*), porque el hilo nuevo intenta adquirir el mismo `LoaderLock` para recibir `DLL_THREAD_ATTACH` .
- Para depurar el arranque de una DLL en `x64dbg` , activá `Options -> Preferences -> Events -> DLL Entry` (o `DLL Load`) y ejecutá la DLL con un anfitrión propio o `rundll32.exe copia_dll.dll,FuncionExportada` .

3. **Export Forwarding y *DLL Proxying / Sideloading*:**

En escenarios de *DLL Sideloading*, un ejecutable legítimo firmado busca una DLL por nombre en su propio directorio (p. ej. `version.dll` o `winhttp.dll`). Si un tercero coloca allí una DLL propia con el mismo nombre, tiene que reenviar (*forward*) las exportaciones originales hacia la DLL real de `C:\Windows\System32` para que la aplicación anfitriona no falle.

- **Cómo se codifica en el PE un *Forwarded Export*:** En `IMAGE_EXPORT_DIRECTORY` , si el valor de `AddressOfFunctions[i]` (una RVA) cae **dentro del rango del propio directorio de exportaciones** (`DataDirectory[0].VirtualAddress <= RVA < DataDirectory[0].VirtualAddress + DataDirectory[0].Size`), el cargador no lo interpreta como código ejecutable, sino como un puntero a una cadena ASCII con el formato `"NombreDLLDestino.NombreFuncion"` (o `"NombreDLL.#Ordinal"`).
- **Cómo se genera en C (MSVC) para pruebas de laboratorio:**

```
// Reenvío estático en el linker sin escribir código wrapper:
#pragma comment(linker,
"/export:GetFileVersionInfoW=C:\\Windows\\System32\\version.GetFileVersionInfoW")
```

- **Script de auditoría en Python (`pefile`) para detectar exportaciones reenviadas:**

```
import pefile

pe = pefile.PE("muestra.dll")
if hasattr(pe, "DIRECTORY_ENTRY_EXPORT"):
for exp in pe.DIRECTORY_ENTRY_EXPORT.symbols:
name = exp.name.decode() if exp.name else f"ord_{exp.ordinal}"
if exp.forwarder:
print(f"[FORWARDED] {name} -> {exp.forwarder.decode()}")
else:
print(f"[CODE RVA ] {name} -> {exp.address:#08x}")
```

Práctica

En tu VM, con un **binario propio** (compilá con /GS /guard:cf en MSVC) y PE-bear:

1. Encontrá el **Rich header** y confirma qué sugiere al compararlo con una compilación propia de MinGW.
2. Abrí el directorio de **recursos** y ubicá version info y, si pusiste uno, un RCDATA.
3. Mirá **Load Config**: ¿apunta a un security cookie? ¿Hay tabla CFG?
4. Mirá el **debug directory**: ¿quedó la ruta del PDB? Anotá qué revela de tu entorno.
5. En un programa propio, obtené una API con LoadLibraryW + GetProcAddress y confirma por qué no figura como import estático.
6. En el depurador, seguí una llamada de tu programa hasta ntdll y registra lo que puedes concluir del stub en tu build, sin generalizar números de servicio entre versiones.
7. Con Get-AuthenticodeSignature (PowerShell) o signtool verify /pa (Windows SDK), registra el estado de firma y los datos del firmante; anota que una firma válida no demuestra que el programa sea benigno.
8. En una aplicación propia, identifica si está empaquetada y qué manifiesto/flags de carga aplican; compara el nombre solicitado de una DLL con la ruta realmente observada en el proceso.

Preguntas de repaso

▼ **¿Qué te dice la presencia (o ausencia) del Rich header?**

> Que el binario se armó (o no) con el toolchain de **MSVC**; además sus `@comp.id` ayudan a datar y agrupar. Su ausencia sugiere MinGW u otro compilador.

▼ **Buscás una API en los imports y no está, pero sabés que el programa la usa. ¿Dónde más mirás?**

> En los **imports diferidos** (delay-load): esa DLL se carga la primera vez que se usa, vía `__delayLoadHelper2`, y no aparece en la tabla de imports normal.

▼ **¿Para qué sirve `.pdata` al analizar un x64 sin símbolos?**

> Para **delimitar funciones** (cada entrada cubre un rango) y entender el prólogo/epílogo; en x64 el unwind vive ahí, no en el stack.

▼ **¿Qué información sensible puede filtrar el debug directory?**

> La **ruta del `.pdb`**: nombre de proyecto, usuario y estructura de carpetas del que compiló; y el GUID/age para emparejar símbolos.

▼ **Un programa usa una API que no figura en sus imports ni en sus cadenas. ¿Qué buscás?**

> Resolución dinámica por **hash**: acceso al PEB (`gs:[60h]` / `fs:[30h]`), recorrido de la lista de módulos y de los exports, un bucle que calcula un hash y un `cmp` contra una constante.

▼ **¿Qué es un import `api-ms-win-core-…`?**

> Un **API set**: el nombre de un contrato que el cargador resuelve contra la DLL que lo implementa; puede no existir un archivo con ese nombre.

▼ **¿Por qué una instrucción `syscall` en el código del propio programa es una señal a documentar?**

> Porque el programa entra al kernel sin pasar por `ntdll`, donde suelen observar depuradores y herramientas; además los números de servicio cambian entre builds.

▼ **¿Qué defensas podés leer en Load Config sin ejecutar el binario?**

> El **security cookie** (`/GS`), la tabla **SafeSEH** (32 bits) y las tablas de **CFG** (destinos válidos de llamadas indirectas).

Puerta de salida

Podés: (1) ubicar y explicar Rich header, recursos, delay-load, `.pdata`, debug/PDB y Load Config y firma Authenticode; (además) explicar por qué la IAT no determina por sí sola la ruta efectiva de una DLL; (2) decir qué pista da cada uno antes del entry point; (3) deducir compilador y defensas de un binario solo leyendo sus directorios; (4) reconocer resolución dinámica de APIs (por nombre, por recorrido del PEB y por hash); (5) ubicar la capa nativa (`ntdll`, `Nt*`, `syscall`) en una pila de llamadas y reconocer una llamada al sistema directa.

Lo que queda fuera

Escribir resolvedores o llamadas al sistema propias, las tablas de números de servicio por versión, el formato completo de `LDR_DATA_TABLE_ENTRY` (la mayoría de sus campos son reservados) y las técnicas para ocultar imports en sí no se desarrollan: la lección enseña a reconocerlas.

Fuentes de esta lección

- Microsoft Learn: Understanding executable file signing, Dynamic-link library search order, Dynamic-link library security.
- `Reversing desde cero, cap. 05` — VA, RVA y offsets: cómo se navega dentro del PE.
- El formato PE (1.4) — base: cabeceras, secciones, imports e IAT, sobre la que se apoya esta lección.
- Curso IDA · tutes 25–28 — uso de `.pdb`/símbolos para volver legible un binario.
- Microsoft Learn: PE Format, GetProcAddress, LoadLibraryW, PEB, PEB_LDR_DATA, Windows API sets, Using Nt and Zw versions of the native system services, NtQueryInformationProcess.

Síntesis de estudio de esta lección

Núcleo conceptual: el recorrido VA/RVA/offset del capítulo 05, la base de PE de 1.4 y el uso de símbolos del Curso IDA. **Conocimiento externo:** Rich header, directorio de recursos, delay-load, `.pdata`/unwind, debug/PDB, Load Config, resolución dinámica de APIs (incluido el hashing), API sets, Authenticode, búsqueda de DLL en apps empaquetadas y no empaquetadas, resolución efectiva de DLL y la capa nativa. **Ejemplo ilustrativo:** el resolvedor por hash (constante inventada). **Recomendación editorial:** leer los directorios antes del entry point y la práctica con binario propio.

Nota de lectura sobre el listado x64 — Por qué [rsi+20h] apunta a DllBase al recorrer el PEB

En el listado ilustrativo x64 de esta lección, `mov rsi, [rax+20h]` lee `Ldr->InMemoryOrderModuleList.Flink`. Como ese enlace apunta al campo `InMemoryOrderLinks` (ubicado en el offset `+0x10` dentro de `LDR_DATA_TABLE_ENTRY` en x64, o `+0x08` en x86) y no al inicio `+0x00` de la estructura, todos los accesos posteriores con base en `rsi` llevan restado ese desplazamiento.

Ampliación editorial basada en fuentes externas — Tabla completa de offsets de PEB, PEB_LDR_DATA y LDR_DATA_TABLE_ENTRY en x86 y x64

De acuerdo con PEB (winternl.h) y PEB_LDR_DATA (winternl.h) en Microsoft Learn *(Hecho confirmado)*:

Estructura y campo	Offset en x86 (32 bits)	Offset en x64 (64 bits)	Relativo desde InMemoryOrderLinks (x86 / x64)
Puntero al `PEB`	`fs:[0x30]`	`gs:[0x60]`	—
`PEB->Ldr` (`PEB_LDR_DATA*`)	`+0x0C`	`+0x18`	—
`Ldr->InLoadOrderModuleList`	`+0x0C`	`+0x10`	—
`Ldr->InMemoryOrderModuleList`	`+0x14`	`+0x20`	—
`Ldr->InInitializationOrderModuleList`	`+0x1C`	`+0x30`	—
`LDR_DATA_TABLE_ENTRY.InLoadOrderLinks`	`+0x00`	`+0x00`	`-0x08` / `-0x10`
`LDR_DATA_TABLE_ENTRY.InMemoryOrderLinks`	`+0x08`	`+0x10`	`0x00` / `0x00` *(base de RSI)*
`LDR_DATA_TABLE_ENTRY.InInitializationOrderLinks`	`+0x10`	`+0x20`	`+0x08` / `+0x10`
`LDR_DATA_TABLE_ENTRY.DllBase`	**`+0x18`**	**`+0x30`**	**`+0x10` (x86) / `+0x20` (x64)**
`LDR_DATA_TABLE_ENTRY.EntryPoint`	`+0x1C`	`+0x38`	`+0x14` (x86) / `+0x28` (x64)
`LDR_DATA_TABLE_ENTRY.FullDllName` (`UNICODE_STRING`)	`+0x24`	`+0x48`	`+0x1C` (x86) / `+0x38` (x64)
`LDR_DATA_TABLE_ENTRY.BaseDllName` (`UNICODE_STRING`)	`+0x2C`	`+0x58`	`+0x24` (x86) / `+0x48` (x64)

Taxonomía anti-análisis: reconocer, no evadir

> **Objetivo de la lección**
>
> Tener un **mapa** de las técnicas que un binario usa para dificultar su análisis y saber **reconocerlas** cuando aparecen. **Prerrequisitos:** debugger (2.6), PE y TLS (1.4, 2.11), ofuscación (2.10).

Esto es **reconocimiento defensivo**: clasificar y detectar estas técnicas para analizar mejor. No se enseña a **evadirlas** para proteger software comercial; la mayoría de los títulos de `Teorias_Numeradas` que las usan con ese fin quedan fuera del recorrido.

Las cuatro familias

Familia	Qué intenta	Ejemplos de señales
Anti-debug	Detectar que hay un debugger y cambiar de comportamiento o cerrarse	`IsDebuggerPresent`, `PEB.BeingDebugged`, `NtGlobalFlag`, flags del heap, `NtQueryInformationProcess`, chequeos de tiempo (`rdtsc`, `GetTickCount`), detección de breakpoints (`0xCC`, registros `DR`)
Anti-VM / anti-sandbox	Detectar que corre en una máquina virtual o sandbox	Claves de registro de VMware/VBox, direcciones MAC conocidas, drivers/dispositivos, pocos núcleos/poca RAM, nombres de usuario típicos de sandbox
Anti-disassembly	Confundir al desensamblador estático	Instrucciones solapadas, saltos al medio, `jmp` a registro, basura entre bloques (lo viste en 2.10)
Anti-dump	Dificultar volcar el proceso de memoria	Borrar la cabecera PE en memoria, `SizeOfImage` alterado, secciones sin permisos esperados

Enfoque tradicional

El capítulo 22 introduce anti-debug "desde lo más básico": qué es (técnicas para **detectar** un debugger y reaccionar), el comando `hide` de x64dbg para ocultarse, y cómo un programa detecta **breakpoints por software y por hardware** (`Reversing desde cero, cap. 22`).

El capítulo 24 usa una **aplicación propia** que aplica varias técnicas a la vez: al ejecutarla suelta "no encontró debugger", pero si hay un IDA abierto —**aunque no cargue el programa**— lo detecta por el **nombre del proceso/ventana** y muestra "detected by IDA"; lo mismo con `x32dbg` ("detected by x32dbg"). Es el ejemplo perfecto de **detección por entorno**, no solo por breakpoint (`Reversing desde cero, cap. 24`).

Cómo detectarlas al analizar

- **Por imports/strings:** `IsDebuggerPresent`, `CheckRemoteDebuggerPresent`, `NtQueryInformationProcess`, nombres de procesos (`ida`, `x32dbg`, `ollydbg`), claves de VM.
- **Por comportamiento:** el programa "anda distinto" cuando lo abrís con un debugger o dentro de una VM. Ese cambio **es** la pista.

- **Dónde miran temprano:** acordate de los **TLS callbacks** (2.11): un chequeo anti-debug puesto ahí corre antes de `main`.

Existen herramientas que **ocultan** el debugger (el comando `hide` de x64dbg, plugins tipo ScyllaHide). Se mencionan **solo para que las reconozcas** si aparecen; esquivar protecciones queda fuera del marco de esta lección, que es de **reconocimiento y documentación**.

Ejemplo paso a paso: dos formas del mismo chequeo

*(Ejemplo **ilustrativo**: listado representativo de un anti-debug típico.)*

El mismo "¿me están debuggeando?" se ve de dos maneras:

```
; (a) por API — la más fácil de reconocer
call  IsDebuggerPresent          ; kernel32
test  eax, eax
jnz   detectado

; (b) a mano, leyendo el PEB (x64) — sin llamar a ninguna API
mov   rax, gs:[0x60]             ; RAX = puntero al PEB
movzx eax, byte ptr [rax+2]      ; PEB.BeingDebugged (offset 0x02)
test  al, al
jnz   detectado
```

La forma (a) la cazás en los imports (`IsDebuggerPresent`). La (b) **no usa imports**: la pista es el acceso a `gs:[0x60]` (en x86 sería `fs:[0x30]`) y el offset `+2`. Reconocer el patrón del PEB es lo que te permite detectar el anti-debug aunque no haya una API delatora. Lo documentás y seguís; no hace falta evadirlo para entenderlo.

Más variantes del mismo chequeo

Las técnicas cambian de forma, pero cada una deja una **firma** reconocible. Estas tres aparecen a menudo (**conocimiento externo**; lo citado viene de la documentación de Microsoft):

`CheckRemoteDebuggerPresent`. Recibe un handle de proceso y un puntero a un `BOOL`, y escribe `TRUE` si ese proceso está siendo depurado. El "remote" del nombre no significa otra máquina: según Microsoft, el debugger está en **otro proceso**. La señal en el listado es el `call` y la lectura posterior de la variable, no el valor de retorno (que solo indica si la llamada funcionó).

`NtQueryInformationProcess` con `ProcessDebugPort`. Es una función de `ntdll` (la capa nativa de 2.14); Microsoft la documenta como interna y sujeta a cambios, y no tiene biblioteca de importación, así que el programa suele obtenerla con `GetProcAddress`. Con la clase `ProcessDebugPort` (valor 7) devuelve un número de puerto: **distinto de cero** indica que el proceso corre bajo un debugger de modo usuario.

*(Ejemplo **ilustrativo**, x64: la consulta del puerto de depuración.)*

```
; rbx = GetProcAddress(GetModuleHandleW(L"ntdll.dll"), "NtQueryInformationProcess")
call  cs:GetCurrentProcess
mov   rcx, rax                    ; ProcessHandle
mov   edx, 7                      ; ProcessInformationClass = ProcessDebugPort
lea   r8, [rsp+30h]               ; &puerto (DWORD_PTR)
mov   r9d, 8                      ; tamaño del buffer
mov   qword ptr [rsp+20h], 0      ; ReturnLength = NULL
call  rbx
cmp   qword ptr [rsp+30h], 0      ; distinto de 0 -> hay debugger
jnz   detectado
```

Qué delata: la cadena `"NtQueryInformationProcess"` (o su hash, si la resolución está ofuscada), la constante `7` en `EDX` y la comparación del buffer contra cero.

Chequeos de tiempo. Dos lecturas de `rdtsc` (o de `GetTickCount` / `QueryPerformanceCounter`) alrededor de un fragmento corto y una comparación de la diferencia contra un umbral: si paso a paso tardó demasiado, "hay alguien mirando". La firma es el **par de lecturas** con una resta y un `cmp` contra una constante grande.

Anti-VM por `CPUID`. La instrucción `cpuid` con `EAX = 1` devuelve en el bit 31 de `ECX` el indicador de **hipervisor presente**; con `EAX = 0x40000000` un hipervisor devuelve su identificación en `EBX`, `ECX` y `EDX` (en el de Microsoft, los textos `"Micr"`, `"osof"`, `"t Hv"`). Ver `cpuid` seguido de `bt ecx, 31` o de comparaciones contra esas cadenas es una señal anti-VM. **Límite de la señal:** en un Windows con Hyper-V o seguridad basada en virtualización (VBS) activa, el propio sistema corre sobre un hipervisor, así que el bit también aparece en una máquina física. Un programa que lo usa como prueba de "estoy en una VM" se equivoca, y eso también se documenta.

Cómo documentar lo que encontrás

Cada chequeo hallado se registra en una fila, con la misma disciplina de evidencia de la Etapa 4:

Dirección / función	Técnica	Familia	Señal que la delata	Efecto observado	Cómo se confirmó
`0x140001120` (TLS)	`PEB.BeingDebugged`	Anti-debug	`gs:[60h]` + `[rax+2]`	Con debugger muestra "modo seguro"	Breakpoint en el `jnz`, `AL = 1`
`0x1400013A0`	`ProcessDebugPort`	Anti-debug	`GetProcAddress` + `edx = 7`	Igual que la anterior	Valor del buffer con y sin debugger
`0x140001500`	`CPUID` bit 31	Anti-VM	`cpuid` + `bt ecx, 31`	Mensaje "entorno virtual"	Mismo resultado en VM y en host con VBS

*(Filas **ilustrativas**: direcciones y efectos de un programa de práctica hipotético.)*

La columna "cómo se confirmó" es la que distingue una sospecha de un hallazgo: sin ella, la fila es una hipótesis.

Ampliación editorial basada en fuentes externas — Verificación de marcos de pila y detección de *Call Stack Spoofing* mediante `.pdata` y validación de sitios de `CALL`

En la lección 2.15 vimos que las técnicas de *Call Stack Spoofing* fabrican marcos de pila sintéticos para ocultar el origen real de una llamada al sistema, y en 2D.11 vimos el layout binario de `RUNTIME_FUNCTION` (`.pdata`) y `UNWIND_INFO` (`.xdata`) (*Microsoft Learn: x64 stack usage y RtlVirtualUnwind*) (**Hecho confirmado**). Un analista o motor defensivo verifica la autenticidad de una pila de llamadas en x64 aplicando tres comprobaciones encadenadas sobre cada marco:

1. **Respaldo de módulo (`RtlLookupFunctionEntry`):**

Para cada `ControlPc` (`RIP` actual o dirección de retorno en la pila), debe existir una entrada `RUNTIME_FUNCTION` válida dentro de la sección `.pdata` de un módulo PE firmado/respaldado en disco (`Image` en el VAD, no `MEM_PRIVATE`). Las únicas funciones x64 sin `.pdata` son las *funciones hoja* (*leaf functions*), que por definición de la ABI **no pueden modificar `RSP` ni llamar a ninguna otra función** (solo pueden estar en el frame `0` del tope de la pila, nunca en un frame intermedio).

2. **Coherencia del tamaño de marco (`UNWIND_CODE`):**

La distancia entre el `RSP` de un marco y el `RSP` del marco padre debe coincidir exactamente con la suma de los códigos `UWOP_ALLOC_SMALL` (`OpInfo * 8 + 8`), `UWOP_ALLOC_LARGE` y `UWOP_PUSH_NONVOL` (`8` bytes por registro no volátil salvado) declarados en `UNWIND_INFO` .

3. **Validación de la instrucción previa a `ReturnAddress` (detección de *Gadgets*):**

Cuando `call destino` se ejecuta, empuja en la pila la dirección de la instrucción inmediatamente siguiente (`ReturnAddress`). Por lo tanto, los **2 a 7 bytes anteriores a `ReturnAddress`** en memoria ejecutable deben decodificarse como una instrucción `CALL` legítima:

Bytes previos a `ReturnAddress`	Instrucción x64	Interpretación en auditoría de pila
`E8 xx xx xx xx` (`ReturnAddress - 5`)	`call rel32`	Llamada directa normal. Se verifica que el destino calculado (`ReturnAddress + rel32`) coincida con el inicio de la función hija.
`FF 15 xx xx xx xx` (`ReturnAddress - 6`)	`call qword ptr [rip+disp32]`	Llamada indirecta vía IAT o puntero global (normal en llamadas a DLLs).
`FF D0` .. `FF D7` o `41 FF D0` .. `D7` (`-2` / `-3`)	`call rax` .. `r15`	Llamada indirecta por registro (vtable, callback o `GetProcAddress`).
`FF 23` / `FF 26` / `FF 27` (`jmp [rbx]` / `[rsi]` / `[rdi]`)	`jmp qword ptr [reg]`	**Anomalía crítica (*Gadget de Call Stack Spoofing*):** la dirección en la pila apunta después de un `JMP` , no de un `CALL` , o el `call rel32` apunta a una función distinta de la que está en el frame hijo.

Inspección rápida en WinDbg ante una pila sospechosa:

```
0:000> kn                                  ; Lista los números de frame, Child-SP y Call
Site
0:000> !fnent <Call_Site>                  ; Verifica la entrada RUNTIME_FUNCTION y el
tamaño del marco en .xdata
0:000> ub <Call_Site> L1                   ; Desensambla la instrucción inmediatamente
ANTERIOR al retorno (debe ser CALL)
```

Práctica

En tu VM, con un **programa propio** (podés compilar uno que llame a `IsDebuggerPresent` y cambie un mensaje):

1. Hacé una tabla de tres columnas: técnica, familia, cómo la detectaste (import / string / comportamiento).
2. Ejecutá tu binario con y sin debugger y anotá la diferencia.
3. Poné el chequeo en un **TLS callback** y verificá que corre antes de `main`.
4. **Documentá** el cambio de comportamiento: qué muestra el programa con y sin debugger, y qué señal (import, string, lectura del PEB) lo delata. El objetivo es reconocer y dejar registrado, no evadir.
5. Agregá a tu programa `CheckRemoteDebuggerPresent` y la consulta de `ProcessDebugPort` (resolviendo `NtQueryInformationProcess` con `GetProcAddress`). Anotá el valor que devuelve cada una con y sin debugger.
6. Agregá la lectura de `CPUID` (bit 31 y, si está, la identificación del hipervisor) e imprimí el resultado en la VM. Escribí en la tabla de documentación por qué esa señal no prueba por sí sola que el equipo sea virtual.

Preguntas de repaso

▼ **¿Cuál es la diferencia entre anti-debug y anti-disassembly?**

> Anti-debug detecta/estorba la ejecución **dinámica** bajo un debugger; anti-disassembly confunde el análisis **estático** del desensamblador (instrucciones solapadas, saltos al medio). Son familias distintas.

▼ **En el cap. 24, ¿por qué el programa dice "detected by IDA" sin que IDA cargue el binario?**

> Porque detecta por **entorno**: busca el proceso/ventana de IDA abierto en el sistema, no necesita estar siendo debuggeado por él. Es detección por nombre de proceso.

▼ **Nombrá tres señales de anti-debug visibles sin ejecutar.**

> Imports/uso de `IsDebuggerPresent`, `CheckRemoteDebuggerPresent`, `NtQueryInformationProcess`; lectura de `PEB.BeingDebugged` / `NtGlobalFlag`; o strings con nombres de debuggers.

▼ **¿Por qué conviene recordar los TLS callbacks acá?**

> Porque un chequeo anti-debug puesto en un callback corre **antes de** `main`: si solo mirás desde el entry point, ya decidió.

▼ **¿Qué significa un `ProcessDebugPort` distinto de cero y por qué el programa suele obtener la función con `GetProcAddress`?**

> Que el proceso corre bajo un debugger de modo usuario. `NtQueryInformationProcess` es una función interna de `ntdll` sin biblioteca de importación, así que se resuelve en tiempo de ejecución; la cadena con su nombre (o su hash) es una pista.

▼ **¿Por qué el bit de hipervisor de `CPUID` no prueba que el programa corre en una VM?**

> Porque un Windows con Hyper-V o VBS activo corre sobre un hipervisor aunque la máquina sea física, así que el bit también aparece ahí.

▼ **¿Qué columna convierte una fila de la tabla de documentación en un hallazgo?**

> "Cómo se confirmó": el breakpoint, el valor leído o la comparación con y sin debugger. Sin ella, la fila es una hipótesis.

▼ **¿Por qué el cambio de comportamiento con y sin debugger es, en sí, una pista?**

> Porque significa que el programa **chequea** su entorno: si hace algo distinto al correr bajo un debugger (o en una VM), tiene anti-análisis. Ese cambio te dice dónde buscar el chequeo, aunque no lo evites.

Puerta de salida

Podés: (1) clasificar una técnica en anti-debug/anti-VM/anti-disassembly/anti-dump; (2) detectar anti-debug por imports, strings y comportamiento; (3) explicar la detección por entorno del cap. 24; (4) relacionar un chequeo temprano con un TLS callback; (5) reconocer `CheckRemoteDebuggerPresent`, `ProcessDebugPort`, un chequeo de tiempo y un chequeo de `CPUID` por su firma en el listado; (6) registrar cada chequeo en una tabla con su confirmación.

Lo que sigue en 2.28 (y lo que queda fuera)

Esta lección se enfoca en el reconocimiento y la documentación defensiva. Cómo neutralizar estos chequeos en un entorno controlado de laboratorio (usando `ScyllaHide` y hooks de `Frida` sobre binarios propios o en VM descartable) se aborda en la lección **2.28 · Neutralizar anti-análisis en laboratorio**. Los anti-debug propietarios de protectores comerciales sobre software de terceros quedan fuera de alcance.

Fuentes de esta lección

- `Reversing desde cero, cap. 22` — qué es anti-debug, `hide` de x64dbg, detección de breakpoints software y hardware.
- `Reversing desde cero, cap. 24` — app propia con varias técnicas; detección por nombre de proceso ("detected by IDA/x32dbg").
- Teorias_Numeradas · clasificación — títulos de anti-debug (material por título, no leído; muchos con fin de evasión comercial, fuera de alcance).

- Microsoft Learn (conocimiento externo): IsDebuggerPresent, CheckRemoteDebuggerPresent, NtQueryInformationProcess (clase `ProcessDebugPort`, función interna sin biblioteca de importación) y Hypervisor Feature Discovery (bit de hipervisor presente y hoja `0x40000000`).

Síntesis de estudio de esta lección

Núcleo conceptual: el `hide` y la detección de breakpoints del capítulo 22 y la detección por entorno del 24. **Conocimiento externo:** la taxonomía de las cuatro familias y las señales concretas (PEB, NtGlobalFlag, `ProcessDebugPort`, chequeos de tiempo, `CPUID`, anti-VM, anti-dump). **Ejemplo ilustrativo:** los listados del PEB y de `ProcessDebugPort`, y las filas de la tabla de documentación. **Recomendación editorial:** el enfoque de reconocer y no evadir, la tabla de documentación y la práctica con binario propio.

Ampliación editorial basada en fuentes externas — Evasión en User-Mode: API Hooking de EDRs, Syscalls Directas (Hell's Gate / Halo's Gate) y DLL Unhooking

El análisis de binarios modernos en entornos corporativos exige comprender cómo interactúa el software con los agentes de seguridad (antivirus/EDR) en el espacio de usuario (Microsoft Learn: x64 Architecture & System Calls):

1. Mecanismo de los Inline API Hooks

Los sistemas de seguridad inyectan una DLL en cada proceso para supervisar llamadas sensibles. Para interceptar la ejecución, modifican los primeros bytes de las funciones de `ntdll.dll` reemplazando el prólogo original por un salto incondicional (*inline hook*):

```
; --- ntdll.dll original (Limpia) ---
mov r10, rcx
mov eax, 0x18         ; SSN (System Service Number de NtAllocateVirtualMemory)
test byte ptr [0x7ffe0308], 1
jne short ...
syscall
ret

; --- ntdll.dll hookeada por un EDR ---
jmp qword ptr [rip + 0x10240]   ; Salto (0xE9 o 0xFF 0x25) al motor de análisis del EDR
```

2. Técnicas de Evasión Reversibles

- **Syscalls Directas (Direct Syscalls):** El programa no invoca a `ntdll.dll`; en su lugar, define un stub propio en ensamblador con la instrucción `syscall` y el número SSN correspondiente.
- **Algoritmos Hell's Gate y Halo's Gate:** Como los números SSN cambian entre versiones y compilaciones de Windows:
  - *Hell's Gate (2020):* Parsea la tabla de exportaciones de `ntdll.dll` en memoria, localiza la función deseada y lee los bytes `4C 8B D1 B8 [SSN]`. Si la función tiene un `jmp` (está hookeada), falla.
  - *Halo's Gate:* Supera la limitación inspeccionando las funciones vecinas inmediatamente adyacentes (offsets ±32 bytes). Si la función vecina no está hookeada, lee su SSN y le suma o resta el desplazamiento para deducir el SSN de la función bloqueada.
- **DLL Unhooking:** El binario abre `C:\Windows\System32\ntdll.dll` directamente desde disco con `CreateFileW` + `MapViewOfFile`, localiza la sección `.text` limpia, cambia los permisos de la copia en memoria con `VirtualProtect(PAGE_EXECUTE_READWRITE)` y sobrescribe los hooks con `memcpy`, restaurando el prólogo original de todas las APIs.

3. Criterio de Detección Defensiva

La presencia de instrucciones `syscall` fuera del rango de memoria virtual asignado a `ntdll.dll` o `win32u.dll` constituye una anomalía heurística de máxima prioridad que delata intentos deliberados de evasión de telemetría.

Ampliación editorial basada en fuentes externas — De Direct Syscalls a Indirect Syscalls y Call Stack Spoofing

Aunque las *Direct Syscalls* (Hell's Gate / Halo's Gate) permiten evitar los hooks de `ntdll.dll`, los agentes de seguridad modernos (EDRs) y el propio kernel de Windows implementan contramedidas que detectan esta técnica (Microsoft Learn: x64 Architecture & System Calls y telemetría ETWti):

1. La Limitación de las Syscalls Directas (Inspección de RIP)

Cuando la CPU ejecuta la instrucción `syscall` en Ring 3, el procesador transfiere el control al manejador del kernel (`KiSystemCall64`). Durante esta transición, el kernel captura el registro `RIP` de origen en la estructura del hilo (*trap frame*). Mediante telemetría de kernel (como **ETWti** —*Microsoft-Windows-Threat-Intelligence*— o *ProcessInstrumentationCallback*):

- Si el `RIP` ejecutor reside en la sección `.text` de `ntdll.dll` o `win32u.dll`, la llamada se considera conforme al diseño del sistema.
- Si el `RIP` apunta a la sección `.text` del propio ejecutable anfitrión o a memoria privada (`MEM_PRIVATE`), se genera una alerta inmediata por ejecución de llamadas al sistema anómalas.

2. La Solución: Indirect Syscalls

Para satisfacer la validación del `RIP` sin pasar por los hooks del EDR, el cargador prepara los registros de la llamada en su propio código, pero en lugar de emitir el opcode `0F 05` (`syscall`), realiza un salto (`jmp`) a una instrucción `syscall ; ret` legítima ya existente dentro de `ntdll.dll` :

```
; --- Stub en el binario (Indirect Syscall) ---
mov r10, rcx
mov eax, 0x18                      ; SSN deducido dinámicamente con Halo's Gate
jmp qword ptr [rip + pfn_syscall]  ; Salta a la instrucción "syscall; ret" dentro de ntdll
```

Al ejecutarse el `syscall` físicamente dentro del espacio de memoria de `ntdll.dll`, el `RIP` registrado en el kernel resulta 100 % legítimo.

3. El Problema del Call Stack Walking y Call Stack Spoofing

Aun con *Indirect Syscalls*, los EDRs avanzados inspeccionan la pila de llamadas retrospectiva (*Call Stack Unwinding* guiado por las tablas `.pdata` / `UNWIND_INFO` vistas en la lección 2.14). Si el frame inmediatamente anterior a `ntdll!NtAllocateVirtualMemory` no es una API de alto nivel (como `kernelbase.dll!VirtualAllocEx`), sino memoria no respaldada o un salto anómalo, la llamada es detectada. Para evadir esto, los cargadores avanzados implementan **Call Stack Spoofing** (sintetizando marcos de pila falsos para simular que la llamada se originó en una secuencia legítima de Windows).

Nota sobre Tartarus' Gate: Algunos EDRs modernos colocan su hook `jmp` no en el byte `+0`, sino después de `mov r10, rcx` (en el byte `+3`). La variante *Tartarus' Gate* verifica si el hook está desplazado y ajusta el puntero de lectura para no confundir el prólogo.

Ampliación editorial basada en fuentes externas — PEB.NtGlobalFlag en x86 vs. x64 y clases 0x1E / 0x1F de NtQueryInformationProcess

Según GFlags Flag Table y NtQueryInformationProcess en Microsoft Learn *(Hecho confirmado)*:

- **`PEB.NtGlobalFlag` (valor `0x70`):** Combina `FLG_HEAP_ENABLE_TAIL_CHECK (0x10) | FLG_HEAP_ENABLE_FREE_CHECK (0x20) | FLG_HEAP_VALIDATE_PARAMETERS (0x40) = 0x70`. En **x86** se ubica en `fs:[30h] -> [peb + 0x68]`; en **x64** se desplaza a `gs:[60h] -> [peb + 0xBC]`.
- **`ProcessDebugObjectHandle` (clase `0x1E = 30`):** Consulta el handle al `DEBUG_OBJECT` del kernel. Si no hay debugger, `NtQueryInformationProcess` devuelve `0xC0000353` (`STATUS_PORT_NOT_SET`).
- **`ProcessDebugFlags` (clase `0x1F = 31`):** Consulta `NoDebugInherit`; devuelve `0` si hay depurador activo y `1` si no está siendo depurado.

Ghidra (y Cutter): la alternativa libre, con práctica

Objetivo de la lección

Hacer en **Ghidra** el mismo recorrido que hiciste en IDA, para no depender de una sola herramienta: importar, dejar que autoanalice, seguir strings y referencias, corregir tipos, leer el decompilador y documentar. **Prerrequisitos:** IDA y el análisis estático (2.7), estructuras (2.7).

Por qué sumar Ghidra y Cutter a tu entorno

Los cursos clásicos enseñaban únicamente con IDA de 32 bits y OllyDbg. Incorporar **Ghidra** y **Cutter (radare2)** es indispensable hoy para trabajar con binarios de 64 bits sin depender de una única herramienta comercial. Descargá siempre Ghidra desde su repositorio oficial y verificá su hash SHA-256 antes de instalarlo en la VM.

Por qué una segunda herramienta

Ghidra es gratis y open source (NSA), trae **decompilador** integrado y corre en cualquier sistema. Saber leer el mismo binario en IDA **y** en Ghidra te da dos ventajas: cuando una decompila mal una función, la otra suele aclararla; y no quedás atado a una licencia. Cutter (la GUI de **radare2**) es otra opción libre, más liviana.

El recorrido equivalente al de IDA

1. **Proyecto e import.** Ghidra trabaja con un *project* que contiene uno o varios binarios. `File → Import File`, elegí el `.exe`; Ghidra detecta el formato PE y el procesador.
2. **Autoanálisis.** Al abrirlo por primera vez, ofrece correr los *analyzers*: dejalos con la opción por defecto la primera vez. Son el equivalente al análisis inicial de IDA (arma funciones, resuelve referencias, nombra imports).
3. **Navegación.** Las ventanas clave: **Symbol Tree** (funciones, imports, exports), **Defined Strings** (el equivalente a las strings de IDA), **Listing** (el desensamblado) y **Decompile** (el pseudo-C, como Hex-Rays).
4. **Seguir el hilo.** Doble clic en una string te lleva a su uso; clic derecho → **References** → **Show References to** es el `X` (xrefs) de IDA. Con eso llegás a la lógica igual que en 2.7.
5. **Corregir tipos.** En el decompilador, clic derecho → **Retype Variable** y **Rename Variable/Function**; para estructuras, el **Data Type Manager**. Es el `T`/renombrado de IDA: cada tipo que corregís hace el pseudo-C más legible.
6. **Documentar.** Comentarios (`;`), *bookmarks* y renombrado quedan en el proyecto. Exportá un resumen de hallazgos para tu informe (Etapa 4).

Tipos y estructuras: donde Ghidra más mejora el pseudo-C

El decompilador es tan bueno como los tipos que le das. Tres herramientas cubren casi todo (**conocimiento externo**, guías de Ghidra):

- **Data Type Manager.** Ventana con los tipos del programa y los **archivos de tipos** (`.gdt`) que trae Ghidra; para un PE aplica automáticamente los de Windows, por eso `CreateFileW` ya aparece con sus parámetros. Ahí creás estructuras a mano (*New → Structure*) con los campos que dedujiste, como en IDA y el análisis estático.
- **Auto Create Structure.** En el decompilador, clic derecho sobre una variable que es un puntero → *Auto Create Structure*: Ghidra mira los accesos `[ptr+N]` de la función y arma una estructura con un campo por offset. Es un punto de partida, no el resultado: renombrá los campos y corregí tamaños cuando sepas qué son.
- **Parse C Source.** Si tenés el encabezado `.h` de tu propio programa (o la definición de una estructura documentada), *File → Parse C Source* lo convierte en tipos de Ghidra. Es lo que más rápido transforma un listado de `param_1 + 0x18` en `ctx->contador`.

Regla de trabajo: cada vez que el pseudo-C muestra aritmética de punteros sobre la misma variable, hay una estructura sin declarar.

Scripting y modo headless

Ghidra expone toda su API para scripts en **Java** y **Python** (Jython en el *Script Manager* y, en versiones recientes, también CPython mediante PyGhidra). Es el equivalente a IDAPython de la Etapa 5. Dentro de un script, `currentProgram` es el programa abierto y hay funciones de conveniencia de la API "plana" (*FlatProgramAPI*), como `getReferencesTo(direccion)`.

*(Ejemplo **ilustrativo**, no ejecutado en esta edición: listar las veinte funciones sin nombre más grandes, para decidir por dónde empezar.)*

```
# Ghidra, Script Manager (Python): funciones FUN_ ordenadas por tamaño
fm = currentProgram.getFunctionManager()
sin_nombre = [f for f in fm.getFunctions(True) if f.getName().startswith("FUN_")]
sin_nombre.sort(key=lambda f: f.getBody().getNumAddresses(), reverse=True)
for f in sin_nombre[:20]:
    print(f.getEntryPoint(), f.getName(), f.getBody().getNumAddresses())
```

El **modo headless** corre el autoanálisis y los scripts sin interfaz, para procesar varios binarios en lote o repetir un análisis exactamente igual:

```
analyzeHeadless C:\proyectos_ghidra LabCLS -import hola.exe -postScript listar_fun.py
```

Eso deja el resultado del script en la salida de consola y el binario analizado en el proyecto, que es evidencia reproducible.

Version Tracking: llevar el trabajo de una versión a otra

Si ya renombraste y comentaste la versión 1 de un programa y aparece la versión 2, no hace falta empezar de cero. **Version Tracking** compara dos programas (origen y destino), busca funciones y datos que se corresponden con distintos **correlacionadores** (bytes idénticos, mismo código con otras direcciones, mismas

referencias, mismos símbolos) y te deja **aceptar** cada coincidencia y **trasladar** al destino nombres, comentarios y tipos. Lo que queda sin correspondencia es justamente lo que cambió. Se usa a fondo en 2.20 Patch diffing; acá alcanza con saber que existe.

Ejemplo paso a paso: la misma función en IDA y en Ghidra

*(Ejemplo **ilustrativo**: pseudo-C representativo de los dos decompiladores.)*

La misma rutina de chequeo, decompilada por cada herramienta antes de renombrar:

```
// IDA (Hex-Rays)                        // Ghidra
int sub_401050(char *a1) {               undefined4 FUN_00401050(char *param_1) {
  if ( strlen(a1) != 6 )                   size_t sVar1 = strlen(param_1);
    return 0;                              if (sVar1 != 6) return 0;
  return strcmp(a1, "s3cr3t") == 0;        iVar2 = strcmp(param_1,"s3cr3t");
}                                          return (uint)(iVar2 == 0);
                                         }
```

Dicen lo mismo con otra prosa: la longitud debe ser 6 y el texto `"s3cr3t"`. Dónde **conviene comparar**: a veces Ghidra resuelve mejor una `switch` o una estructura que Hex-Rays enreda (y al revés). Después de **renombrar** (`FUN_00401050` → `check`, `param_1` → `clave`) y **tipar**, el pseudo-C de Ghidra queda tan legible como el de IDA:

```
bool check(char *clave) {
  if (strlen(clave) != 6) return false;
  return strcmp(clave, "s3cr3t") == 0;
}
```

`analyzeHeadless` en Windows: ruta, `-scriptPath` y `-overwrite`

El ejecutable es `analyzeHeadless.bat`, dentro de `<ghidra>\support\` (no está en el PATH). Si tu script no está en los directorios internos de Ghidra ni en `%USERPROFILE%\ghidra_scripts`, pasá `-scriptPath` o falla con *Script not found*. En Ghidra 11.x conviene ponerle al `.py` la cabecera de runtime (`#@runtime Jython` o `#@runtime PyGhidra`). Y si reimportás el mismo binario al mismo proyecto (p. ej. al repetir la práctica), agregá `-overwrite` (o `-process`) o choca con el archivo ya existente:

```
C:\ghidra_11.3.2\support\analyzeHeadless.bat C:\proyectos_ghidra LabCLS ^
  -import C:\lab\hola.exe -overwrite -scriptPath C:\lab\scripts -postScript listar_fun.py
```

Práctica

En tu VM, con el **mismo crackme** que analizaste en IDA (ver Banco de prácticas):

1. Importalo en un proyecto de Ghidra y corré el autoanálisis.
2. Encontrá la misma string clave que usaste en IDA y llegá, por referencias, a la función de decisión.
3. Leé la función en el **decompilador**; renombrá dos variables y confirmá que el pseudo-C mejora.
4. Compará lado a lado con la vista de IDA: ¿coincide el decompilado? ¿Alguna lo hace más claro?
5. (Opcional) Abrí el mismo binario en **Cutter** y repetí el paso 2.
6. Elegí una función de tu programa que reciba un puntero a estructura. Usá *Auto Create Structure*, renombrá los campos y compará el pseudo-C antes y después; si tenés el `.h`, importalo con *Parse C Source* y compará con la estructura automática.
7. Corré el script de ejemplo desde el *Script Manager* y después en modo headless sobre el mismo binario. Guardá la salida como evidencia.

Preguntas de repaso

▼ **¿Qué ventaja concreta da leer el mismo binario en IDA y en Ghidra?**

> Que cuando una **decompila mal** una función, la otra suele aclararla; además no dependés de una licencia. Son dos lecturas del mismo código.

▼ **¿Cuál es el equivalente en Ghidra de las "strings" y de los "xrefs" de IDA?**

> **Defined Strings** para las cadenas y **References** → **Show References to** (clic derecho) para los xrefs.

▼ **¿Qué hacés la primera vez que abrís un binario en Ghidra?**

> Dejar correr el **autoanálisis** con la opción por defecto: arma funciones, resuelve referencias y nombra imports, como el análisis inicial de IDA.

▼ **¿Cómo mejorás el decompilado de una función?**

> **Renombrando** variables/funciones y **corrigiendo tipos** (Retype Variable, Data Type Manager); cada corrección hace el pseudo-C más legible, igual que `T`/rename en IDA.

▼ **¿Qué hace *Auto Create Structure* y por qué no es el resultado final?**

> Crea una estructura con un campo por cada offset que la función usa sobre ese puntero. Es un borrador: no conoce los nombres ni el tamaño total, y puede faltarle campos que solo se usan en otras funciones.

▼ **¿Para qué sirve el modo headless?**

> Para correr el autoanálisis y scripts sin interfaz: procesar varios binarios en lote o repetir el mismo análisis exactamente igual, lo que lo vuelve reproducible.

▼ **¿Qué te ahorra Version Tracking cuando aparece una versión nueva del programa?**

> Rehacer el trabajo: encuentra las funciones correspondientes y traslada nombres, comentarios y tipos; lo que queda sin correspondencia es lo que cambió.

▼ **¿Por qué conviene bajar Ghidra siempre del sitio oficial y verificar su hash?**

> Porque al descargar herramientas de análisis desde el repositorio oficial y comprobar su hash SHA-256 tenés garantía de integridad y evitás ejecutar instaladores alterados por terceros.

Puerta de salida

Podés: (1) importar y autoanalizar un binario en Ghidra; (2) llegar a la función de decisión por strings y referencias; (3) mejorar el decompilado renombrando y tipando; (4) comparar la lectura de IDA y Ghidra del mismo binario; (5) crear y aplicar una estructura con el Data Type Manager o *Auto Create Structure*; (6) correr un script propio desde el *Script Manager* y en modo headless. Con esto cerrás el bloque **2B · Windows intermedio**.

Dónde sigue esto (Lección 2.31)

La automatización avanzada con `PyGhidra` / `DecompInterface`, la representación intermedia `P-Code` y el lenguaje de especificación de procesadores **SLEIGH** (para modelar y decompilar el bytecode de tu propia mini-VM) se desarrollan en la lección **2.31 · Extensiones de Ghidra y SLEIGH**. El trabajo colaborativo con Ghidra Server queda fuera del curso.

Fuentes de esta lección

- **Documentación oficial de Ghidra (NSA):** Ghidra Class · Beginner e Intermediate.
- IDA y el análisis estático (2.7) — el recorrido metodológico que esta lección replica en Ghidra.
- Guías avanzadas de Ghidra: Improving Disassembly and Decompilation (tipos y estructuras en el decompilador), Version Tracking y API de Ghidra (`FlatProgramAPI`, `FunctionManager`).

Síntesis de estudio de esta lección

Núcleo conceptual: el recorrido metodológico de análisis estático (strings → referencias → tipos → decompilador) aplicado sobre Ghidra y Cutter. **Conocimiento externo:** Data Type Manager, *Auto Create Structure*, *Parse C Source*, scripting con PyGhidra, modo `analyzeHeadless` y *Version Tracking*. **Recomendación editorial:** la comparación lado a lado entre Hex-Rays y el decompilador de Ghidra sobre un mismo binario.

Packers didácticos: UPX, stub, OEP, dump e IAT

Objetivo de la lección

Entender qué hace un **packer** y recorrer el desempaquetado **didáctico** de UPX: reconocer el stub, llegar al **OEP**, **dumpear** el proceso y reconstruir la **IAT**. **Prerrequisitos:** PE, secciones e IAT (1.4), PE fino (2.14), debugger (2.6).

Solo UPX didáctico, sobre binarios propios

Esta lección usa **UPX** —un compresor **abierto y reversible**— sobre **programas propios**. UPX trae su propio `upx -d` que descomprime, así que siempre tenés una red de seguridad. **No** se cubre el desempaquetado de **protectores comerciales** (Themida, ACProtect, ASProtect, VMProtect…): eso queda fuera del recorrido, como la Etapa 5 excluida. El objetivo es **entender el mecanismo**, no vulnerar protecciones de terceros.

Qué hace un packer

Un packer toma tu `.exe`, **comprime** (o cifra) el código original y le pone adelante un **stub**: un pequeño programa que, al ejecutarse, **descomprime** el original en memoria y le salta. En disco ves poco; el código real aparece recién en runtime. Señales de UPX en el PE (lo viste en 1.4 y 2.14):

- Secciones llamadas `UPX0` (vacía en disco, grande en memoria: ahí se descomprime) y `UPX1` (el stub + datos comprimidos).
- **Pocos imports** (el stub resuelve el resto en runtime).
- El **entry point** cae en `UPX1`, no en el código original.

El **OEP** (*Original Entry Point*) es la dirección donde empezaba tu programa antes de empacar. Desempaquetar didácticamente es: **llegar al OEP**, **dumpear** la memoria ya descomprimida y **reconstruir la IAT** para que el dump vuelva a ser un `.exe` que corre.

El método (sobre un binario propio)

El capítulo 13 lo plantea como práctica de unpacking **simple**: los ejemplos no tienen IAT redirection ni stolen bytes/code, y su anti-debug es tan básico que alcanza el comando `hide` de x64dbg (`Reversing desde cero, cap. 13`).

1. **Reconocé el packer.** Secciones `UPX0` / `UPX1`, entry en `UPX1`.
2. **Llegá al OEP.** El stub termina restaurando registros y **saltando** al OEP. En UPX el patrón clásico es un `popad` seguido de un salto lejano (`jmp`) a una dirección en la zona que estaba vacía (`UPX0`). Un breakpoint de hardware sobre el stack o seguir hasta ese `jmp` te deja en el OEP.
3. **Dumpeá** el proceso cuando estás en el OEP (con Scylla, o el script que muestra el curso).
4. **Reconstruí la IAT.** El tute 15 del Curso IDA sigue justo acá: después del OEP, **dumpear y reconstruir la IAT** para que el ejecutable vuelva a resolver sus imports (`Curso IDA, tute 15`).

5. **Comprobá con `upx -d`.** Como es UPX, descomprimí el mismo binario con `upx -d` y comparás tu dump con el original: así validás que llegaste bien.

Ejemplo paso a paso: el salto al OEP

*(Ejemplo **ilustrativo** y **de 32 bits (PE32)**: `popad` / `pushad` existen solo en x86, así que este patrón es el de un UPX sobre un binario de 32 bits; en x64 el stub se ve distinto. Direcciones construidas para mostrar el patrón.)*

```
; ...final del stub de UPX, en la sección UPX1...
0045A1F0  popad                     ; restaura los registros que guardó al inicio (pushad)
0045A1F1  jmp     0x00401000        ; salto lejano a UPX0 → ese es el OEP
;--- en 0x00401000 ya estás en el código ORIGINAL ---
00401000  push    ebp
00401001  mov     ebp, esp          ; prólogo típico del main
```

El `popad` + `jmp` fuera de `UPX1` (hacia la zona `UPX0`, antes vacía) es la firma de "acá terminó de descomprimir y salta a tu programa". Parás en `0x00401000`, dumpeás y reconstruís la IAT.

Y en 64 bits, ¿dónde está el `popad`?

No existe a nivel de arquitectura x86-64. En modo largo (*Long Mode* de 64 bits), los opcodes de 32 bits `0x60` (`pushad`) y `0x61` (`popad`) fueron invalidados por Intel y AMD; si la CPU intenta ejecutarlos en x64, genera de inmediato una excepción por opcode no válido (`#UD` - *Invalid Opcode Exception*). Además, en x64 existen 16 registros de propósito general (`RAX`..`R15`) en lugar de 8, por lo que el stub de UPX para x64 debe guardar y restaurar los registros explícitamente mediante instrucciones `push` y `pop` individuales de 64 bits.

Anatomía del Stub de Descompresión UPX en x64 (Entrada y Cola)

Al iniciarse el binario empacado, el entry point en la sección `UPX1` salva el contexto en la pila con una secuencia de `push rax`, `push rcx`, ..., `push r15`. Al terminar de descomprimir el código original sobre la sección `UPX0` (que inicialmente estaba vacía y asignada con permisos de escritura), el stub ejecuta el desenrollado inverso y emite el salto de cola (*Tail Jump*) hacia el **OEP**:

```
; ========================================================================
; Final del stub de descompresión en UPX x64 (al final de la sección UPX1)
; ========================================================================
00007FF780021A40  41 5F               pop     r15
00007FF780021A42  41 5E               pop     r14
00007FF780021A44  41 5D               pop     r13
00007FF780021A46  41 5C               pop     r12
00007FF780021A48  41 5B               pop     r11
00007FF780021A4A  41 5A               pop     r10
00007FF780021A4C  41 59               pop     r9
00007FF780021A4E  41 58               pop     r8
00007FF780021A50  5F                  pop     rdi
00007FF780021A51  5E                  pop     rsi
00007FF780021A52  5D                  pop     rbp
00007FF780021A53  48 83 C4 08         add     rsp, 8     ; descarta el slot alineado de RSP
salvado
00007FF780021A57  5B                  pop     rbx
00007FF780021A58  5A                  pop     rdx
00007FF780021A59  59                  pop     rcx
00007FF780021A5A  58                  pop     rax
00007FF780021A5B  E9 A0 F5 FD FF      jmp     00007FF780001000  ; ¡Tail Jump inter-sección hacia
UPX0!

; ========================================================================
; En 0x00007FF780001000: Código ORIGINAL x64 completamente desempacado (OEP)
; ========================================================================
00007FF780001000  48 83 EC 28         sub     rsp, 28h   ; Prólogo típico x64 (reserva 32 bytes de
shadow space)
00007FF780001004  E8 57 02 00 00      call    00007FF780001260
```

Cómo cazar el OEP en x64 sin depender de una instrucción fija

Dado que cualquier empaquetador moderno puede alterar el orden de los registros o intercalar instrucciones basura (*junk code*) antes del salto final, el método riguroso en ingeniería inversa se basa en la transición de memoria $W \oplus X$ (escribir y luego ejecutar en otra sección):

1. **Método x64dbg (Memory Breakpoint on Execution):**

- Abrí el binario x64 en `x64dbg`.
- Presioná `Alt+M` para abrir la pestaña del mapa de memoria (*Memory Map*).
- Localizá la sección `UPX0` (habitualmente la primera sección con memoria asignada en `ImageBase + 0x1000`).
- Hacé clic derecho sobre la cabecera o primera página de `UPX0` $ o$ **Breakpoint de memoria** $ o$ **Ejecutar (F2)**.
- Presioná `F9` (*Run*). El desempaquetador escribe a toda velocidad en `UPX0` sin detenerse (los accesos de escritura no activan un breakpoint de ejecución).
- En el instante exacto en que la CPU ejecuta el `jmp 0x00007FF780001000`, salta el breakpoint: el registro `RIP` apunta a la primera instrucción del código original (**OEP**).
- En ese punto abrís **Scylla** (`Plugins` $ o$ `Scylla`), hacés clic en `IAT Autosearch` $ o$ `Get Imports` $ o$ `Dump Process` y finalmente `Fix Dump` sobre el archivo guardado.

2. **Método WinDbg (Hardware Execution Breakpoint):**

```
$$ Localizar la ImageBase y la RVA de la primera sección (UPX0)
lm m target64
!dh target64 -f
$$ Supongamos ImageBase = 0x00007ff7`80000000 y RVA de UPX0 = 0x1000:
ba e 1 target64+0x1000
g
$$ El debugger se detendrá en el hardware breakpoint en el primer ciclo de reloj ejecutado en
UPX0:
r rip
u rip L5
```

Señales genéricas de empaquetado

UPX se reconoce por el nombre de sus secciones, pero un packer cualquiera deja otras huellas que se pueden combinar (**conocimiento externo**; los permisos y constantes están en la documentación de Microsoft):

Señal	Dónde se ve	Por qué sugiere empaquetado
Entropía alta de una sección (cerca de 8 bits por byte)	PE-bear, Detect It Easy o un script	Los datos comprimidos o cifrados parecen aleatorios; el código normal tiene entropía bastante menor
Tamaño **en disco** chico y tamaño **virtual** grande	Tabla de secciones	La sección se llena en memoria al descomprimir
Sección **escribible y ejecutable** a la vez	Flags de la sección (`MEM_WRITE` + `MEM_EXECUTE`)	El stub escribe código y después lo ejecuta en el mismo lugar
Pocos imports, con `LoadLibrary` / `GetProcAddress`	Tabla de imports	Los demás se resuelven en ejecución (2.14)
Entry point fuera de la primera sección de código	Cabecera opcional	El primer código que corre es el stub
En ejecución: `VirtualAlloc` / `VirtualProtect` que dejan memoria ejecutable, escritura en ella y salto	Breakpoints en esas APIs	Es el desempaquetado en vivo

Ninguna alcanza sola: un instalador con datos comprimidos tiene entropía alta sin ser un packer, y un programa legítimo puede cambiar permisos de memoria. Como con la huella de compilador (2.13), se suman señales.

*(Ejemplo **ilustrativo**: cálculo de entropía de Shannon de un bloque de bytes, para entender qué muestran las herramientas.)*

```
import math
from collections import Counter

def entropia(datos: bytes) -> float:
    # bits por byte: 0 si todos los bytes son iguales, 8 si son uniformes
    if not datos:
        return 0.0
    n = len(datos)
    return -sum(c / n * math.log2(c / n) for c in Counter(datos).values())
```

Por qué un dump puede no funcionar

Llegar al OEP y dumpear no garantiza un ejecutable que corra. Las causas típicas, de la más común a la menos (**conocimiento externo**):

- **IAT sin reconstruir:** el dump tiene en la IAT direcciones absolutas de esta ejecución, no una tabla de imports que el loader pueda resolver.
- **Cabeceras de secciones sin ajustar:** en memoria cada sección está en su dirección virtual; si el dumper no reescribe los offsets en disco para que coincidan, el archivo queda desalineado.
- **Base distinta:** si el proceso cargó en otra base por ASLR y el dump no conserva o ajusta las relocalizaciones, las direcciones absolutas del código no coinciden.
- **Estado ya modificado:** si dumpeás después del OEP, las variables globales ya pueden tener valores de esta ejecución.
- **Código que corre antes:** TLS callbacks (2.11) o anti-dump (2.15).

Para **analizar** no siempre hace falta que el dump corra: abierto en IDA o Ghidra, con la base correcta, ya muestra el código descomprimido. Que corra es la prueba de que reconstruiste bien, no un requisito para leerlo.

> **El dump de Scylla crashea al ejecutarlo: es el ASLR, no la IAT**
>
> Al volcar un proceso que cargó con **ASLR activo**, las direcciones absolutas ya quedaron reubicadas a la base de *esa* ejecución, pero el dump conserva el flag `DYNAMIC_BASE (0x0040)` y Windows intenta re-reubicarlo con una `.reloc` ya consumida → `0xC0000005` al hacer doble clic. No reconstruiste mal la IAT: es el ASLR. Solución: en Scylla activá **Update header ImageBase** (que el `ImageBase` del dump sea la base real del volcado) y en PE-bear **desmarcá Dynamic Base** (o poné `Relocations stripped 0x0001` en File Header). Así el loader lo carga exactamente en la base para la que quedaron fijadas sus direcciones.

Práctica

En tu VM, con un **programa propio** (necesitás UPX, gratis y open source; versiones de referencia: UPX 4.x, x64dbg, Scylla):

1. Compilá un `hola.exe` propio **de 32 bits (x86)** —para que valga el patrón `popad` de arriba— y guardá su hash.

2. Comprimilo: `upx hola.exe`. Mirá en PE-bear las secciones `UPX0` / `UPX1` y cómo cambió el entry point y los imports.
3. En x64dbg (x32dbg), llegá al OEP siguiendo el `popad` / `jmp`. Anotá el OEP como **RVA** (dirección − base de imagen), no como VA absoluta.
4. Dumpeá y reconstruí la IAT (Scylla).
5. **Validá**: `upx -d hola.exe` recupera el original; comparalo con tu dump por **RVA del OEP y comportamiento** (no por la VA absoluta, que puede diferir según dónde cargue). ¿Coinciden el punto de entrada y los imports resueltos?
6. Repetí con un `hola.exe` propio **de 64 bits**: llegá al OEP con un breakpoint de memoria en ejecución sobre la región de `UPX0` y anotá qué instrucción del stub hizo el salto.
7. Calculá la entropía de cada sección del original y del comprimido (con una herramienta o con la función de arriba) y armá una tabla con las señales genéricas que muestra cada uno.

Preguntas de repaso

▼ **¿Qué son las secciones `UPX0` y `UPX1`?**

> `UPX1` trae el **stub** y los datos comprimidos; `UPX0` está **vacía en disco** pero reservada en memoria: ahí el stub **descomprime** el código original. El entry point apunta a `UPX1`.

▼ **¿Qué es el OEP y por qué lo buscás?**

> El *Original Entry Point*: la dirección donde arrancaba tu programa antes de empacar. Al llegar ahí, el código ya está **descomprimido** en memoria, listo para dumpear.

▼ **¿Cuál es el patrón clásico para detectar el salto al OEP en UPX?**

> Un **`popad`** (restaura los registros que guardó el `pushad` inicial del stub) seguido de un **`jmp` lejano** que sale de `UPX1` hacia la zona `UPX0`. Ese salto cae en el OEP.

▼ **¿Por qué hay que reconstruir la IAT después de dumpear?**

> Porque el stub resolvió los imports en runtime; el dump "crudo" no tiene una tabla de imports válida. Reconstruir la IAT (p. ej. con Scylla) hace que el `.exe` recuperado vuelva a resolver sus funciones.

▼ **En x64 no hay `popad`. ¿Cómo llegás al OEP de un UPX de 64 bits?**

> Buscando el salto a la región que estaba vacía, o con un breakpoint de memoria en ejecución sobre esa región: se dispara en la primera instrucción ejecutada ahí, que es el OEP. Ese método no depende del packer.

▼ **¿Por qué la entropía alta sola no prueba que un binario esté empaquetado?**

> Porque cualquier dato comprimido o cifrado la tiene (un instalador con recursos comprimidos, por ejemplo). Se combina con otras señales: tamaños en disco y virtual, secciones escribibles y ejecutables, pocos imports, entry point en el stub.

▼ **Llegaste al OEP y dumpeaste, pero el dump no arranca. ¿Qué revisás primero?**

> La IAT (¿se reconstruyó?), las cabeceras de secciones (¿coinciden offsets en disco y virtuales?) y la base (¿se conservaron las relocalizaciones?). Para leer el código no hace falta que arranque.

▼ **¿Por qué UPX es seguro para practicar y un protector comercial no entra en esta guía?**

> UPX es **abierto y reversible** (`upx -d` deshace el empaquetado), y se practica sobre **binarios propios**. Desempaquetar protectores comerciales apunta a vulnerar software de terceros: queda fuera del marco educativo/defensivo.

Puerta de salida

Podés: (1) reconocer un binario UPX por sus secciones/entry/imports; (2) explicar qué es el OEP; (3) llegar al OEP por el patrón `popad` / `jmp` y dumpear; (4) decir por qué hay que reconstruir la IAT; (5) validar tu resultado con `upx -d`; (6) llegar al OEP de un UPX de 64 bits sin el patrón `popad`; (7) reconocer un posible empaquetado por señales genéricas y explicar por qué una sola no alcanza; (8) diagnosticar por qué un dump no corre.

Dónde sigue esto (Bloque 2E) y límite del curso

El reconocimiento de protectores multicapa se amplía en **2.27 · Protectores comerciales y packers multicapa**, y la detección genérica del OEP mediante la regla $W \oplus X$ (página escrita y luego ejecutada) se practica con DynamoRIO/Pin en **2.29** y con Frida Stalker en **2.30**. Por el límite ético del curso, no se desempaquetan protectores comerciales sobre software ajeno.

Fuentes de esta lección

- `Reversing desde cero, cap. 13` — unpacking simple: llegar al OEP y dumpear; ejemplos sin IAT redirection/stolen; anti-debug básico (`hide`).
- `Curso IDA, tute 15` — dumpeado y reconstrucción de la IAT después del OEP.
- Curso IDA · tutes 14–18 — serie de unpacking didáctico (UPX, ASPack) con IDA y Scylla.
- Video «10 Unpacking 1 - UPX» del tronco del curso → índice de videos.
- Conocimiento externo: UPX (proyecto oficial); Microsoft Learn: PE Format (flags de sección), VirtualProtect y Memory Protection Constants.

Síntesis de estudio de esta lección

Núcleo conceptual: el método de OEP del capítulo 13 y la reconstrucción de IAT del tute 15. **Conocimiento externo:** las secciones `UPX0` / `UPX1`, el patrón `popad` / `jmp`, el método por breakpoint de memoria en x64, las señales genéricas (entropía, permisos, tamaños), las causas de un dump fallido y `upx -d`. **Recomendación editorial:** el marco de solo-UPX-didáctico-sobre-binarios-propios y los ejemplos ilustrativos (listado del stub y función de entropía; no hay un laboratorio compilado con hashes reales en esta edición).

Ampliación editorial basada en fuentes externas — Procedimiento paso a paso del breakpoint de hardware en el stack («ESP/RSP trick»)

El paso 2 de esta lección menciona el breakpoint de hardware sobre el stack. Su fundamento combina la propiedad LIFO de la pila con los registros `DR0–DR3` (*Practical Malware Analysis*, Cap. 18; Intel SDM Vol. 3B §17.2) *(Hecho confirmado)*:

1. En el inicio del stub de UPX, ejecutá **una sola vez** `F8` sobre el `pushad` inicial (que guarda los 8 registros en el stack).
2. En el panel de registros de x64dbg, hacé clic derecho en el nuevo valor de `ESP` → *Follow in Dump*.
3. Seleccioná los primeros 4 bytes en el *Dump*, clic derecho → *Breakpoint* → *Hardware, Access* → *Dword*.
4. Presioná `F9` **(Run)**: cuando el stub termine de descomprimir y ejecute `popad` para restaurar esos mismos 4 bytes, el procesador disparará la interrupción de depuración y se detendrá exactamente sobre el `jmp OEP`.